

手势识别实验手册

文档版本 01
发布日期 2024-07-05



版权所有 © 华为云计算技术有限公司 2023。 保留一切权利。

非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明



HUAWEI和其他华为商标均为华为技术有限公司的商标。
本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受华为云计算技术有限公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，华为云计算技术有限公司对本文档内容不做任何明示或暗示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导，本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

手势识别

第一章 实验准备

1.1. 实验描述

本实验的目标是使用 YOLOv5 模型对各种手势进行识别，机器狗根据手势做出相应动作。YOLOv5 是革命性的 “You Only Look Once” 物体检测模型的第五次迭代，旨在实时提供高速、高精度的结果。在实验过程中，我们收集大量手势数据集，对预训练权重进行模型训练和调优，并最终转换模型格式，将其在昇腾开发板上进行推理。通过海康工业相机和华为昇腾开发板结合，实现实时手势检测。再进行不同的手势，完成机器狗相应动作。

1.2. 实验目的

- 了解神经网络的基本原理
- 了解实现机器狗手势识别的原理和方法
- 掌握数据集的标注方法
- 了解 onnx 模型的简单分析方法
- 掌握如何将 onnx 模型转换为 om 模型的方法
- 学会如何将模型应用于机器狗上
- 如何使用华为云 modelarts 进行模型训练

1.3. 实验建议

本实验主要是在 YOLOv5 上用 Python 编程，模型转换过程可以在自己的 PC 上进行，

程序执行则需要 在华为昇腾 Atlas 200I DK 开发板 (Linux) 上执行。实验中未注明的执行地方的, 一律在开发板上执行。

前置基础课:

- (1) Python 数据分析
- (2) 机器学习
- (3) 数据集标注技术

1.4. 实验环境

本次实验使用的硬件包括: PC 机、昇腾 Atlas 200I DK 开发板、摄像头。实验开始前, 请按照如下步骤将各硬件连接:

- (4) 使用数据线将开发板连接至 PC 机;
- (5) 将摄像头连接至开发板;
- (6) 给开发版供电, 确保其能正常开机;

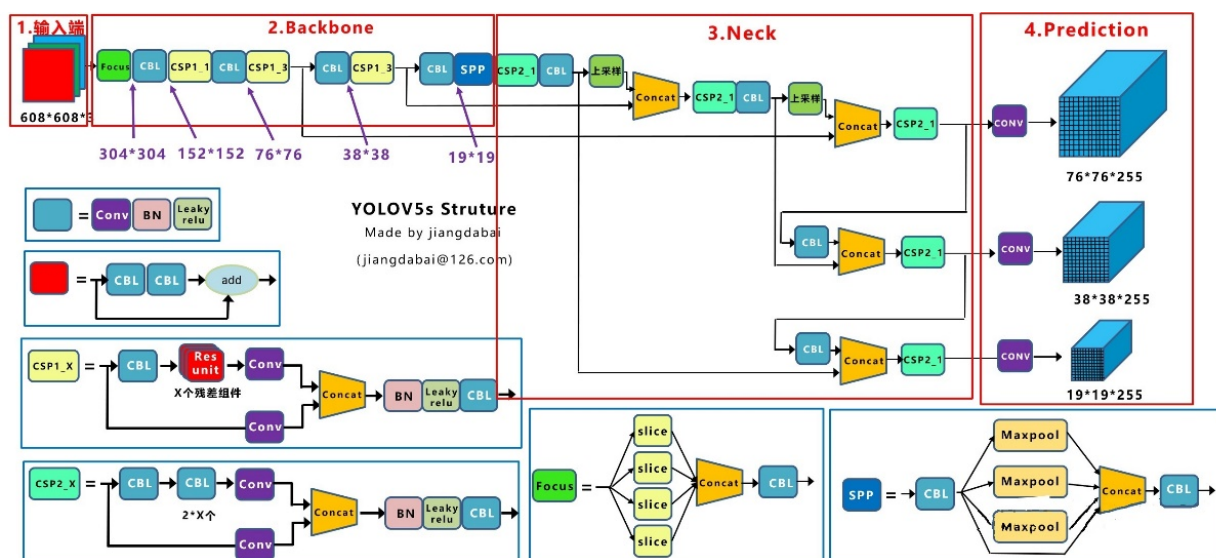
1.5. 背景知识 1 YOLOv5 介绍

YOLOv5 是一个计算机视觉模式设计的对象检测任务, 它是 YOLO 系列的升级版, YOLO 系列以其实时物体检测能力而著称。



如上图所示，基于 YOLOv5 的运动检测，检测出了 person、tennis racket、chair 等等，检测结果相对准确。

YOLOv5 网络结构：



上图即 YOLOv5 的网络结构图，可以看出其分为输入端、Backbone、Neck、Prediction 四个部分。

Backbone: 主干网络，其大多时候指的是提取特征的网络。主干网络的作用就是提取图片中的信息，供后面的网络使用。经常使用的网络是 resnet、VGG 等（非自己设计），这些网络在分类等问题上的特征提取能力很强。

neck:其作用是更好地融合/提取 **backbone** 给出的 **feature**，从而提高网络的性能。

基本组件：

- CBL：由 Conv+Bn+Leaky_relu 激活函数三者组成。
- Res unit：借鉴 Resnet 网络中的残差结构，让网络可以构建的更深。
- CSPX：借鉴 CSPNet 网络结构，由卷积层和 X 个 Res unit 模块 Concat 组成。
- SPP：采用 1×1 ， 5×5 ， 9×9 ， 13×13 的最大池化的方式，进行多尺度融合。
- 其他基础操作：
 - Concat：张量拼接，会扩充两个张量的维度，例如 $26\times 26\times 256$ 和 $26\times 26\times 512$ 两个张量拼接，结果是 $26\times 26\times 768$ 。
 - add：张量相加，张量直接相加，不会扩充维度，例如 $104\times 104\times 128$ 和 $104\times 104\times 128$ 相加，结果还是 $104\times 104\times 128$ 。

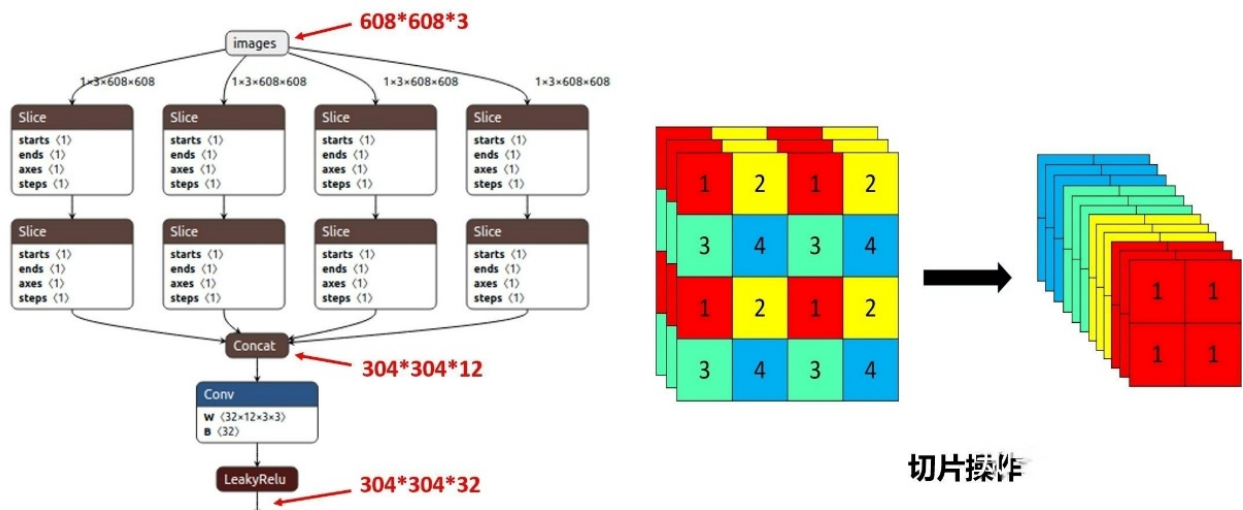
输入端：

- 1) 输入预处理：将输入图像进行缩放、裁剪和归一化等预处理操作，以便于模型的输入。

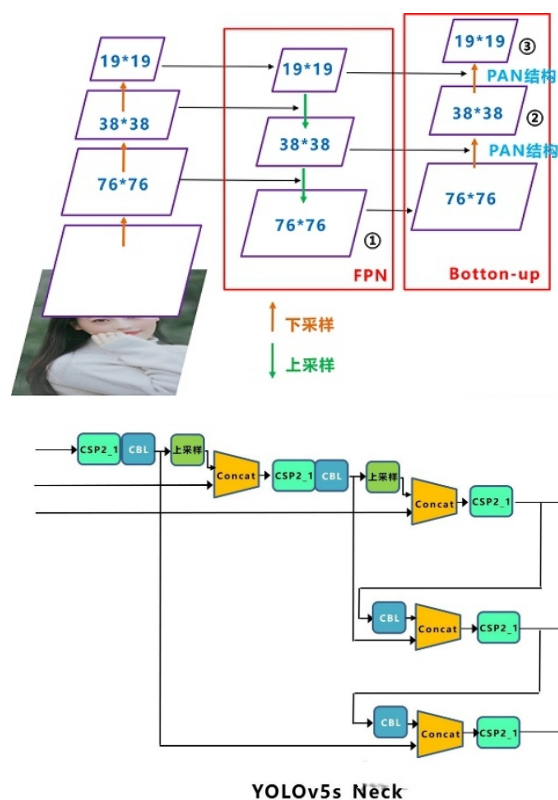


Yolov5 的代码中 datasets.py 的 letterbox 函数中进行了对原始图像自适应的添加最少的黑边。

- 2) 特征提取：将预处理后的图像输入到特征提取网络中，提取出图像中的特征信息。



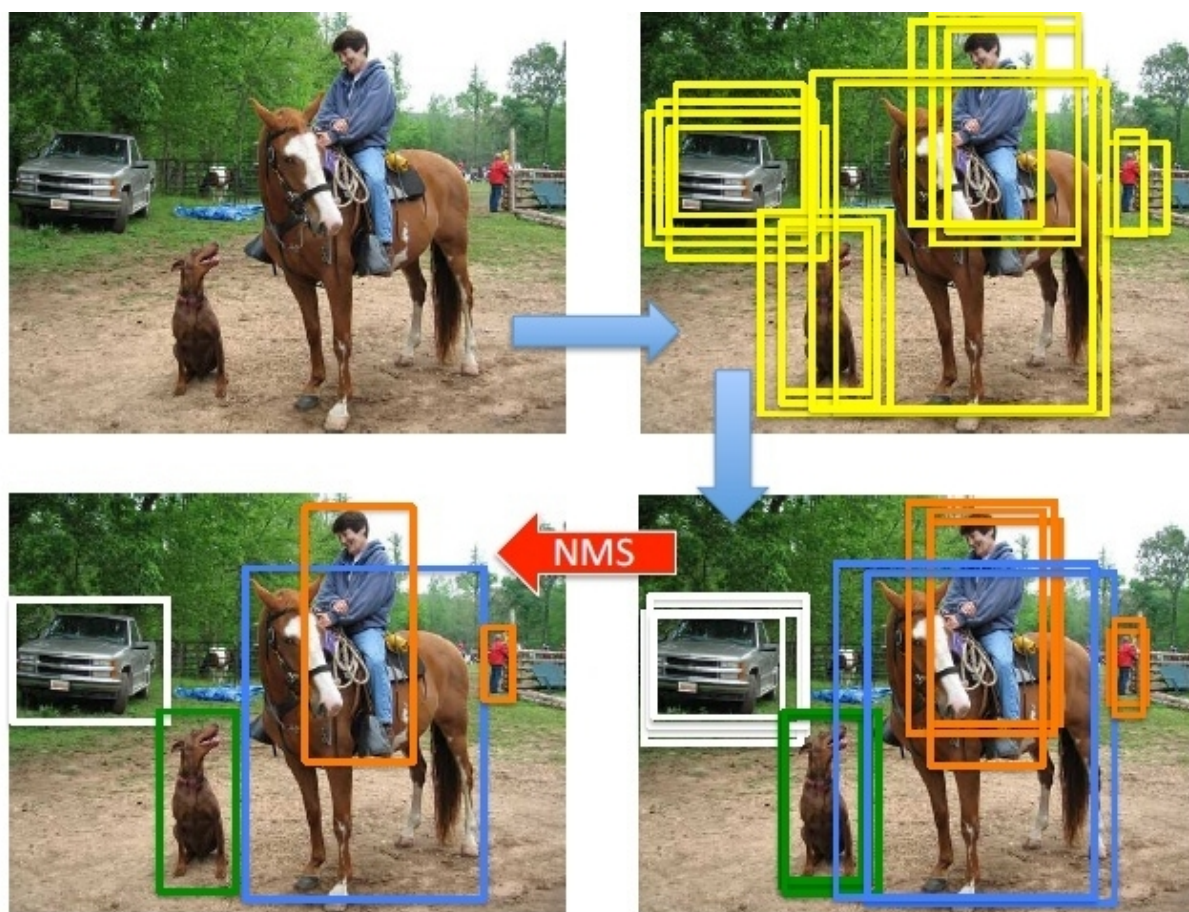
- 3) 特征融合：将特征提取网络中的多个不同层次的特征图进行融合，以得到更全面和丰富的特征信息。



Yolov5 的 Neck 结构中，采用 FPN+PAN 的结构，并且采用借鉴 CSPnet 设计的 CSP2 结构，加强网络特征融合的能力。

- 4) 检测预测：在特征融合的基础上，使用预测头对目标进行检测和分类，同时预测目标的边界框和置信度等信息。

- 5) 后处理：对检测结果进行非极大值抑制（NMS）等后处理操作，以过滤重叠的边界框、去除置信度低的预测和选择最终的检测结果。



1.6. 背景知识 2 labelImg 介绍

labelImg 是一款开源的图像标注工具，它主要用于标注目标检测、语义分割和图像分类等深度学习中需要的数据集。通过使用 labelImg，用户可以快速、准确地为图片中的目标添加标注信息，从而建立数据集。

使用步骤如下：

- (1) 下载LabelImg: 用户可以从<https://github.com/tzutalin/labelImg> 上下载最新版的 LabelImg 代码。
- (2) 安装 Python: LabelImg 是一个基于 Python 开发的工具，因此需要先安装 Python。
- (3) 安装依赖库: 在安装完 Python 后，还需要安装一些必要的依赖库，比如 PyQt5、lxml 等。可以使用 `pip install` 命令进行安装。
- (4) 运行 LabelImg: 安装完所有依赖库后，就可以运行 LabelImg。为了方便使用，可以将

LabelImg 代码中的 labelImg.py 文件添加到环境变量中，从而可以直接在终端中运行。

- (5) 导入图片：在 LabelImg 的界面中，用户可以通过点击 Open Dir 按钮或者使用 File > Open Dir 命令来导入需要标注的图片。导入的图片可以轻松地在界面中查看和编辑。
- (6) 增加标注：在选中需要标注的图片后，用户可以选择 Draw Box 或者 Draw Polygon 等工具来添加标注。用户可以自由选择标注的颜色、大小和形状等选项，以便更好地区分不同的标注。
- (7) 保存标注：在完成对图片的标注后，可以使用 Save 按钮来保存标注信息。LabelImg 支持多种常见的标注格式，如 VOC 标签格式，保存为 xml 文件；yolo 标签格式，保存为 txt 文件；createML 标签格式，保存为 json 格式。

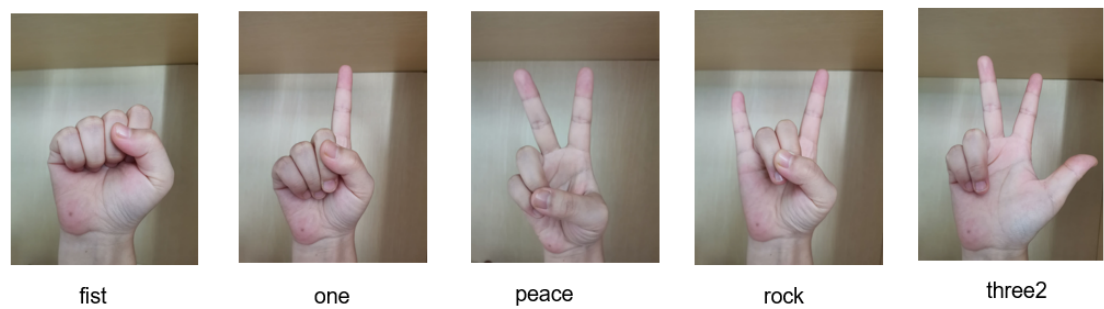
用途：

- (1) 数据集制作：LabelImg 可以帮助用户快速制作目标检测、语义分割和图像分类等深度学习中需要的数据集。
- (2) 模型训练：通过标注的数据集，用户可以训练出精度更高的深度学习模型。
- (3) 图像处理：LabelImg 还可以用于图像处理，例如在对图片进行编辑或者后期处理的时候，可以先用 LabelImg 对图片进行标注，以便更加准确地处理。

注意事项：

- 在使用 LabelImg 时，需要保证数据集的质量。避免标注错误、标注缺失等情况的发生，从而提高深度学习模型的精度。
- 在标注过程中，需要注意保持标注的一致性。例如，对于同一类别的目标，应该保持标注的形状和大小等属性的一致性。
- 在导入图片时，需要注意图片的格式和质量。避免因为图片格式不支持或者图片质量不佳等问题导致标注失败。
- 在标注过程中，需要保证标注的准确性。避免因为标注错误或者不完整等问题导致模型训练失败。
- 保存时最好选择非中文字符文件夹进行保存

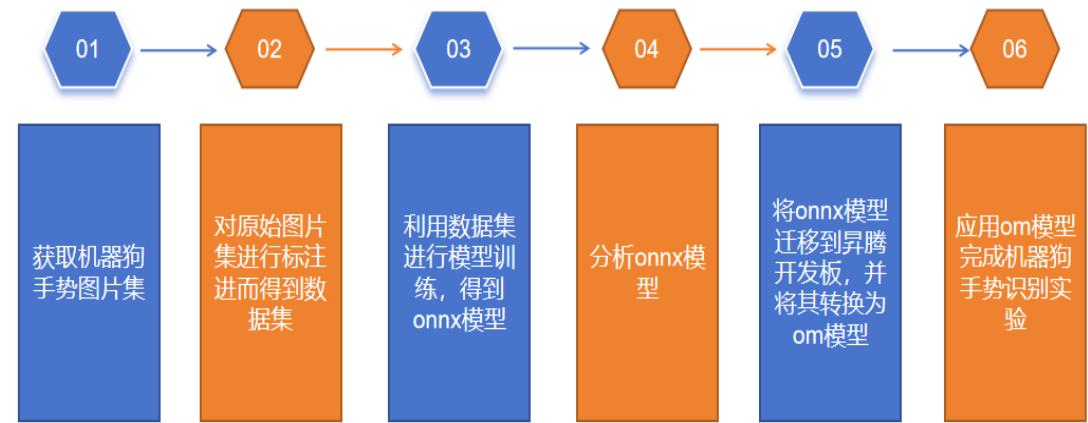
1. 7. 实验道具讲解



当机器狗摄像头识别到 fist 时，机器狗会执行“趴下”动作；
当机器狗摄像头识别到 one 时，机器狗会执行“扭身跳”动作；
当机器狗摄像头识别到 peace 时，机器狗会执行“扭身体”动作；
当机器狗摄像头识别到 rock 时，机器狗会执行“太空步”动作；
当机器狗摄像头识别到 three2 时，机器狗会执行“打招呼”动作。

第二章 实验过程描述

本实验依赖于基于 yolo 的目标检测，然后根据检测的内容做出相应的动作。



第三章 环境准备

3.1. 硬件环境准备

(1) SD 卡中系统烧录

向 SD 卡中烧录镜像的步骤如下。

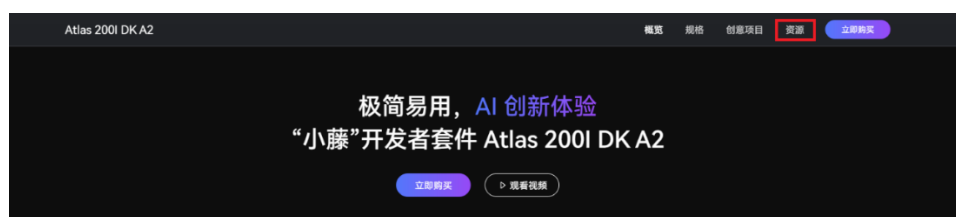
- ① 将 SD 卡插入读卡器，再将读卡器插到 PC 上。



- ② 在浏览器中进入 [Ascend 官网](#)，选择“产品”->“开发者套件”



- ③ 点击“资源”，进入资源下载界面



- ④ 选择“制卡工具下载”，下载完成后进行安装。

寻你所需，玩转AI开发

快速入门

准备

请提前准备好下列物品，或查看[硬件准备清单](#)，快速上手不卡顿

● MicroSD卡（推荐64GB）及读卡器（可选）

● 获取制卡工具，一键烧录镜像完成开发环境准备

制卡工具下载

校验文件下载

入门

提供入门 Atlas 200I DK A2 的全流程指导，帮助你30分钟内轻松完成从启动开发套件到运行首样例的过程

快速开始（课程）

快速开始（文档）

参考工具

提供Atlas 200I DK A2 进一步开发所需的参考工具及配套资源，可按需下载

模型适配工具

使能开发者0代码完成数据标注、模型训练、模型部署到昇腾开发者套件

软件包下载

校验文件下载

配套Conda环境包

模型适配工具运行所必须的配套conda依赖环境，需要提前安装conda并导入conda环境包

软件包下载

校验文件下载

⑤ 打开镜像烧录工具 Ascend AI Devkit Imager，选择 Altas 200I DK A2

⑥ 打开“配置网络信息”，选择“Type-C”选项卡。可以看到 Type-C 接口默认 IP 为 192. 168. 0. 2。选择默认 IP。

Error! Unknown document property name.

镜像网络信息配置

以下为镜像默认IP信息，您可以根据PC/路由器IP信息重新填写网络信息

网口信息 ETH0 ETH1 **Type-C**

☐ 自动获取IP地址 ☒ 使用下面的IP地址

IP地址 192 168 0 2

子网掩码 255 255 255 0

默认网关 - - -

首选DNS服务器 8 8 8 8

备用DNS服务器 114 114 114 114

确定 取消

⑦ 选择要烧录的 SD 卡

昇腾开发者套件—一键制卡工具

1 选择操作 2 选择SD卡 3 烧录镜像

在线制卡 本地制卡 备份SD卡

镜像版本: Desktop-Image_23.0.RC3_ubuntu22.04

Atlas 200I DK A2

配置网络信息

镜像包含OS、NPU驱动固件、CANN、MindX SDK、图形桌面、远程桌面、USB Audio...

OS版本: Ubuntu 22.04 LTS Arm64

固件与驱动版本: 23.0.RC3

CANN版本: 7.0.RC1

发布时间: 2023-11-15

保存在本地磁盘里

镜像版本: Base-Image_23.0.RC1_ubuntu18.04

Atlas 200 DK

Atlas 200DK镜像,预装Python、CANN、MindX SDK、SSH、Samba、VNC、XRD...

OS版本: Ubuntu 18.04 LTS Arm64

固件与驱动版本: 1.0.13.alpha

CANN版本: 5.1.RC2.alpha007

发布时间: 2022-11-28

需要下载: 8.7 GB

SD卡大小 62.5 GB

Mass Storage Device USB Device

Mass Storage Device USB Device-62.5 GB

挂载到: F:\, H:\, I:\

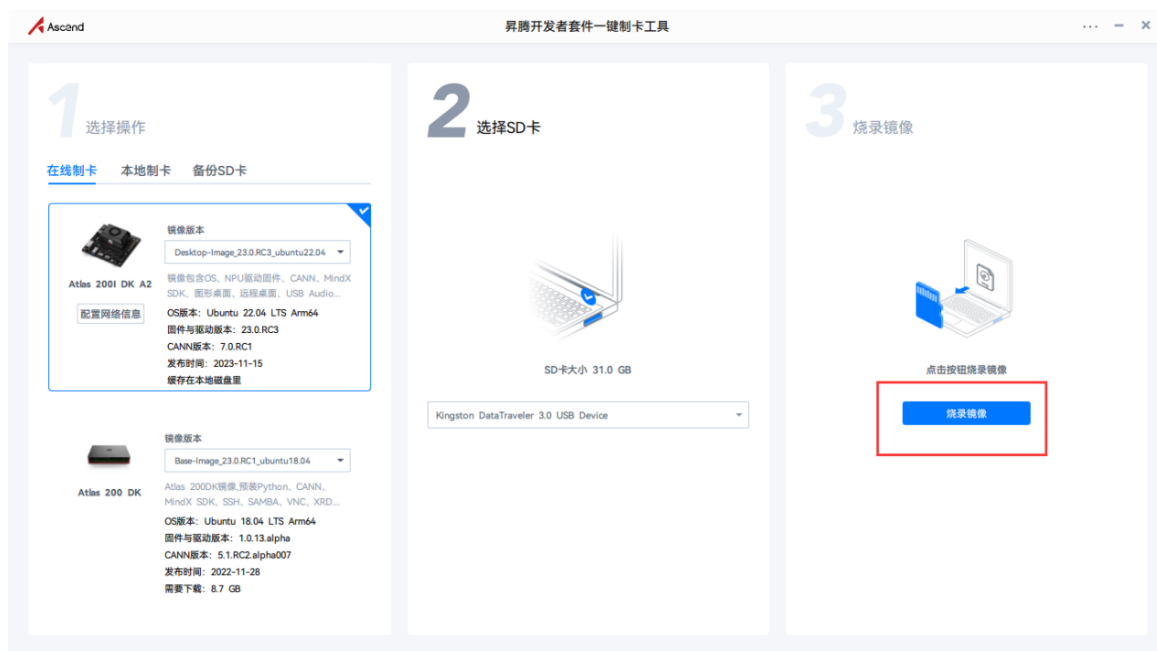
Kingston DataTraveler 3.0 USB Device-31.0 GB

挂载到: G:\

点击按钮烧录镜像

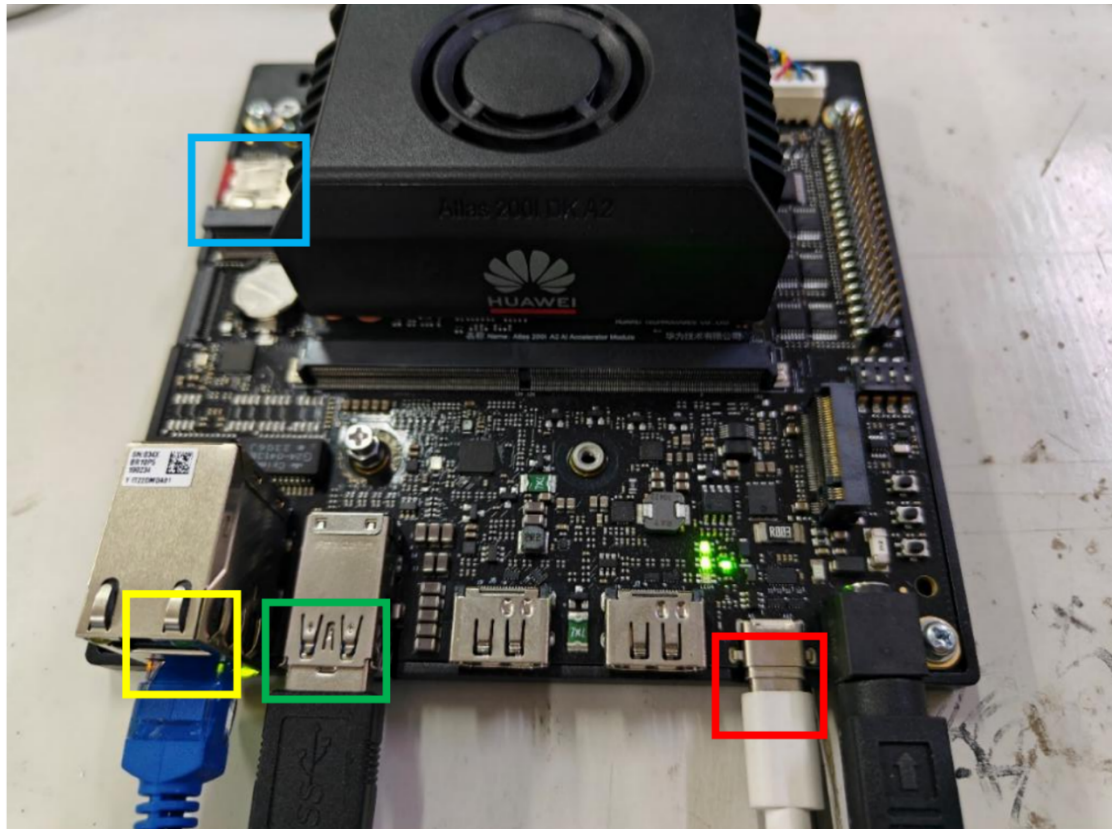
烧录镜像

⑧ 点击“烧录镜像”，等待镜像烧录完成。



(2) 开发板连接

将烧录好系统的 SD 卡插入到开发板中（下方蓝色框中），昇腾开发板和 PC 机通过 Type-C 线连接，将 Type-C 线和电源线连接到开发板（下方红色框中），并将海康工业相机和机器狗的 USB 线连接到开发板（下方绿色框中）。同时可以连接网线（下方黄色框中），让开发版连接互联网，方便环境依赖包的下载。连接方式如下图所示。



(3) PC 端远程连接开发板

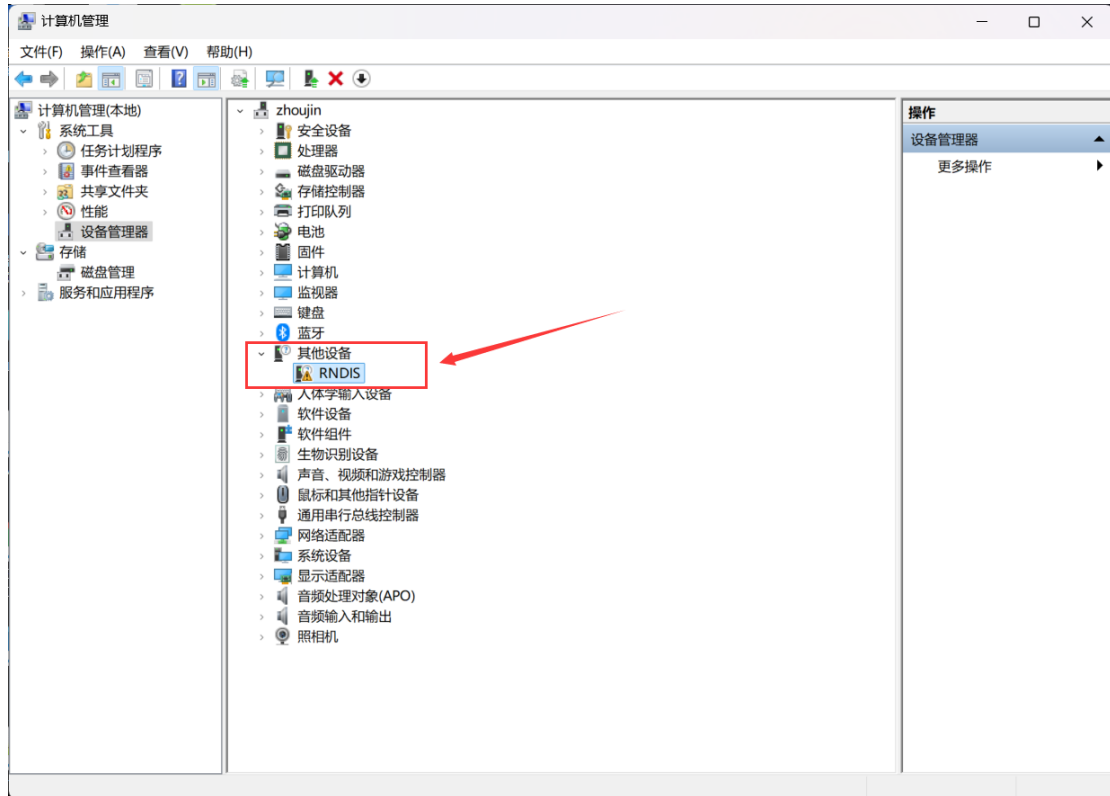
本节介绍当开发者套件通过 Type-C 接口和 PC 连接时，如何在 PC 将接口 IP 地址修改为和开发者套件接口 IP 地址同一个网段(如开发者套件 Type-C 接口为 192.168.0.2, PC 对应 USB 或 Type-C 接口为 192.168.0.101)，并使用 SSH 工具远程登录开发者套件。

1) 安装 Windows 的 USB 网卡驱动

本步骤以 Windows 10 系统为例。

①在 PC 上右键单击“此电脑”，选择“更多”→“管理”，进入“计算机管理”界面。

②在“计算机管理”操作界面中选择“设备管理器→“其他设备”，如下图所示，RNDIS 为未识别状态。



注意：如果未出现 RNDIS，请按以下方法排查：

确保开发者套件 Type-C 接口和 PC 已通过连接线正常连接。

更换具备数据传输能力的 Type-C 数据线（一般手机充电线拥有数据传输能力），继续查看是否出现 RNDIS。

如果尝试现场所有 Type-C 数据线，仍无法出现 RNDIS，则考虑使用 RJ45 网线连接 PC 和开发者套件的以太网口的方式登录开发者套件。

③右键单击“RNDIS”，选择“更新驱动程序(P)”。

④在弹出的“更新驱动程序”→“RNDIS 窗口”中选择“浏览我的计算机以查找驱动程序软件(R)”。



⑤然后选择“让我从计算机上的可用驱动程序列表中选择(L)”。



⑥在“常见硬件类型”列表中选择“网络适配器”，单击“下一页(N)”。

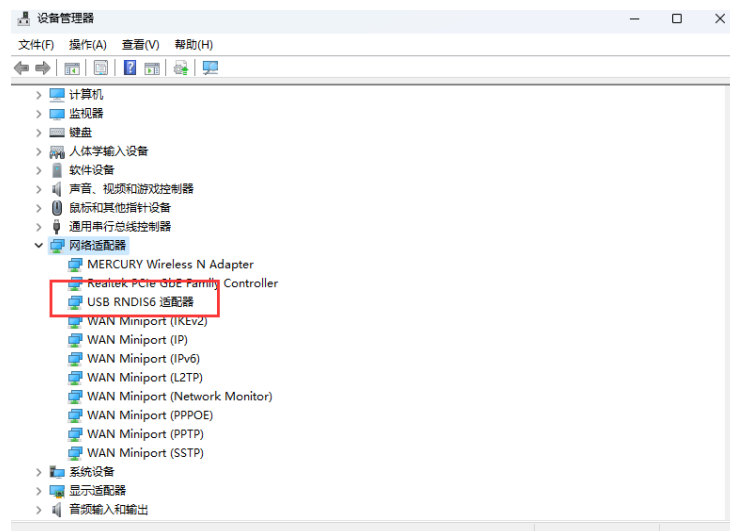


⑦在“选择要为此硬件安装的设备驱动程序”界面中选择“Microsoft”厂商的“USB RNDIS6 适配器”。



⑧单击“下一页”，在弹出的“更新驱动程序警告”窗口选择“是”。

⑨返回“设备管理器”→“网络适配器”，可看到已经正常显示了 USB RNDIS6 适配器的驱动。



(4) 配置 PC 接口 IP 地址

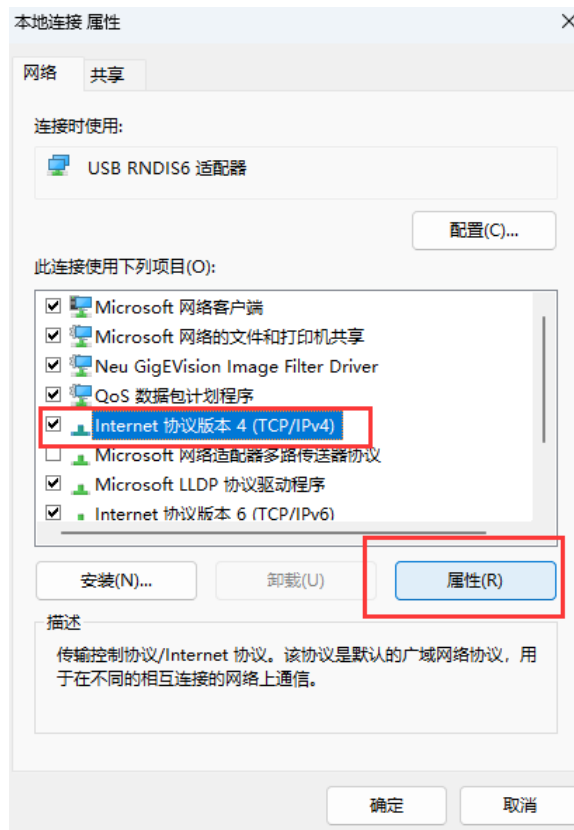
本步骤以 Windows 10 系统为例。

①在 PC 上打开“开始”，单击“设置”按钮，进入“Windows 设置”界面。

②选择“网络和 Internet”，单击“更改适配器选项”。

③鼠标右键单击“本地连接”后鼠标左键单击“属性”进入“本地连接 属性”界面（使用 Type-C 接口连接时一般为“本地连接 x”，x 为数字，以实际显示的数字为准）。

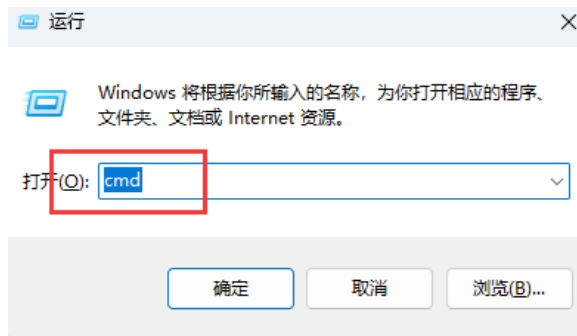
④选择“Internet 协议版本 4（TCP/IPv4）”，单击“属性”。



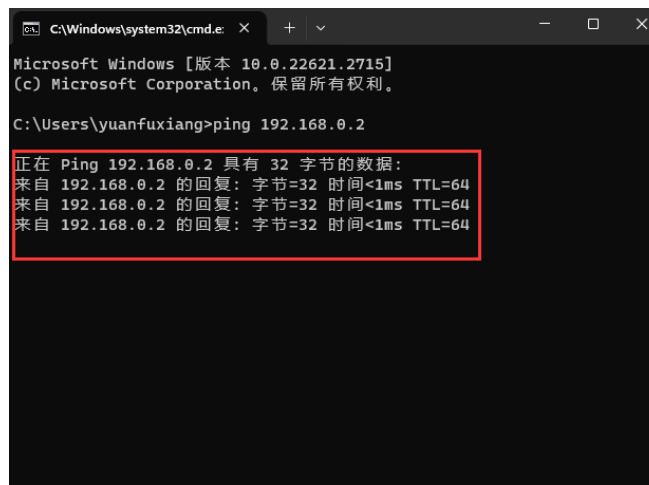
⑤勾选“使用下面的 IP 地址”选项，填写 IP 地址（图示以 192.168.0.101 为例）和子网掩码，默认网关与 DNS 服务器地址为空，单击“确定”保存。



⑥使用快捷键“Win+R”，在运行窗口输入 cmd 进入命令行窗口。



⑦输入 ping 192.168.0.2 命令，查看是否能够访问开发板 IP。

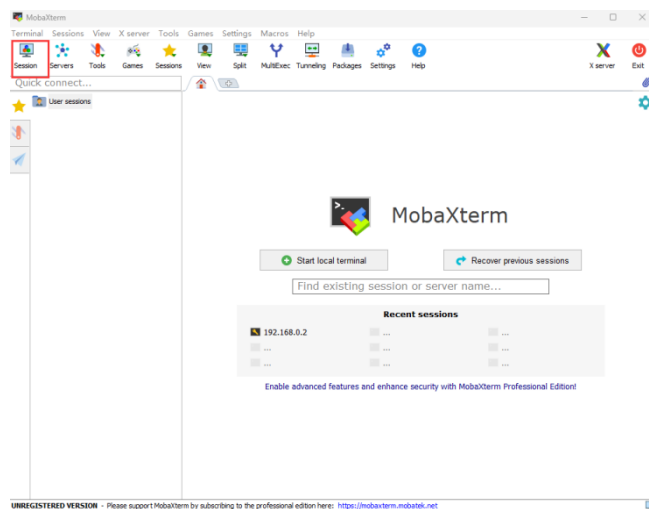


(5) 使用 ssh 远程登录开发板

本文使用 MobaXterm 工具在 PC 上远程登录开发板，单击[下载链接](#)获取 MobaXterm 软件压缩包，解压获得“MobaXterm_Personal_22.2.exe”。

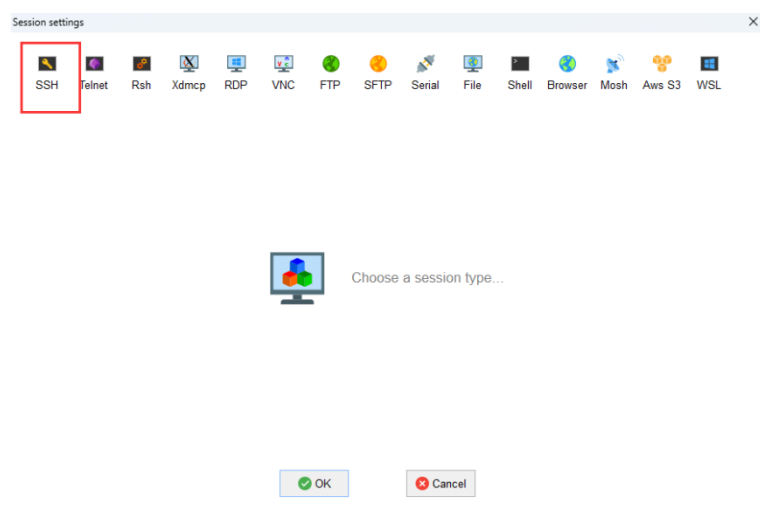
①双击 “MobaXterm_Personal_22.2.exe” 启动 SSH 登录工具。

②单击左上方的 “Session” 进入界面。

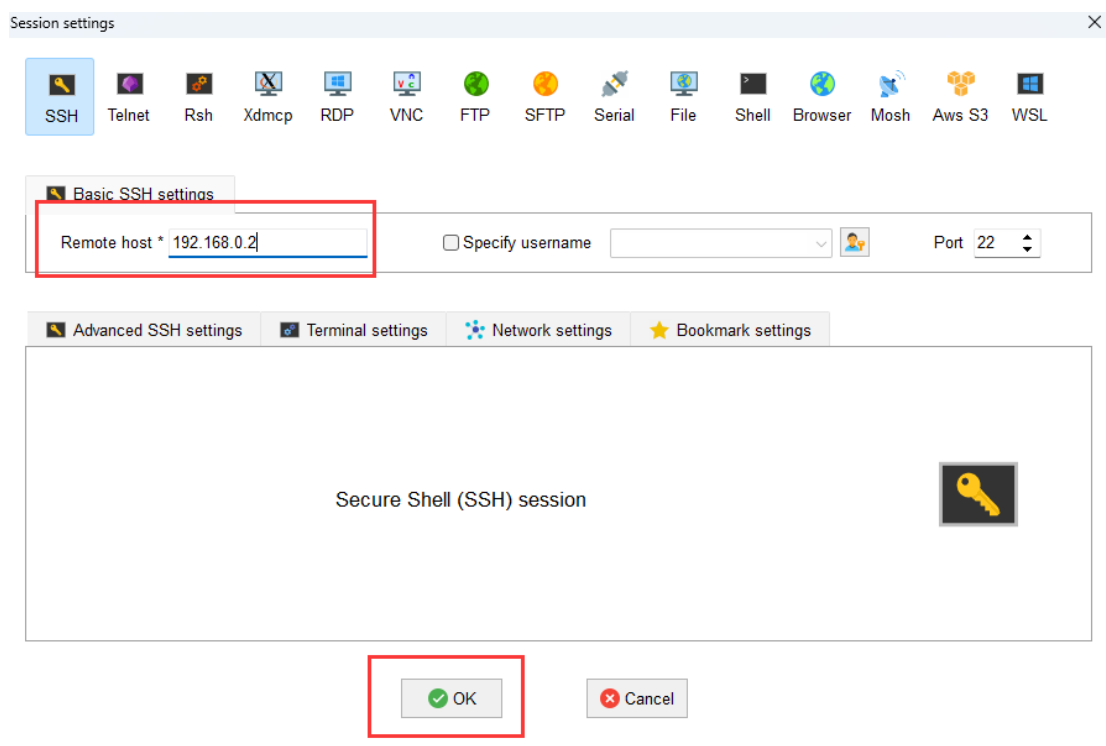


Error! Unknown document property name.

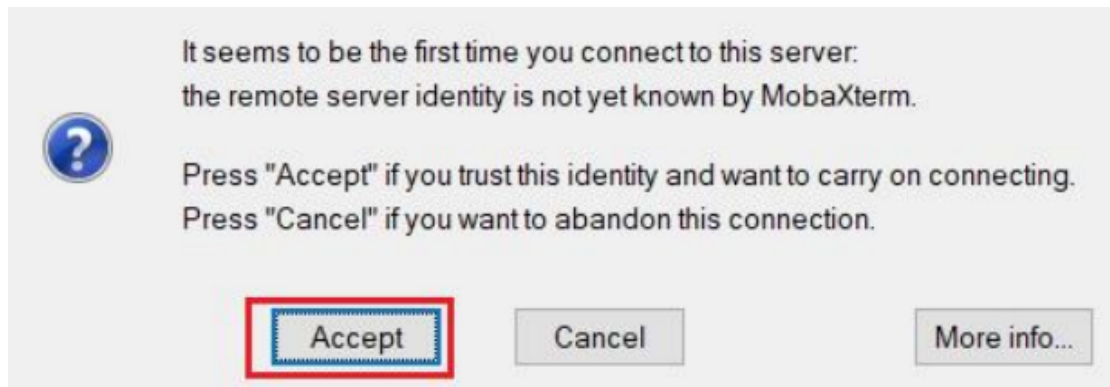
③单击左上方的“SSH”进入 SSH 连接配置界面



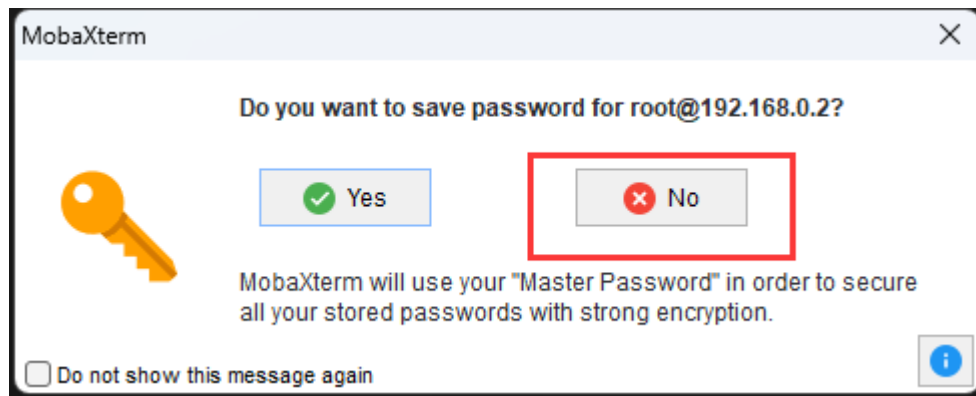
④根据硬件连接方式填写开发者套件连接 PC 的接口实际 IP 地址（一键制卡中配置的 IP 地址）。图示以 192.168.0.2 为例，请根据实际修改。



⑤单击“OK”按钮，首次连接开发者套件时，SSH 工具提示是否信任连接的设备，单击“Accept”。



⑥进入远程登录界面后，输入 `root` 用户名登录密码（默认为 `Mind@123`）登录开发者 套件，请修改默认密码，并妥善保管新密码。SSH 工具界面会出现保存密码提示，可以单击“No”，不保存密码直接登录开发者套件。



⑦远程登录开发者套件成功界面如图所示。

```
login as: root
root@192.168.0.2's password:

• MobaXterm Personal Edition v22.2 •
  (SSH client, X server and network tools)

► SSH session to root@192.168.0.2
  • Direct SSH      : ✓
  • SSH compression : ✓
  • SSH-browser     : ✓
  • X11-forwarding  : ✓ (remote display is forwarded through SSH)
  ► For more info, ctrl+click on help or visit our website.

Ascend=devkit

Welcome to Atlas 200I DK A2
This system is based on Ubuntu 22.04 LTS (GNU/Linux 5.10.0+ aarch64)

This system is only applicable to individual developers and cannot be used for commercial purposes.

By using this system, you have agreed to the Huawei Software License Agreement.
Please refer to the agreement for details on https://www.hiascend.com/software/protocol

Reference resources
* Home page: https://www.hiascend.com/hardware/developer-kit-a2
* Documentation: https://www.hiascend.com/hardware/developer-kit-a2/resource
* Online courses: https://www.hiascend.com/edu/courses
* Online experiments: https://www.hiascend.com/zh/edu/experiment
* Forum: https://www.hiascend.com/forum/
* Code: https://gitee.com/HUAWEI-ASCEND/ascend-devkit

Last login: Sat Dec  2 17:13:19 2023 from 192.168.0.101
(base) root@davinci-mini:~#
```

4) 机器狗中继连接外网

注：本教程主要为暂时没有外置无线网卡的用户提供连接外网方案，教程以中继手机热点为例进行说明，中继无线路由器同理。

1、电脑连接机器狗的 wifi，点击电脑右下角的 wifi 图标，点击属性查看 IPv4 DNS 服务器的 ip 地址。如下图所示。



2、打开电脑浏览器，输入 IPv4 DNS 服务器的 ip，默认登录用户和密码均为 **admin**。

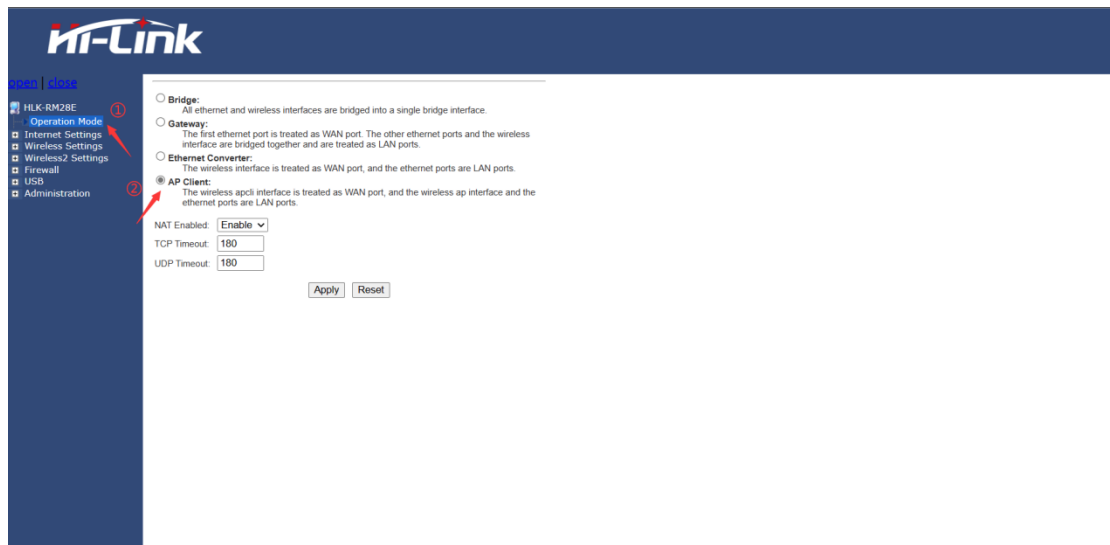
如下图所示。



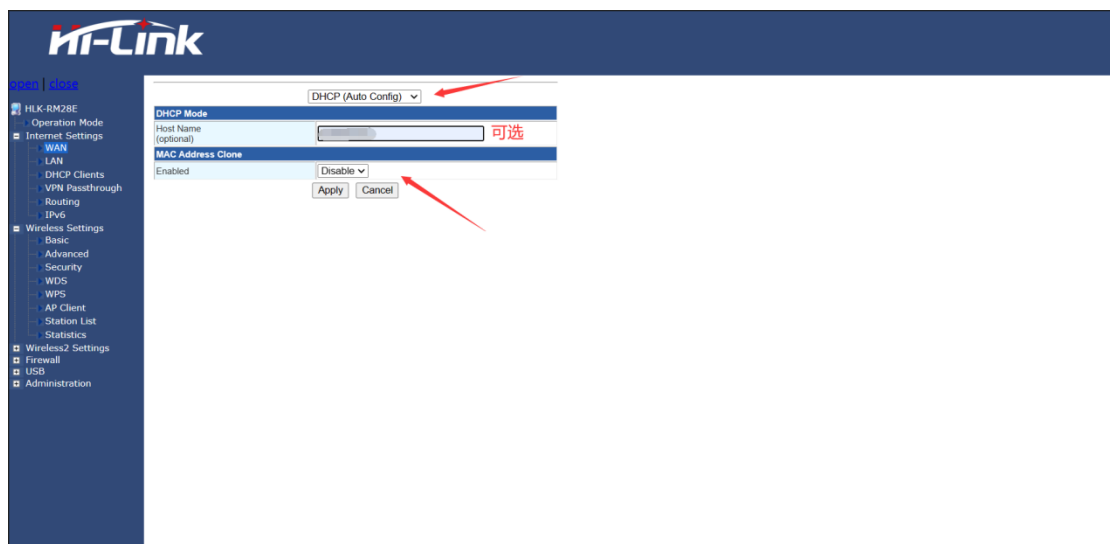
3、打开手机热点。

4、进入 Hi-link 设置页面，点击 Operation Mode，选择 AP Client，最后点击 Apply。

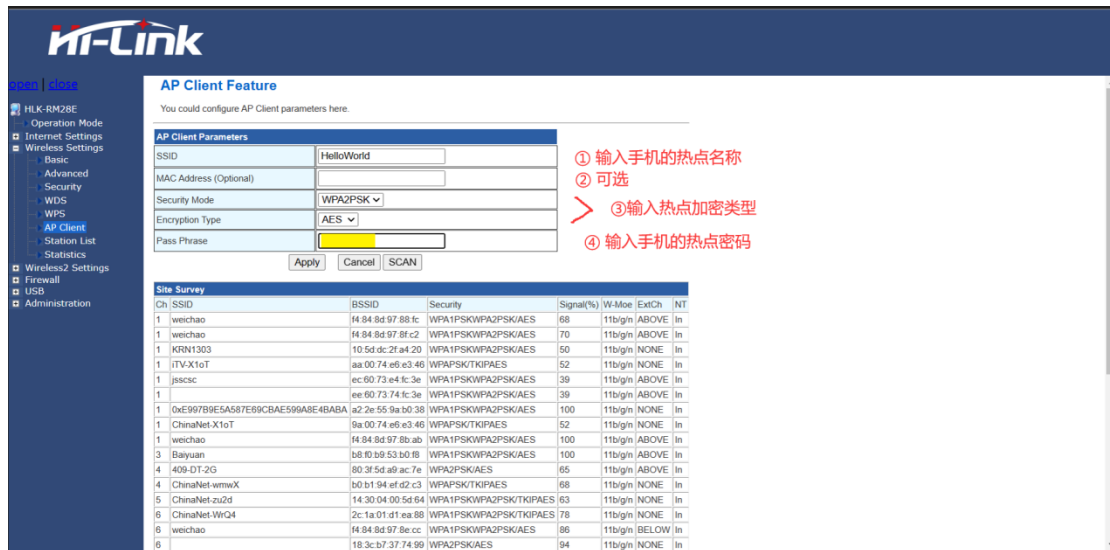
如下图所示。



5、展开 Internet Settings，点击 WAN，参照下图进行设置，最后点击 Apply。



6、展开 Wireless Settings，点击 AP Client，查看 site Survey 表中是否有自己的手机热点名称，如若没有点击 SCAN 进行扫描，在 AP client Parameters 表中参照下图进行设置，最后点击 Apply。



7、重启机器狗，注意中继时热点需一直处于开启状态，电脑连接机器狗的 wifi，使用远程软件登录，此时机器狗处于连接外网状态。测试是否有网络，点击浏览器，搜索任意内容，如“百度学术”，能正常显示下图说明正常。



3.2. 软件环境（YOLO 训练）

首先安装软件 YOLOv5 步骤如下：

（1）激活虚拟环境

```
(base) C:\Users\lenovo>conda activate Lab2
(Lab2) C:\Users\lenovo>
```

Error! Unknown document property name.

```
1.conda activate Lab2
```

(2) 进入工程目录下，安装依赖

```
18 个文件      309,786 字节
9 个目录 500,259,495,936 可用字节

(Lab2) E:\Workspace\Project\yolov5>pip install -U -r requirements.txt|
```

```
1.pip install -r requirments.txt
```

(3) 验证安装成功

```
Successfully installed MarkupSafe-2.1.3 Pillow-10.0.0 PyYAML-6.0.1 absl-py-1.4.0 asttokens-2.2.1 backcall-0.2.0 cachetoc
ls-5.3.1 certifi-2023.7.22 charset-normalizer-3.2.0 colorama-0.4.6 contourpy-1.1.0 cycler-0.11.0 decorator-5.1.1 executi
ng-1.2.0 filelock-3.12.2 fonttools-4.41.1 gitdb-4.0.10 gitpython-3.1.32 google-auth-2.22.0 google-auth-oauthlib-1.0.0 gr
pcio-1.56.2 idna-3.4 importlib-metadata-6.8.0 importlib-resources-6.0.0 ipython-8.12.2 jedi-0.18.2 jinja2-3.1.2 kiwisolv
er-1.4.4 markdown-3.4.4 matplotlib-3.7.2 matplotlib-inline-0.1.6 mpmath-1.3.0 networkx-3.1 numpy-1.24.4 oauthlib-3.2.2 o
pencv-python-4.8.0.74 packaging-23.1 pandas-2.0.3 parso-0.8.3 pickleshare-0.7.5 prompt-toolkit-3.0.39 protobuf-4.23.4 ps
util-5.9.5 pure-eval-0.2.2 pyasn1-0.5.0 pyasn1-modules-0.3.0 pygments-2.15.1 pyparsing-3.0.9 python-dateutil-2.8.2 pytz-
2023.3 requests-2.31.0 requests-oauthlib-1.3.1 rsa-4.9 scipy-1.10.1 seaborn-0.12.2 six-1.16.0 smmap-5.0.0 stack-data-0.6
.2 sympy-1.12 tensorboard-2.13.0 tensorboard-data-server-0.7.1 thop-0.1.1.post2209072238 torch-2.0.1 torchvision-0.15.2
tqdm-4.65.0 traitlets-5.9.0 typing-extensions-4.7.1 tzdata-2023.3 urllib3-1.26.16 wcwidth-0.2.6 werkzeug-2.3.6 zipp-3.16
.2
```

注意：PyTorch 可能被替换为 CPU 版，训练速度很慢

配置虚拟环境如下：

(1) 新建一个脚本查看是否为 gpu 版本

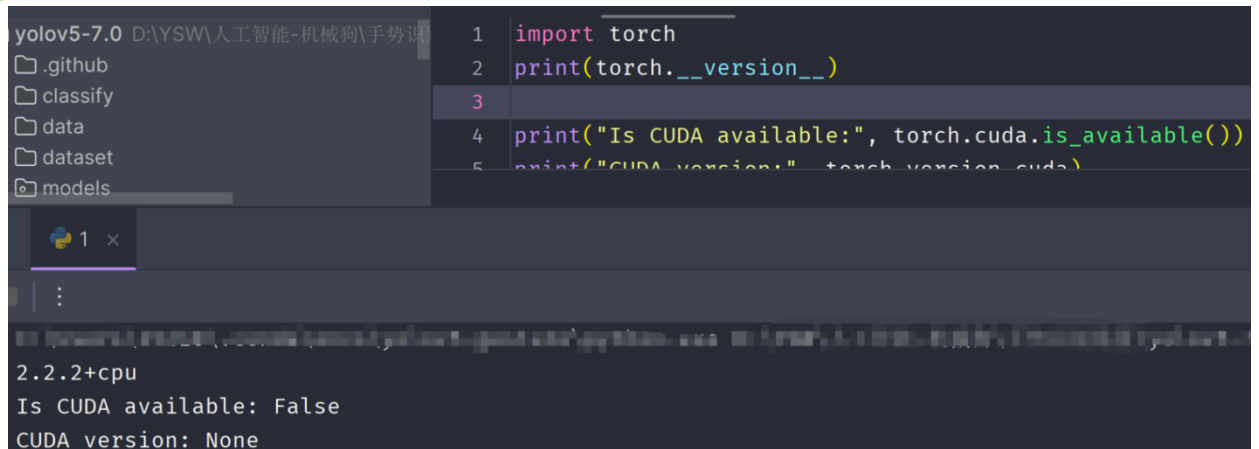
```
import torch
print(torch.__version__)
print(torch.cuda.is_available())

C:\Users\15210\.conda\envs\yolov5-
2.2.2+cu121
True
```

```
1. import torch
```

2. `print(torch.__version__)`

3. `print(torch.cuda.is_available())`



```
1 import torch
2 print(torch.__version__)
3
4 print("Is CUDA available:", torch.cuda.is_available())
5 print("CUDA version:", torch.version.cuda)
```

2.2.2+cpu
Is CUDA available: False
CUDA version: None

1. `import torch`

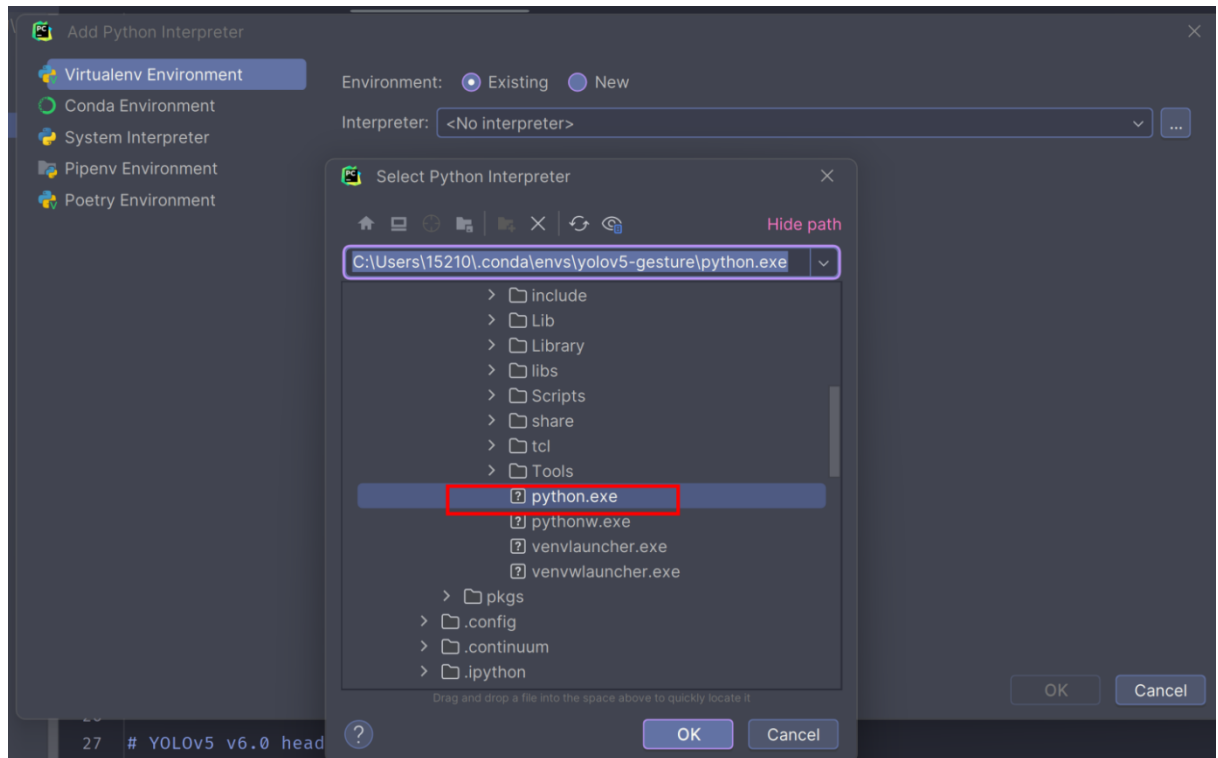
2. `print(torch.__version__)`

3.

4. `print("Is CUDA available", torch.cuda.is_available())`

5. `print("CUDA version: ",torch.version.cuda )`

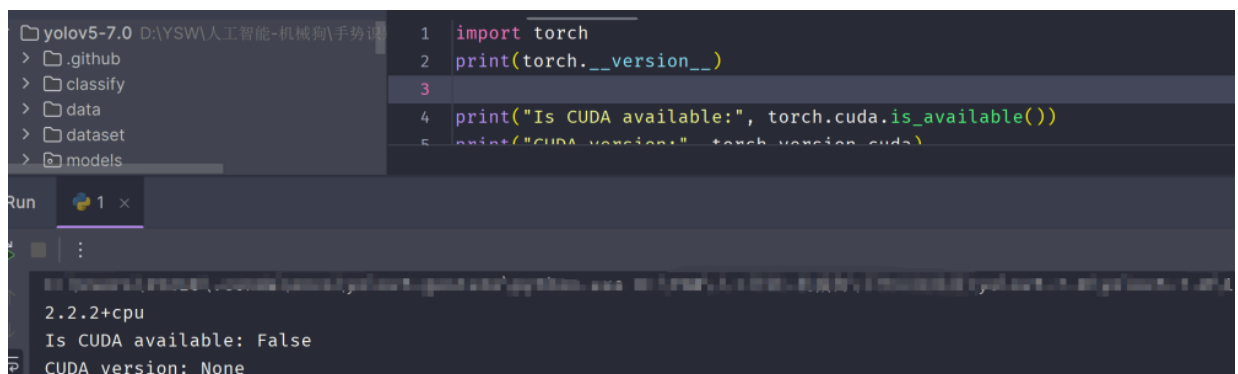
(2) 选择 Add Local Interpreter——>Load Environments——>选择自己的环境



选择 OK，虚拟环境配置成功。

3.3. 安装和配置 CUDA

1、先检查 pytorch 是否满足 cuda 的版本



torch 的版本是 cpu 版本，且不支持 cuda

2、cuda Toolkit 的网站: <https://developer.nvidia.com/cuda-toolkit-archive>

在官网下载对应的 cuda 和 cuda Toolkit，安装成功之后会在环境变量中显示这个

Error! Unknown document property name.

系统变量(S)	
变量	值
CUDA_PATH	C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.4
CUDA_PATH_V12_4	C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.4

在 PATH 中也能看到

```
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.4\bin
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.4\libnvvp
C:\Program Files (x86)\Common Files\MVS\Runtime\Win32_i86
C:\Program Files (x86)\Common Files\MVS\Runtime\Win64_x64
%SystemRoot%\system32
%SystemRoot%
%SystemRoot%\System32\Wbem
%SYSTEMROOT%\System32\WindowsPowerShell\v1.0\
%SYSTEMROOT%\System32\OpenSSH\
D:\Microsoft VS Code\bin
D:\Git\cmd
D:\ProgramData\anaconda3
D:\ProgramData\anaconda3\Library\bin
D:\ProgramData\anaconda3\Library\mingw-w64\bin
C:\Program Files\NVIDIA Corporation\Nsight Compute 2024.1.1\
C:\Program Files (x86)\NVIDIA Corporation\PhysX\Common
C:\Program Files\NVIDIA Corporation\NVIDIA NvDLISR
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.4\include
```

注意红色的地方是没有自动生成的，需要手动添加

在 cmd 中也会显示

```
nvcc -V
```

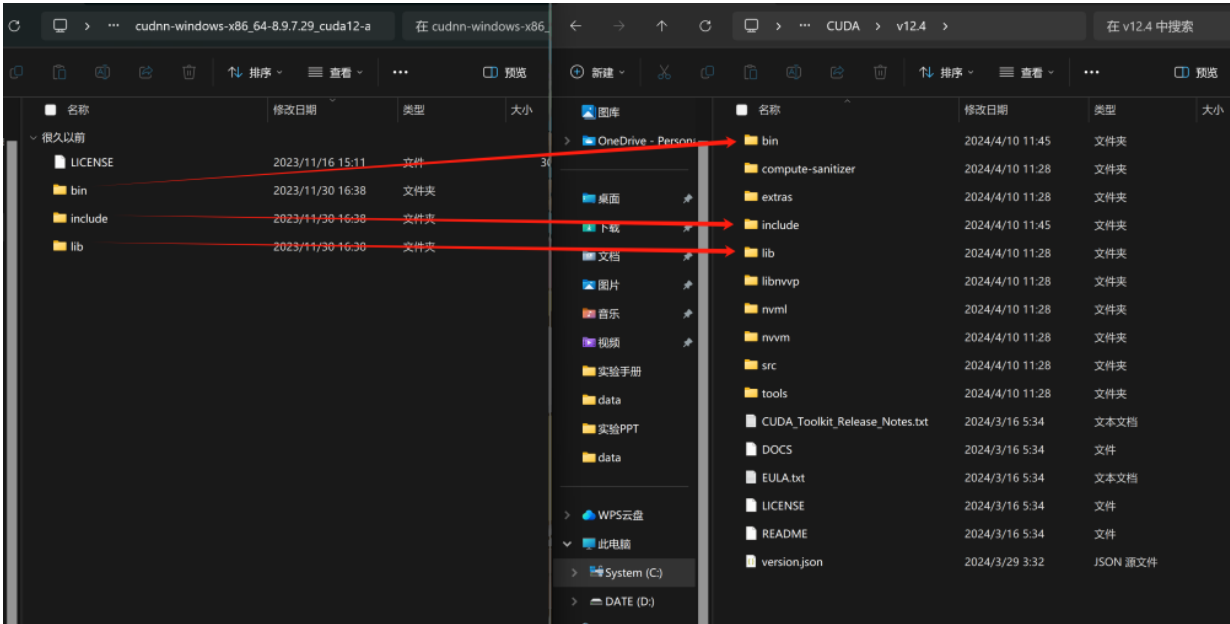
```
C:\WINDOWS\system32\cmd. X + v
Microsoft Windows [版本 10.0.22631.3296]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\15210>nvcc -V
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2024 NVIDIA Corporation
Built on Thu_Mar_28_02:30:10_Pacific_Daylight_Time_2024
Cuda compilation tools, release 12.4, V12.4.131
Build cuda_12.4.r12.4/compiler.34097967_0
```

3、再安装 cuDNN 对应的版本: <https://developer.nvidia.com/rdp/cudnn->

archive

下载对应的 cuDNN 和 cuda 版本之后，解压，然后将 cuDNN 中的文件全部复制到 cuda 中对应的文件



根据步骤来，复制完文件之后，进去这个目录，运行下面两个脚本

```
C:\Windows\System32\cmd.e x + v
Microsoft Windows [版本 10.0.22631.3296]
(c) Microsoft Corporation. 保留所有权利。
C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.4\extras\demo_suite>deviceQuery.exe
deviceQuery.exe Starting...

  CUDA Device Query (Runtime API) version (CUDA static linking)

Detected 1 CUDA Capable device(s)

Device 0: "NVIDIA GeForce RTX 3050 Ti Laptop GPU"
  CUDA Driver Version / Runtime Version      12.4 / 12.4
  CUDA Capability Major/Minor version number: 8.6
  Total amount of global memory:              4096 MBytes (4294443008 bytes)
  (20) Multiprocessors, (128) CUDA Cores/MP:  1485 MHz (1.49 GHz)
  GPU Max Clock rate:                        6001 Mhz
  Memory Clock rate:                         128-bit
  Memory Bus Width:                          2097152 bytes
  Maximum Texture Dimension Size (x,y,z)      1D=(131072), 2D=(131072, 65536), 3D=(16384, 16384, 16384)
  Maximum Layered 1D Texture Size, (num) layers 1D=(32768), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers 2D=(32768, 32768), 2048 layers
  Total amount of constant memory:            zu bytes
  Total amount of shared memory per block:    zu bytes
  Total number of registers available per block: 65536
  Warp size:                                  32
  Maximum number of threads per multiprocessor: 1536
  Maximum number of threads per block:        1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
```

Error! Unknown document property name.

```

C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v12.4\extras\demo_suite>bandwidthTest.exe
[CUDA Bandwidth Test] - Starting...
Running on...

Device 0: NVIDIA GeForce RTX 3050 Ti Laptop GPU
Quick Mode

Host to Device Bandwidth, 1 Device(s)
PINNED Memory Transfers
Transfer Size (Bytes)      Bandwidth(MB/s)
33554432                   10992.5

Device to Host Bandwidth, 1 Device(s)
PINNED Memory Transfers
Transfer Size (Bytes)      Bandwidth(MB/s)
33554432                   12092.3

Device to Device Bandwidth, 1 Device(s)
PINNED Memory Transfers
Transfer Size (Bytes)      Bandwidth(MB/s)
33554432                   146545.3

Result = PASS

NOTE: The CUDA Samples are not meant for performance measurements. Results may vary when GPU Boost is enabled.

```

这样说明 cuDNN 安装成功

4、在 pycharm 中测试

```

import torch
print(torch.__version__)
print(torch.cuda.is_available())

```

3 x

⋮

```

C:\Users\15210\.conda\envs\yolov5-
2.2.2+cu121
True

```

True 说明成功，cu121 是指 cuda 版本 12.1

注意：有时候如果安转了之后还是不能使用 cuda，可能需要卸载掉原来的 torch torchvision torchaudio，再去官网找到命令，重新安装

3.4. 开发板环境（机器狗网址配置）

用户可通过 SSH 远程连接到运动主机。

1. 将开发主机连接到机器人 WiFi。

2. 在开发主机上打开 SSH 连接软件，输入 `ssh firefly@192.168.1.120`，密码为 `firefly`，即可远程连接运动主机；

3. 进入 `jy_exe`，并打开配置文件：

```
cd /home/firefly/jy_exe/conf/  
  
vim network.toml
```

4. 配置文件 `network.toml` 内容如下：

```
ip = '192.168.1.102'  
  
target_port = 43897  
  
local_port = 43893  
  
~
```

5. 修改配置文件第一行中的 IP 地址，使得代码能够接收到机器狗数据：

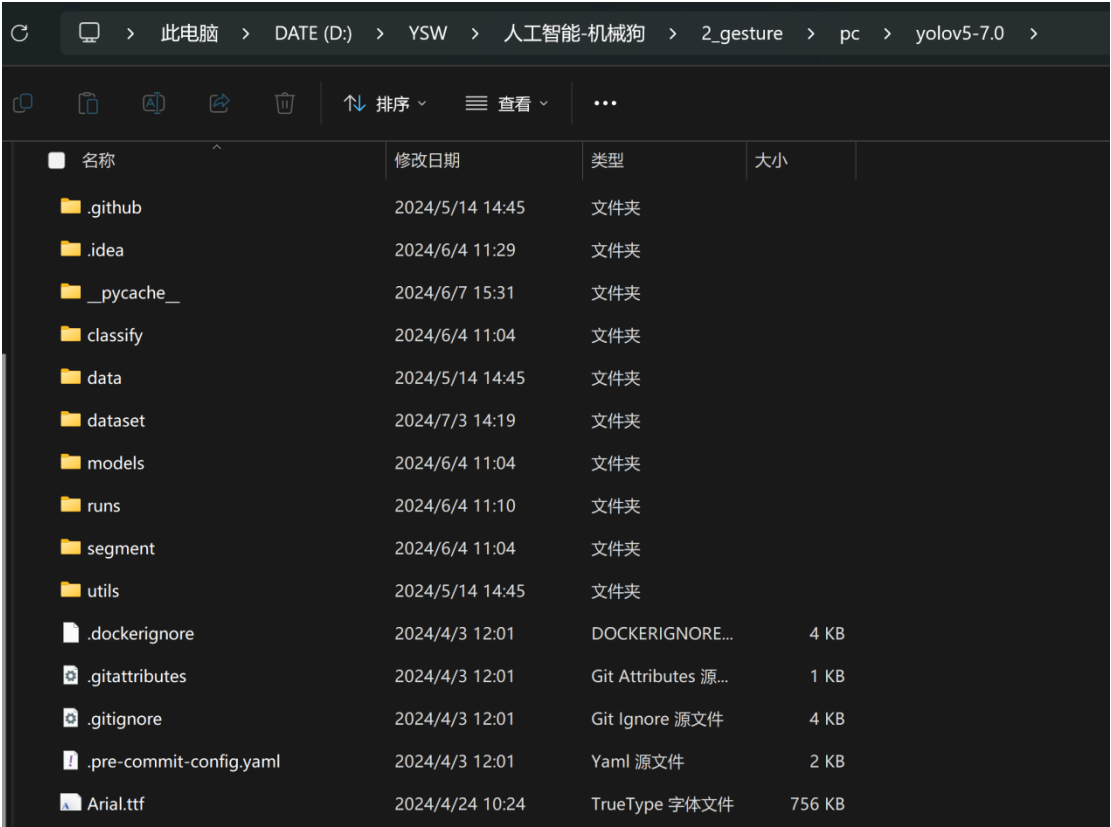
- 如果代码在机器人运动主机内运行，IP 设置为运动主机 IP：192.168.1.120；
- 如果代码在用户自己的开发主机中运行，设置为开发主机的静态 IP：192.168.1.xxx；

6. 重启运动程序使配置生效。

```
cd /home/firefly/jy_exe  
  
sudo ./stop.sh  
  
sudo ./restart.sh
```

第四章 工程文件目录结构

4.1. PC



PC 端 yolov5 的文件夹，主要内容为 **dataset** 中存放的数据文件，其他的地方不需要修改。**train.py** 是主要的训练文件

4.2. Ascend

本章节主要展示的是在开发板上所有的文件，以及每个文件所对应的作用。本章节主要展示开发板上所有的文件及其对应的作用，以便更好地理解整个项目的结构和功能。

```
(base) root@davinci-mini:/opt/lab/GestureRecognition# tree -I "__pycache__|*.pyc" -L 10
.
├── camera
│   ├── HKcamera.py
│   └── det_utils.py
├── command
│   ├── __init__.py
│   └── udp_command.py
├── gesture_main.py
├── gesture_models
│   ├── fusion_result.json
│   ├── gesture.om
│   ├── model.onnx
│   └── predefined_classes.txt
├── getAndSavePicture.py
├── robotstatuswatcher
│   ├── __init__.py
│   └── listener.py
├── sendcommand
│   ├── SendToCommand.py
│   ├── __init__.py
│   └── heartbeat.py
├── socketnetwork
│   ├── __init__.py
│   └── network_utils.py
├── speeds
│   ├── __init__.py
│   └── sportsspeed.py
├── threading_utils
└── ThreadTemplates.py
```

开发板的文件夹总名叫做 ‘GestureRecognition’，gesture_main.py是运行的主程序，其他的功能包如下所示：

Command 文件夹

- **udp_command.py:** 该文件用于存放指令码。它定义了各种控制命令，这些命令通过UDP协议发送到机器狗，以实现对其精确控制。

SendCommand 文件夹

- **heartbeat.py:** 该文件用于发送心跳包。心跳包是一种定期发送的信号，用于检测系统或设备是否在线和正常工作。通过发送心跳包，可以确保机器狗与控制系统之间的通信连接是稳定和可靠的。
- **SendToCommand.py:** 该文件负责发送指令。它包含了一系列函数，用于将指令发送到机器狗，确保指令能够正确地传达和执行。

SocketNetwork 文件夹

- **network_utils.py:** 该文件是与机器狗建立通信的脚本。它包含了各种网络通信的实用函数，用于初始化和维护与机器狗的通信连接，使得控制指令能够通过网络传输到机器狗。

speeds文件夹

- **sportspeed.py**是机器狗运动的速度方程，**threading_utils**中的**ThreadTemplates.py**文件是线程，**camera**中放的是摄像头的使用文件，**robotstatuswatcher**中使用来监听机器狗状态的，**gesture_models**中存放的是模型文件，**getAndSavePicture.py**是采集数据的脚本

ThreadingUtils 文件夹

- **ThreadTemplates.py**: 该文件定义了线程模板。线程是并发执行的基本单位，通过使用线程，可以提高系统的响应速度和处理效率。该文件中的模板可以用于创建和管理多个线程，以实现并行处理。

robotstatuswatcher文件夹

- **listener.py**: 该文件定义了监听机器狗状态的函数。

gesture_models文件夹

- 该文件夹中存放了用来推理的模型文件。

camera文件夹

- **HKcamera.py**: 该文件是调用摄像头的文件。

4.3. ModelArts

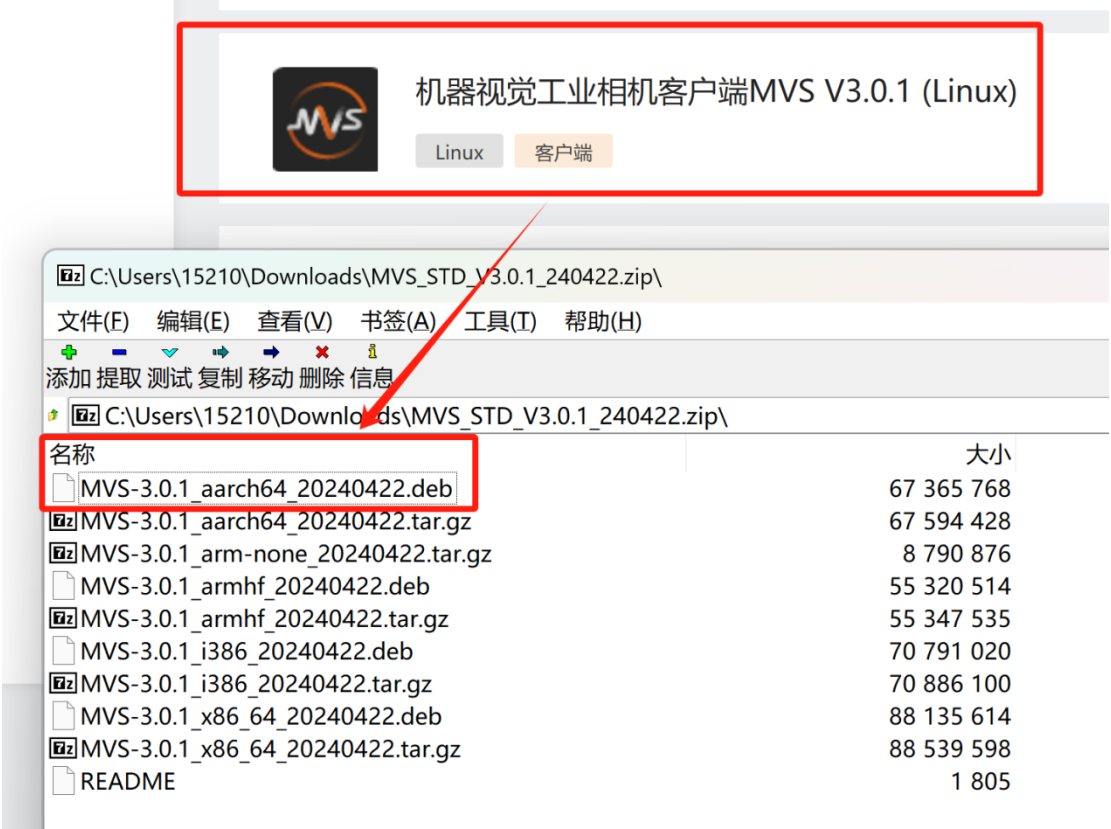


yolov5 与 PC 端相似，output 用来存放训练好的模型。

第五章 数据工程

5.1. 配置开发板的 HK 驱动

下载并安装 MVS 驱动文件（Linux）



海康威视官网下载链接：

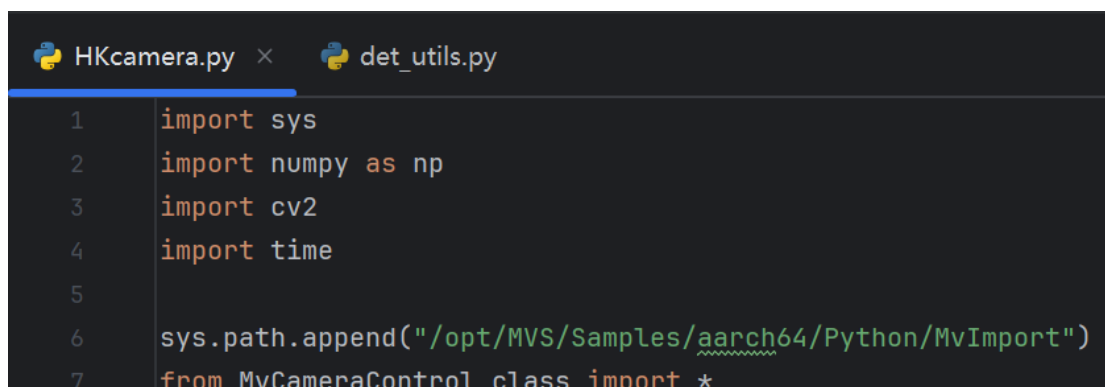
<https://www.hikrobotics.com/cn/machinevision/service/download?module=0>



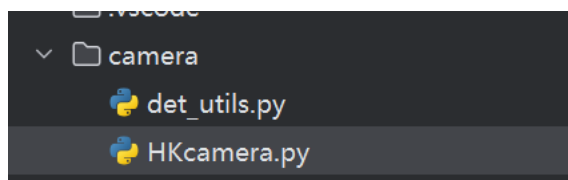
打开 MobaXterm 远程控制，进入开发板安装驱动指令：`dpkg -i MVS-3.0.1_aarch64_20240422.deb`

在代码中使用需要加入包的路径：

```
sys.path.append("/opt/MVS/Samples/aarch64/Python/MvImport")from
MvCameraControl_class import*
```



使用摄像头的代码：`python HKcamera.py`



5.2. 编写数据采集脚本

数据采集的方式是通过调用摄像头的 SDK 来获取图片信息，然后保存到开发板中，脚

本名为 getAndSavePicture.py。

```
1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3.
4. import cv2
5. import time
6.
7. import sys
8. sys.path.append('camera/')
9. from HKcamera import getImage
10. from det_utils import get_labels_from_txt, letterbox, scale_coords, nms
    , draw_bbox # 模型前后处理相关函数
11.
12. # 导入定义的机器狗相关的包
13. from threading_utils.ThreadTemplates import * # 线程的模板与基
    本轴指令的发送
14. from sendcommand import SendToCommand, heartbeat # 发送简单指令与
    心跳包
15. from socketnetwork import network_utils # 通信函数
16. from command.udp_command import * # 存放各种结构体
    与状态数据、指令
17. from speeds.sportspeed import * # 运动精准控制的
    基础版模型
18. from robotstatuswatcher.listener import * # 监听机器狗状态
```

```

19. from HandDistance.ridge_np import *                                # 测量距离

20. #getImage 从 HKcamera 模块导入，用于获取图像。
    get_labels_from_txt, letterbox, scale_coords, nms, draw_bbox 从
    det_utils 模块导入，这些函数通常用于

21. #目标检测模型的前后处理。导入与机器狗相关的包，如线程模板、发送指令、网络通
    信、命令结构体、运动控制和状态监听等。

22. def save_image(img):

23.     image_name = f'train_images/demo/{time.time()}.jpg'

24.     cv2.imwrite(image_name, img)

25.     print(f"图像已保存为: {image_name}")

26.

27. print("按 's' 保存图像，按 'q' 退出程序。")

28.

29. # 主循环

30. #通过 getImage() 获取图像。

31. #使用 cv2.imshow 显示图像。

32. #使用 cv2.waitKey(1) 检测按键事件，如果按下 's'，则调用 save_image 保存图
    像；如果按下 'q'，则退出循环。

33. while True:

34.     img = getImage() # 获取图像

35.

36.     cv2.imshow('Camera Feed', img) # 显示图像

37.     key = cv2.waitKey(1) & 0xFF # 检测按键事件

```

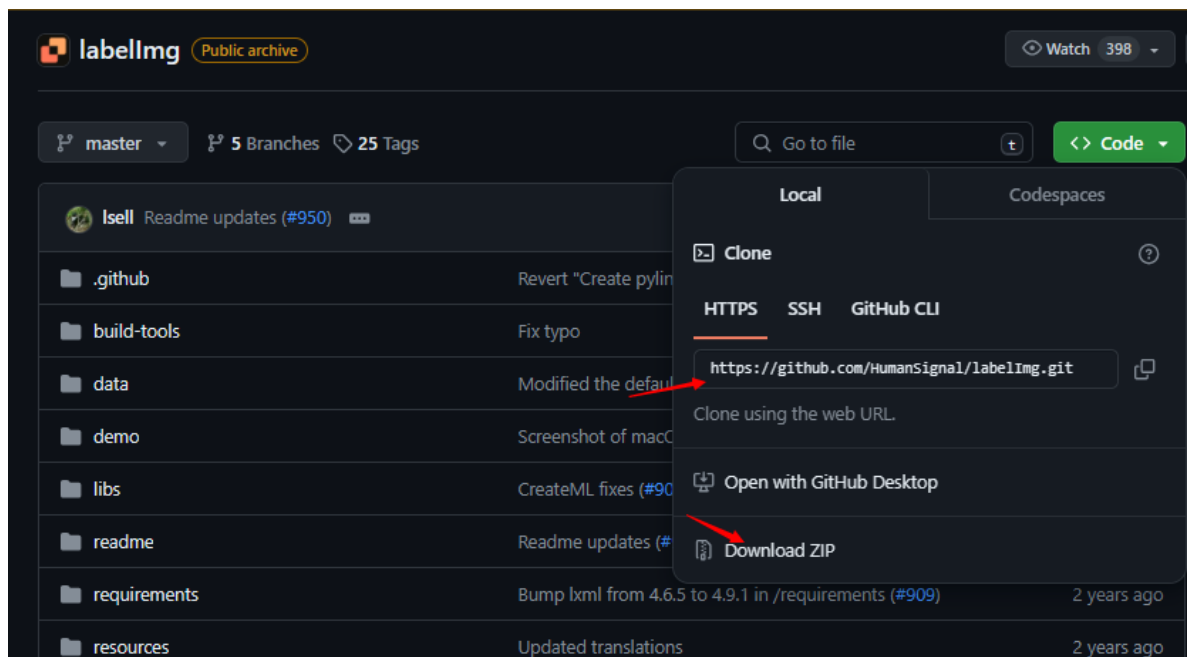
```

38.
39.     if key == ord('s'):
40.         save_image(img) # 保存图像
41.     elif key == ord('q'):
42.         break # 退出循环
43.
44. # 清理工作
45. cv2.destroyAllWindows()

```

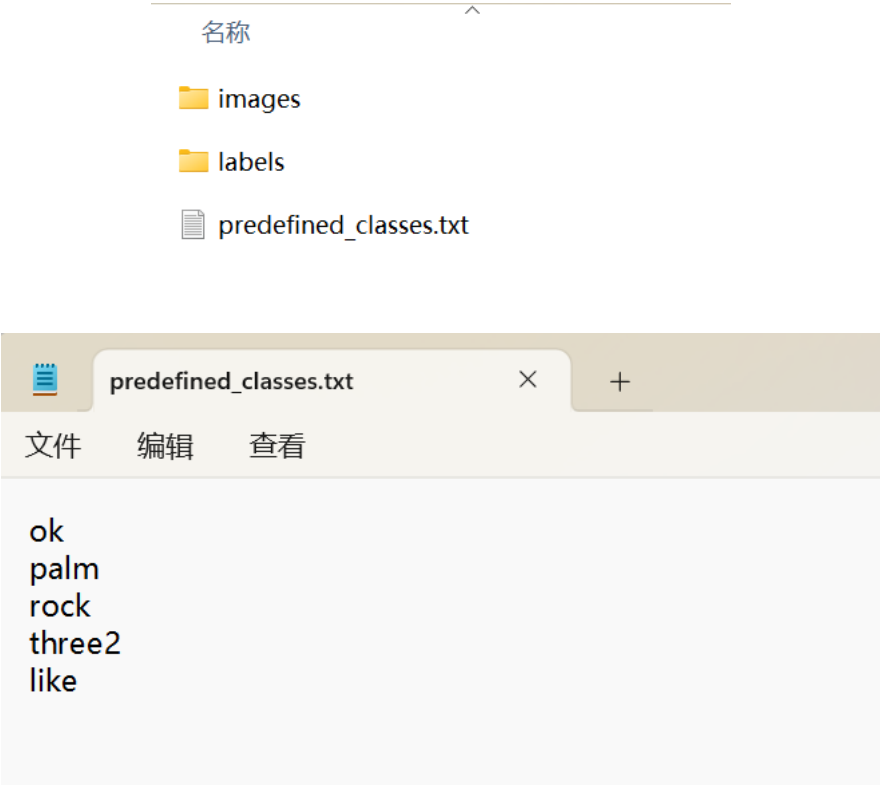
5.3. 配置 PC 端数据标注环境

下载数据标注工具 labelimg: <https://github.com/HumanSignal/labelImg>

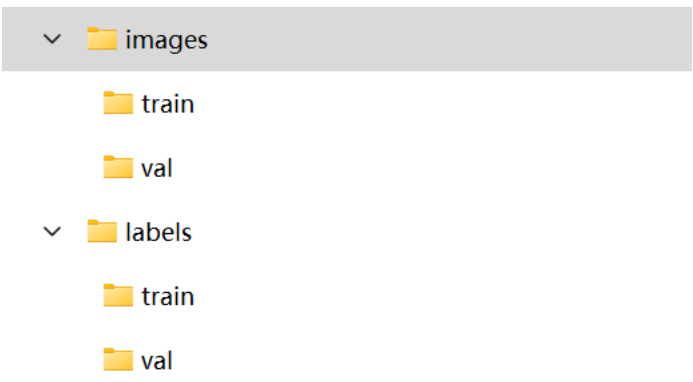


建立两个文件夹,分别是 images 用来存放训练集 train 的图片和验证集 val 的图片,labels 用来存放标注完 train 中对应的图片数据标签和 val 中的对应的图片数据标签, 以及

predefined_classes.txt 存放标签类别。



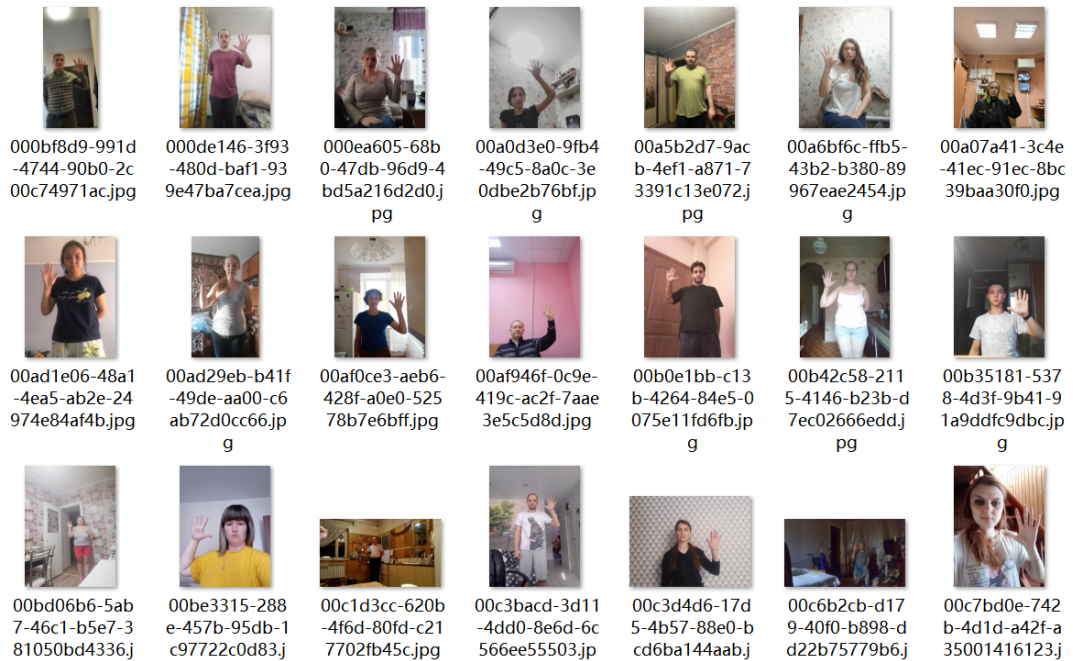
/images/train 对应 labelImg 中的存放目录，而/labels/train 对应 labelImg 的改变存放目录，用于存放标注完图片的数据标签。



5.4. 标注数据集

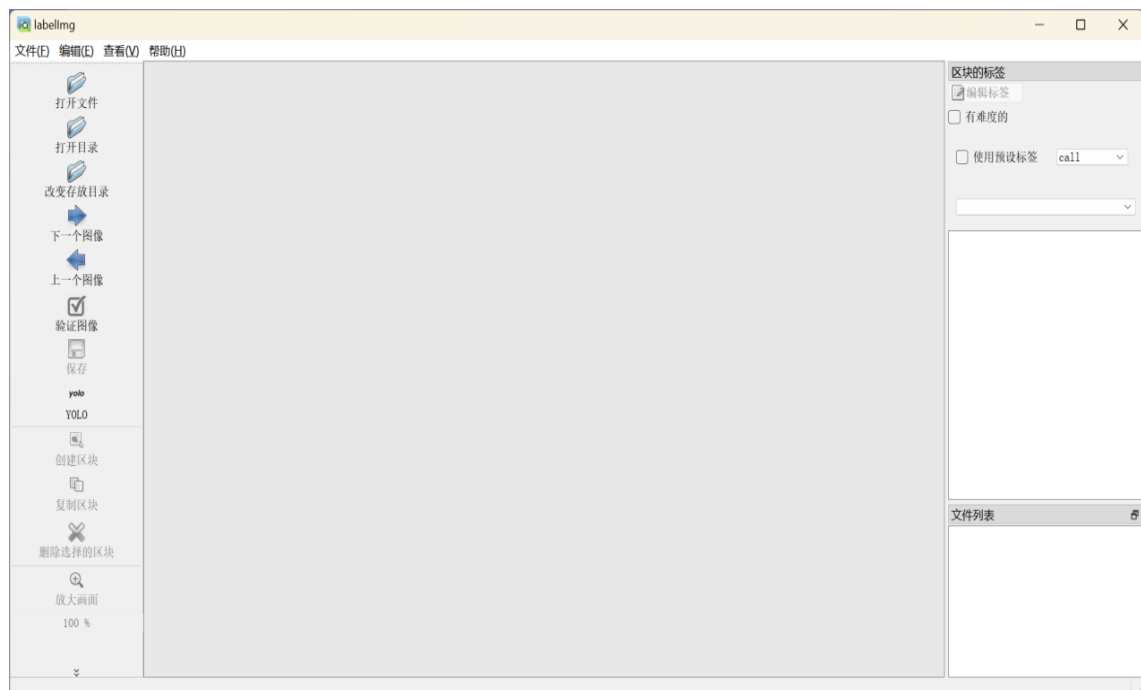
5.4.1. 采集数据

在代码高亮上下载已有不同类型图片，选择其中几种手势，作为手势识别的数据来源，以其中一类 palm 为例，可看到如下图所示。



5.4.2. 使用 labelImg 进行数据集标注

- 1、打开 anaconda 命令行界面
- 2、激活我们已有的环境，如果没有就先创建再激活
- 3、输入 labelImg 回车，打开标注界面，下图我对各个按钮进行了详细解释



- 4、同时在 predefined_classes.txt 中，设置所分类的标签名称；在 data 文件夹中分别新建 images、label 文件夹，同时在 images 文件夹中，新建 train 文件夹，里面存放训练集手势照片，与之对应应在 label 文件夹中，新建 train 文件夹，里面用来存放标注好的数据集信息；

同时在 images 文件夹中，再新建 val 文件夹，用来存放验证集手势照片，与之对应在 label 文件夹中，新建 val 文件夹，里面用来存放标注好的数据集信息

5、在“打开目录”中打开刚刚的 data/images/train 文件夹，在“改变存放目录”中更换保存路径为 data/label/train 文件夹

5.4.3. 数据标定

进入 labelImg-master 所在文件夹地址

```
(base) PS E:\> cd E:\Desktop\亿生为\labelImg-master
(base) PS E:\Desktop\亿生为\labelImg-master> python labelImg.py
E:\Desktop\亿生为\labelImg-master\labelImg.py:73: DeprecationWarning: sipPyTypeDict() is deprecated, the extension module should use sipPyTypeDictRef() instead
class MainWindow(QMainWindow, WindowMixin):
[('palm', [(207, 279), (310, 279), (310, 406), (207, 406)], None, None, False)]
```

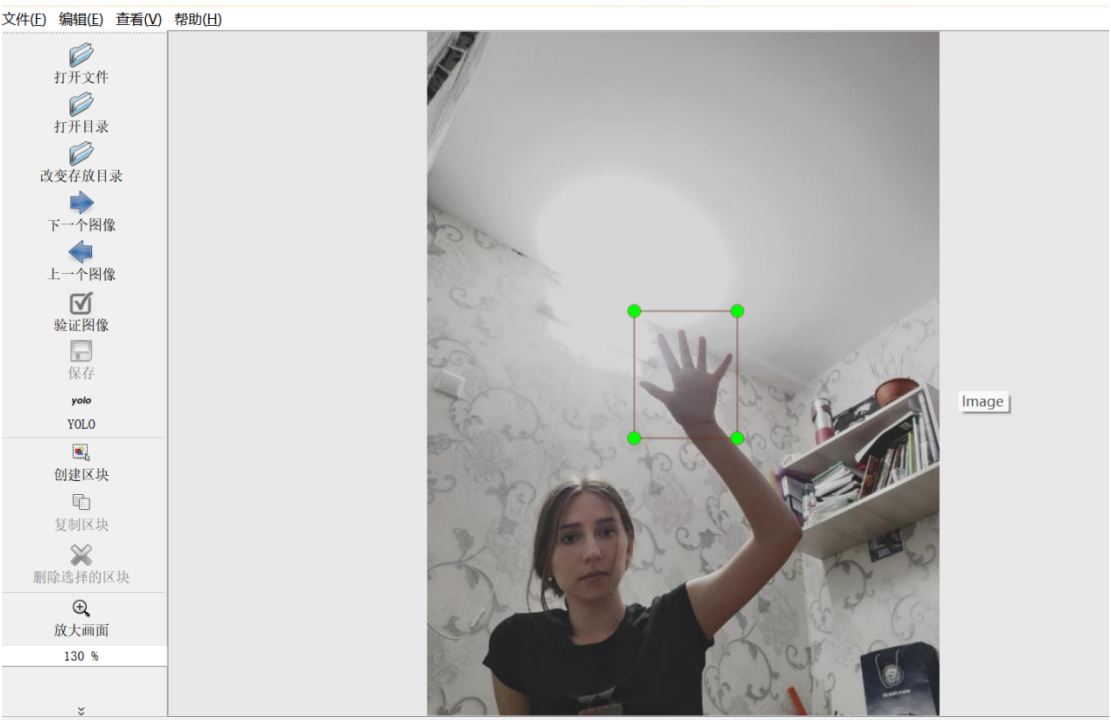
后输入 python labelImg.py 进入数据标注页面，点击”创造区块”，然后出现十字架，进行拖动框住标注区域

然后输入标签，点击 palm，再点击左侧的“保存”标签

在 data/label/train 里面可以看到我们的标签，在此介绍 labelImg 的三种标签格式：

- (1) PascalVOC 标签格式，保存为 xml 文件
- (2) YOLO 标签格式，保存为 txt 文件
- (3) CreateML 标签格式，保存为 json 文件

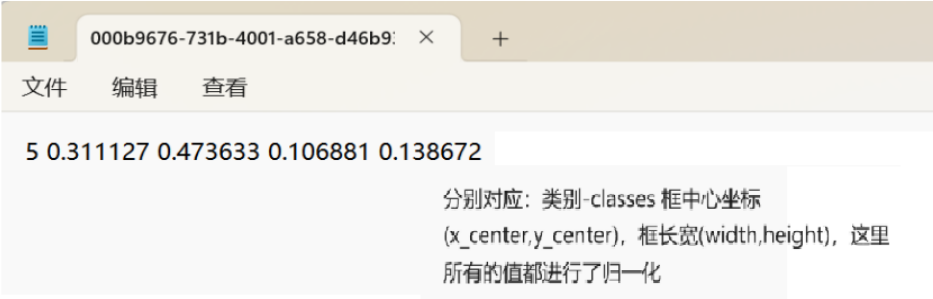
点击“下一个图像”继续标注下一张



5.4.4. 数据预览

数据标注完成后，若标注图片是\labeldata\images\train 中，则可到\labeldata\labels\train

中查看标签文件，images 中存放训练集 train 图，labels 中存放训练集 train 所对应的数据标签 txt 文件，可看到其文件名对应图片名称，txt 中的四列数据分别对应数据标注区块各个含义，具体如下图所示。



第六章 模型本地化训练与推理



6.1. 训练流程

1.模型训练

- a.详细流程：调用 Yolov5 训练文件，读取制作的数据集，来训练我们的模型。
- b.实验注意点：超参数的设置，以及数据集文件结构。
- c.重点知识：Yolov5 训练过程，超参数设置。
- d.完成本步骤后输出：在 runs/train/exp 文件夹下保存的 pt 模型。

2.模型测试

- a.详细流程：调用 detect.py 文件的参数，对模型进行验证，
- b.实验注意点：需要指定测试的图片和 pt 模型路径。
- c.重点知识：检测验证。
- d.完成本步骤后输出：在指定的目录中保存检测后的图片。

3.模型转换

- a.详细流程：在 PC 机上将.pt 模型转换为.onnx 模型；再在 PC 上将 onnx 模型转换为 om

模型。

b.实验注意点: om 模型转换命令参数。

c.重点知识: YOLOv5 转换 onnx 模型流程; onnx 模型转换为 om 模型的命令使用。

d.完成本步骤后输出: 可以在昇腾开发板上推理的 om 模型。

6.开发板相机设置

a.详细流程: 打开工业相机保护盖, 连接工业相机和昇腾开发板, 调整 HKcamera.py 代码参数和补光灯亮度, 使工业相机清晰成像。

b.实验注意点: HKcamera.py 代码内容调整后, 保存后才能生效。

c.重点知识: 昇腾开发板上工业相机的调整。

d.完成本步骤后输出: 运行 HKcamera.py 代码后显示清晰图片。

5.开发板推理

a.详细流程: 编写昇腾开发板推理 om 模型的代码。

b.实验注意点: 代码中 MindX SDK 调用时需要方便拓展。

c.重点知识: 昇腾开发板推理 om 的方法

d.完成本步骤后输出: 运行代码后会输出目标检测后的结果图。

6.2. 训练参数详细介绍与修改

参数通常用于配置训练深度学习模型时的各种选项。这些参数允许用户在训练模型时灵活地设置各种配置选项。下面是对这些参数的解释:

--weights: 指定初始权重的路径, 默认为 ROOT/yolov5s.pt。

--cfg: 指定模型配置文件的路径, 默认为空, 需要用户指定。

--data: 指定数据集配置文件的路径, 默认为 ROOT/dataset/data.yaml。

--train_url: 用于 ModelArts 平台的训练 URL, 默认为 "weights"。

--hyp: 指定超参数配置文件的路径, 默认 ROOT/data/hyps/hyp.scratch-low.yaml。

--epochs: 指定总的训练周期数, 默认为 100。

--batch-size: 指定所有 GPU 的总批量大小, 默认为 16, -1 表示自动批量。

--imgsz 或 --img 或 --img-size: 指定训练和验证时的图像大小(像素), 默认为 640。

--rect: 如果设置, 将使用矩形训练。

--resume: 如果设置, 将恢复最近的训练。

- nosave: 如果设置，只保存最终的检查点。
- noval: 如果设置，只在最后一个周期进行验证。
- noautoanchor: 如果设置，将禁用自动锚点调整。
- noplots: 如果设置，不保存任何绘图文件。
- evolve: 指定用于进化超参数的代数，默认为 300。
- bucket: 指定用于 gsutil 的存储桶。
- cache: 指定图像缓存的方式，可以是 'ram' 或 'disk'，默认为 'ram'。
- image-weights: 如果设置，将使用加权图像选择进行训练。
- device: 指定使用的设备，例如 '0' 或 '0,1,2,3' 表示 GPU 设备号，或者 'cpu' 表示使用 CPU。

1.新建一个 dataset 文件夹，在里面除了存放数据之外再新建一个 data.yaml，对数据集进行配置



1. train: dataset/images/train

2. val: dataset/images/val

3. test:

4.

5. # number of classes

6. nc: 19

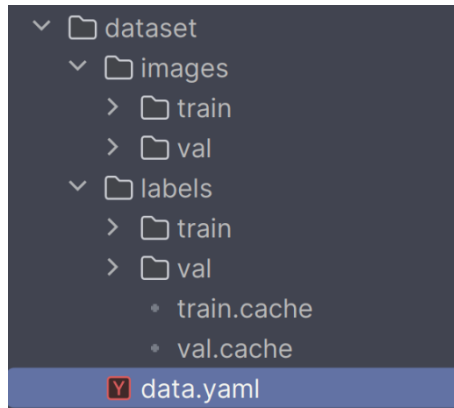
7.

8. # class names

Error! Unknown document property name.

```
9. names: ['call','dislike','fist','four','like','mute','ok','one','palm',
          'peace','rock','stop','stop inverted','three','two up','two up inverted',
          'three2','peace inverted','irrelevant hand']
```

2.整理数据集文件结构，确定和如下图示相同。并且 labels 下需要存放 classes.txt 文件。

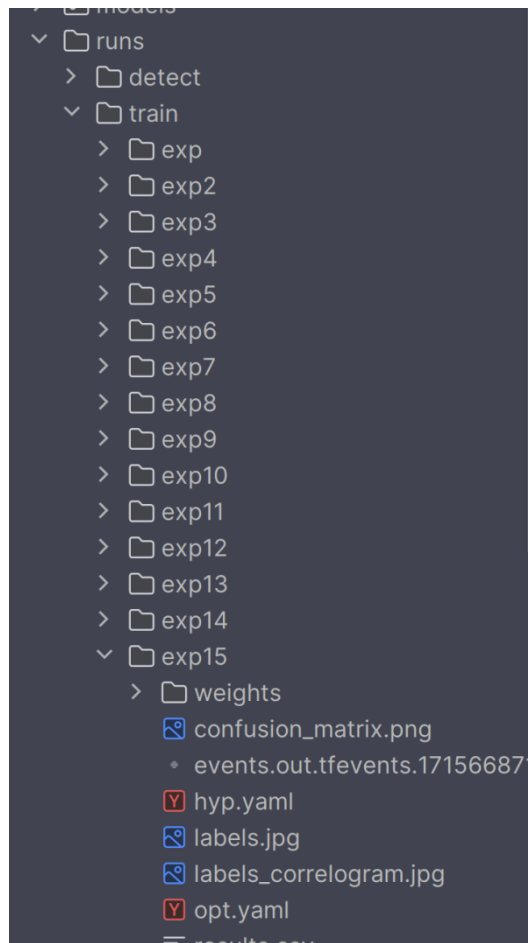


3.修改 train.py 文件的参数，其中主要包括--weights[预训练权重路径]、--cfg[模型结构配置]、--data[数据集配置文件]、--epochs[训练轮数]、--batch-size[批数据大小]。

本文所采取的参数如下：

```
1. python train.py --weights yolov5s.pt --cfg yolov5s.yaml --
   data dataset/data.yaml --epochs 100 --batch-size 4
```

4.运行 train.py,等到训练完成后就可以在 runs 文件下的 train 目录找到训练的结果



6.3. 模型训练

本文的参数选择: `python train.py --img 640 --batch 16 --epoch 500 --data dataset/data.yaml -`
`-cfg models/yolov5s.yaml --weights yolov5s.pt`

```

2         -1 1      18816 models.common.C3 [64, 64, 1]
3         -1 1      73984 models.common.Conv [64, 128, 3, 2]
4         -1 2     115712 models.common.C3 [128, 128, 2]
5         -1 1     295424 models.common.Conv [128, 256, 3, 2]
6         -1 3     625152 models.common.C3 [256, 256, 3]
7         -1 1     1180672 models.common.Conv [256, 512, 3, 2]
8         -1 1     1182720 models.common.C3 [512, 512, 1]
9         -1 1     656896 models.common.SPPF [512, 512, 5]
10        -1 1     131584 models.common.Conv [512, 256, 1, 1]
11        -1 1          0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
12        [-1, 6] 1          0 models.common.Concat [1]
13        -1 1     361984 models.common.C3 [512, 256, 1, False]
14        -1 1     33024 models.common.Conv [256, 128, 1, 1]
15        -1 1          0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
16        [-1, 4] 1          0 models.common.Concat [1]
17        -1 1     90880 models.common.C3 [256, 128, 1, False]
18        -1 1     147712 models.common.Conv [128, 128, 3, 2]
19        [-1, 14] 1          0 models.common.Concat [1]
20        -1 1     296448 models.common.C3 [256, 256, 1, False]
21        -1 1     590336 models.common.Conv [256, 256, 3, 2]
22        [-1, 10] 1          0 models.common.Concat [1]
23        -1 1     1182720 models.common.C3 [512, 512, 1, False]
24        [17, 20, 23] 1     26970 models.yolo.Detect [5, [[10, 13, 16, 30, 33, 23], [30, 61, 62,
45, 59, 119], [116, 90, 156, 198, 373, 326]], [128, 256, 512]]
YOLOv5s summary: 214 layers, 7033114 parameters, 7033114 gradients, 16.0 GFLOPs

Transferred 342/349 items from yolov5s.pt
AMP: checks passed
optimizer: SGD(lr=0.01) with parameter groups 57 weight(decay=0.0), 60 weight(decay=0.0005), 60 bias
train: Scanning D:\YSW\人工智能-机械狗\2_gesture\pc\yolov5-7.0\dataset\labels\train...: 0% | 0/5 00:00

```

```

AttributeError: FreeTypeFont object has no attribute 'getsize'

```

	Class	Images	Instances	P	R	mAP50	mAP50-95: 100%	1/1 00:00
Epoch 3/499	all	2	1	0	0	0	0	1/1 00:00
	GPU_mem	box_loss	obj_loss	cls_loss	Instances	Size		
	1.23G	0.1275	0.0289	0.05135	8	640: 100%	1/1 00:00	
	Class	Images	Instances	P	R	mAP50	mAP50-95: 100%	1/1 00:00
	all	2	1	0	0	0	0	1/1 00:00
Epoch 4/499	all	2	1	0	0	0	0	1/1 00:00
	GPU_mem	box_loss	obj_loss	cls_loss	Instances	Size		
	1.23G	0.0808	0.02902	0.03415	8	640: 100%	1/1 00:00	
	Class	Images	Instances	P	R	mAP50	mAP50-95: 100%	1/1 00:00
	all	2	1	0	0	0	0	1/1 00:00
Epoch 5/499	all	2	1	0	0	0	0	1/1 00:00
	GPU_mem	box_loss	obj_loss	cls_loss	Instances	Size		
	1.23G	0.1234	0.02895	0.04959	8	640: 100%	1/1 00:00	
	Class	Images	Instances	P	R	mAP50	mAP50-95: 100%	1/1 00:00
	all	2	1	0	0	0	0	1/1 00:00
Epoch 6/499	all	2	1	0	0	0	0	1/1 00:00
	GPU_mem	box_loss	obj_loss	cls_loss	Instances	Size		
	1.23G	0.1158	0.03143	0.05045	10	640: 100%	1/1 00:00	
	Class	Images	Instances	P	R	mAP50	mAP50-95: 100%	1/1 00:00
	all	2	1	0	0	0	0	1/1 00:00
Epoch 7/499	all	2	1	0	0	0	0	1/1 00:00
	GPU_mem	box_loss	obj_loss	cls_loss	Instances	Size		
	1.23G	0.1126	0.02856	0.05895	9	640: 100%	1/1 00:00	
	Class	Images	Instances	P	R	mAP50	mAP50-95: 100%	1/1 00:00
	all	2	1	0	0	0	0	1/1 00:00

训练完成之后在 runs\train\exp17 的文件夹下可以看到训练的文件

Results saved to runs\train\exp17

6.4. 模型测试

首先在 yolov5-7.0\runs\train\exp15\weight\1.png 中放入测试图片

Error! Unknown document property name.



修改 detect.py 文件的参数，对模型进行验证，其中主要包括--weights[训练权重路径]、--source[测试图片目录]、--data[数据集配置文件]

本文所采取的参数如下：

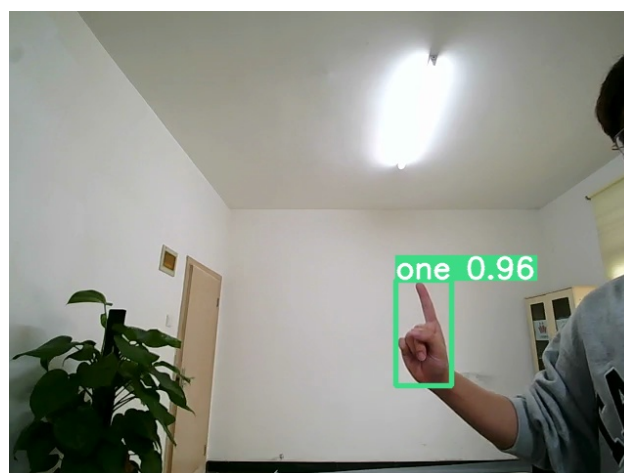
```
python detect.py --weights runs/train/exp15/weights/best.pt --source "D:/YSW/人工智能-机械狗/2_gesture/pc/yolov5-7.0/runs/train/exp15/weights/1.jpg" --data dataset/data.yaml
```

运行成功之后

```
(yolov5-gesture) PS D:\YSW\人工智能-机械狗\2_gesture\pc\yolov5-7.0> python detect.py --weights runs/train/exp15/weights/best.pt --source "D:/YSW/人工智能-机械狗/2_gesture/pc/yolov5-7.0/runs/train/exp15/weights/1.jpg" --data dataset/data.yaml
detect: weights=['runs/train/exp15/weights/best.pt'], source=D:/YSW/-/2_gesture/pc/yolov5-7.0/runs/train/exp15/weights/1.jpg, data=dataset/data.yaml, imgsz=[640, 640], conf_thres=0.25, iou_thres=0.45, max_det=1000, device=, view_img=False, save_txt=False, save_conf=False, save_crop=False, nosave=False, classes=None, agnostic_nms=False, augment=False, visualize=False, update=False, project=runs/detect, name=exp, exist_ok=False, line_thickness=3, hide_labels=False, hide_conf=False, half=False, dnn=False, vid_stride=1
YOLOv5 2024-4-3 Python-3.9.19 torch-2.2.2+cu121 CUDA:0 (NVIDIA GeForce RTX 3050 Ti Laptop GPU, 4096MiB)

Fusing layers...
YOLOv5s summary: 157 layers, 7061368 parameters, 0 gradients, 15.9 GFLOPs
image 1/1 D:\YSW\2_gesture\pc\yolov5-7.0\runs\train\exp15\weights\1.jpg: 480x640 1 one, 80.2ms
Speed: 0.0ms pre-process, 80.2ms inference, 170.1ms NMS per image at shape (1, 3, 640, 640)
Results saved to runs\detect\exp16
```

结果保存在了下面路径 runs\detect\exp16



可以看到已经手势识别成功。

6.5. 模型转换

1. 首先将模型转换为 onnx

```
1. python export.py --weights runs/model.pt --include onnx
```

输入之后会看到有

```
(yolov5-gesture) PS D:\YSW\人工智能-机械狗\4_smart_logistics\pc\yolov5-7.0> python export.py --weights runs/model.pt --include onnx
export: data=D:\YSW\人工智能-机械狗\4_smart_logistics\pc\yolov5-7.0\data\coco128.yaml, weights=['runs/model.pt'], imgsz=[640, 640], batch_size=1, device=cpu, half=False, inplace=False, keras=False, optimize=False, int8=False, dynamic=False, simplify=False, opsset=12, verbose=False, workspace=4, nms=False, agnostic_nms=False, topk_per_class=100, topk_all=100, iou_thres=0.45, conf_thres=0.25, include=['onnx']
YOLOv5 2024-5-14 Python-3.9.19 torch-2.2.2+cu121 CPU

Fusing layers...
YOLOv5s summary: 157 layers, 7020913 parameters, 0 gradients, 15.8 GFLOPs
```

等待片刻会出现转换成功的提示，以及保存的位置

```
Export complete (1.5s)
Results saved to D:\YSW\人工智能-机械狗\2_gesture\pc\yolov5-7.0\runs
Detect:      python detect.py --weights runs\model.onnx
Validate:    python val.py --weights runs\model.onnx
PyTorch Hub: model = torch.hub.load('ultralytics/yolov5', 'custom', 'runs\model.onnx')
Visualize:   https://netron.app
(yolov5-gesture) PS D:\YSW\人工智能-机械狗\2_gesture\pc\yolov5-7.0> |
```

2. 将训练得到的 ONNX 模型文件，修改名字为 model.onnx（方便区别，如果名字是 model 就不需要修改），上传到开发板

```
(base) root@davinci-mini:/opt# ll
total 27888
drwxr-xr-x  2 root root    4096 Apr 12 17:10 ./
drwxr-xr-x 24 root root    4096 Apr  8 2022 ../
-rw-r--r--  1 root root 28545135 Apr 12 17:10 model.onnx
```

运行下面命令

```
1. atc --model=model.onnx --framework=5 --output=model --soc_version=Ascend310B4
```

若提示如下信息，则说明模型转化成功。

ATC run success

```
(base) root@davinci-mini:/opt# atc --model=model.onnx --framework=5 --output=model --soc_version=Ascend310B4
ATC start working now, please wait for a moment.
...
ATC run success, welcome to the next use.
```

```
(base) root@davinci-mini:/opt# ll
total 42844
drwxr-xr-x  2 root root    4096 Apr 12 17:26 ./
drwxr-xr-x 24 root root    4096 Apr  8 2022 ../
-rw-r----- 1 root root    3022 Apr 12 17:26 fusion_result.json
-rw-----  1 root root 15307128 Apr 12 17:26 model.om
-rw-r--r--  1 root root 28545135 Apr 12 17:10 model.onnx
```

第七章 在线 ModelArts 训练与模型转换及应用

7.1. ModelArts 介绍

AI 开发平台 ModelArts 是面向开发者的一站式 AI 开发平台，可快速创建和部署模型，管理全周期 AI 工作流，助力千行百业智能升级。



- AI 开发平台 ModelArts

构筑全栈全生命周期的模型开发工具链，通过全面的 AI 工具和服务，为业务智能快速创新赋能

- AI 高效开发

提供端到端模型生产线，高效开发、调试和调优大模型应用和场景化应用

提供端到端监控工具，智能运营运维

MLOps 高效迭代 AI 模型，持续提升精度

数智融合，数据服务与 AI 开发全流程打通

对接 AI Gallery，便捷使用开源大模型等预置资产

- AI 高效运行

提供 AI 加速套件，支持数据加速、训练加速和推理加速，支持分布式高效训练和推理

提供高性价比昇腾算力

支持大规模异构集群及调度管理

- AI 高效迁移

提供全流程云化昇腾迁移工具链，支撑用户 AI 业务全栈国产化

提供迁移专业服务

ModelArts 进行模型训练及应用的主要流程为： OBS 存储对象建立→源码修改→数据集与源码上传→模型训练→模型优化→模型转换→开发板设置→开发板推理→结果展示

本实验分成如下步骤完成：

1. OBS 存储对象建立	在华为云云存储平台建立存储对象
2. 源码修改	依照平台要求修改源码文件
3. 数据集与源码上传	将修改好的源码的和数据集上传至 OBS 桶
4. 模型训练	在 ModelArts 平台中选择合适的参数与路径进行模型训练
5. 模型优化	调整超参数进行模型优化
6. 模型转换	在 OBS 指定路径中下载模型文件，将 pytorch 框架的模型转换为可以在昇腾开发板推理的 om 模型
7. 开发板相机设置	在昇腾开发板上操作工业相机
8. 开发板推理	在昇腾开发板上编写代码使用 npu 资源推理 om 模型
9. 结果展示	运行编写的程序，在昇腾开发板上实现使用工业相机识别手写字体

7.2. 操作以及演示

7.2.1. 数据集要求

- 文件名规范，不能有中文，不能有+、空格、制表符。
- 保证图片质量：不能有损坏的图片；目前支持的格式包括 jpg、jpeg、bmp、png。

folder	是	表示数据源所在目录。
filename	是	被标注文件的文件名。 表示图像的像素信息。
size	是	width: 必选字段，图片的宽度。 height: 必选字段，图片的高度。
segmented	是	depth: 必选字段，图片的通道数。 表示是否用于分割。 表示物体检测信息，多个物体标注会有多个 object 体。
object	是	name: 必选字段，标注内容的类别。 pose: 必选字段，标注内容的拍摄角度。 truncated: 必选字段，标注内容是否被截断（0 表示完整）。 occluded: 必选字段，标注内容是否被遮挡（0 表示未遮挡）。 difficult: 必选字段，标注目标是否难以识别（0 表示容易识别）。 confidence: 可选字段，标注目标的置信度，取值范围 0-1 之间。 bndbox: 必选字段，标注框的类型，标注信息请参见 表 2 。

- 不要把明显不同的多个任务数据放在同一个数据集内。
- 为了保证模型的预测准确度，训练样本跟真实使用场景尽量相似。
- 为保证模型的泛化能力，数据集尽量覆盖可能出现的各种场景。
- 物体检测数据集中，如果标注框坐标超过图片，将无法识别该图片为已标注图片。

PASCAL VOC 格式说明

type	形状	标注信息
bndbox	矩形框	左上和右下两个点坐标。
		<xmin>100<xmin>
		<ymin>100<ymin>
		<xmax>200<xmax>
		<ymax>200<ymax>

7.2.2. OBS 上传文件规范

- 如不需要提前上传训练数据，请创建一个空文件夹用于存放工程后期生成的文件。
如：“/bucketName/data-cat”。
- 如需要提前上传待标注的图片，请创建一个空文件夹，然后将图片文件保存在该文件夹下，图片的目录结构如：“/bucketName/data-cat/cat.jpg”。
- 如您将已标注好的图片上传至 OBS 桶，请按照如下规范上传。
- 物体检测数据集要求用户将标注对象和标注文件存储在上一级目录，并且一一对应。
例如标注对象文件名为“IMG_20180919_114745.jpg”，那么标注文件的文件名应为“IMG_20180919_114745.xml”。
- 物体检测的标注文件需要满足 PASCAL VOC 格式。
- 只支持 JPG、JPEG、PNG、BMP 格式的图片，在 OBS 管理控制台上传时，单张图片的大小不能超过 5MB，单次上传的图片总大小不能超过 8MB，数据量大时推荐使用 OBS Browser+上传。
- 标签名是由中文、大小写字母、数字、中划线或下划线组成，且不超过 32 位的字符串。

数据集需要创建一个 OBS 桶，然后将图片用一个桶里的文件夹存放，参照如下所示：

桶名称：demo-yolo

文件夹：images

对象存储 / demo-yolo / images

对象

已删除对象 碎片

对象是数据存储的基本单位。在OBS中文件和文件夹都是对象。您可以上传任何类型（文本、图片、视频等）的文件，并在桶中对这些文件进行管理。[了解更多](#)

上传对象 新建文件夹 恢复 删除 修改存储类别

输入对象名前缀搜索

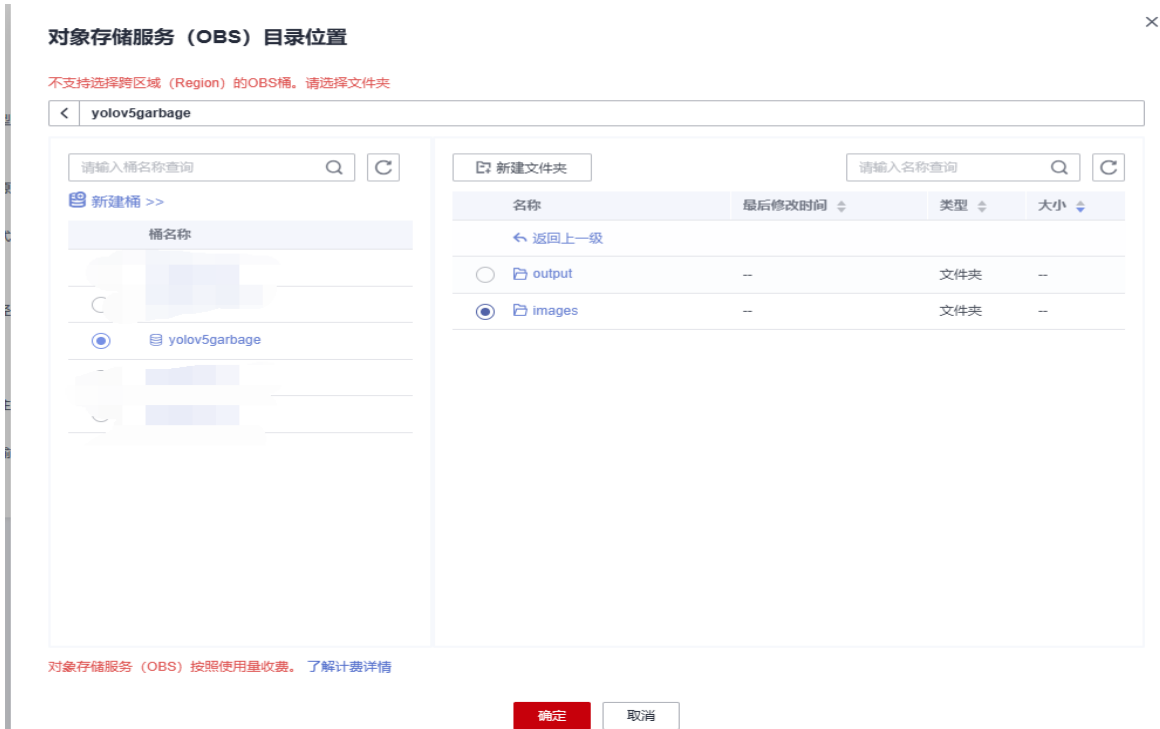
☐	对象名称	存储类别	大小	加密状态	恢复状态	最后修改时间	操作
返回上一级							
☐	0126f4833c23e928c5c8009c78d41CEE-fuben-fu...	标准存储	201.05 KB	未加密	--	2023/12/21 14:45:03 GMT+08:00	下载 分享 更多
☐	105-fuben.jpg	标准存储	411.47 KB	未加密	--	2023/12/21 14:45:02 GMT+08:00	下载 分享 更多
☐	104-fuben.jpg	标准存储	411.47 KB	未加密	--	2023/12/21 14:45:02 GMT+08:00	下载 分享 更多
☐	105-fuben-fuben.jpg	标准存储	411.47 KB	未加密	--	2023/12/21 14:45:02 GMT+08:00	下载 分享 更多
☐	106-fuben.jpg	标准存储	405.15 KB	未加密	--	2023/12/21 14:45:02 GMT+08:00	下载 分享 更多
☐	104-fuben-fuben.jpg	标准存储	411.47 KB	未加密	--	2023/12/21 14:45:02 GMT+08:00	下载 分享 更多
☐	103-fuben-fuben.jpg	标准存储	411.47 KB	未加密	--	2023/12/21 14:45:02 GMT+08:00	下载 分享 更多
☐	103-fuben.jpg	标准存储	411.47 KB	未加密	--	2023/12/21 14:45:02 GMT+08:00	下载 分享 更多
☐	0126f4833c23e928c5c8009c78d41CEE-fuben3.jpg	标准存储	201.05 KB	未加密	--	2023/12/21 14:45:02 GMT+08:00	下载 分享 更多
☐	103.jpg	标准存储	405.15 KB	未加密	--	2023/12/21 14:45:02 GMT+08:00	下载 分享 更多
☐	0126f4833c23e928c5c8009c78d41CEE-fuben2-f...	标准存储	201.05 KB	未加密	--	2023/12/21 14:45:02 GMT+08:00	下载 分享 更多

7.2.3. 创建项目

1. 登录 ModelArts 管理控制台，在左侧导航栏单击“自动学习”，进入新版自动学习页面。
2. 在您需要的自动学习项目列表中，单击“创建项目”，进入创建自动学习项目界面。
3. 创建项目



4. 在创建自动学习项目页面，计费模式默认“按需计费”。
- 在下图此位置选择数据的位置。



选择相应的名称和导入路径



创建物体检测

[返回自动学习](#)

★ 计费模式

按需计费

★ 名称

ExeML_c809

描述

质量检测

4/500

★ 数据集

瓶盖

创建数据集

★ 输出路径

/yolov5quality/output_path/

选择

★ 训练规格

[限时免费]GPU: 1*NVIDIA-V100(32GB) | CP...

参数	说明
“名称”	项目的名称。名称只能包含数字、字母、下划线和中划线，长度不能超过 64 位且不能为空。名称请以字母开头。 名称不允许重复。
“描述”	对项目的简要描述。
“数据集”	可选择“已有数据集”，或单击“创建数据集”前往新建数据集。 “已有数据集”：您可以从 ModelArts 数据管理中选择一个同类型的数据集，用于创建自动学习项目。 “数据集”的下拉框中，仅展示同类型的数据集供选择。 创建数据集：前往创建数据集页面创建一个新的数据集。具体可参考如何创建数据集。
“输出路径”	选择自动学习数据输出的统一 OBS 路径。说明：“输出路径”是存储自动学习在运行过程中所有产物的路

径。

选择自动学习训练节点所使用的资源规格，将会根据不同的规格计费。

“训练规格”

GPU: 8*NVIDIA-V100(32GB) | CPU: 72 核
512GBGPU: 1*NVIDIA-V100(32GB) | CPU: 8 核
64GB[限时费]GPU: 1*NVIDIA-V100(32GB) | CPU: 8 核 64GB

说明: 只有北京四区域支持限时免费规格。若您购买了套餐包, 可优先选择您对应规格的套餐包, 在“配置费用”处会显示您的套餐余量, 以及超出的部分如何计费, 请您关注, 避免造成不必要的资源浪费。

1. 单击“创建项目”，图像分类项目创建成功后页面自动跳转到“自动学习工作流”。
2. 物体检测项目的工作流，将依次运行如下节点：
 - 1) 数据标注：对您的数据进行标注情况确认。
 - 2) 数据集版本发布：将已完成标注的数据进行版本发布。
 - 3) 数据校验：对您的数据集的数据进行校验，是否存在数据异常。
 - 4) 物体检测：将发布好的数据集版本进行训练，生成对应的模型。
 - 5) 模型注册：将训练后的结果注册到模型管理中。
 - 6) 服务部署：将生成的模型部署为在线服务。

7.2.4. 查找创建好的项目

在自动学习总览页，您可以通过搜索框，根据自动学习的属性类型（项目名称）快速搜索过滤到相应的工作流，可节省您的时间。

1. 登录 ModelArts 管理控制台，在左侧导航栏选择自动学习，进入自动学习总览页面。
2. 在自动学习列表上方的搜索框中，根据您需要的属性类型，例如，名称、状态、项目类型或标签，过滤出相应的工作流。
3. 属性类型



- 单击搜索框右侧的  按钮，可选择自动学习的基础设置，需要的显示列。
- 表格内容折行：默认为关闭状态，启用此能力可让表格内容自动折行，禁用此功能可截断文本。
- 操作列：默认为开启状态，启用此能力可让操作列固定在最后一列永久可见。
- 自定义显示列：默认所有显示项全部勾选，您可以根据实际需要定义您的显示列。

设置

基础设置

- 表格内容折行 ☐ 自动折行
启用此能力可让表格内容自动折行，禁用此功能可截断文本。
- 操作列 ☒ 列固定
启用此能力可让操作列固定在最后一列永久可见。

自定义显示列



- ☐ 名称（默认）
- ☒ 状态
- ☒ 项目类型
- ☒ 当前节点
- ☒ 标签
- ☒ 运行次数
- ☒ 创建时间
- ☒ 描述
- ☐ 操作（默认）

单击“确定”即可按照设置好的显示列进行显示。同时可支持对自动学习项目显示页进行排序，单击表头中的箭头 ，就可对该列进行排序。

7.2.5. 图片标注

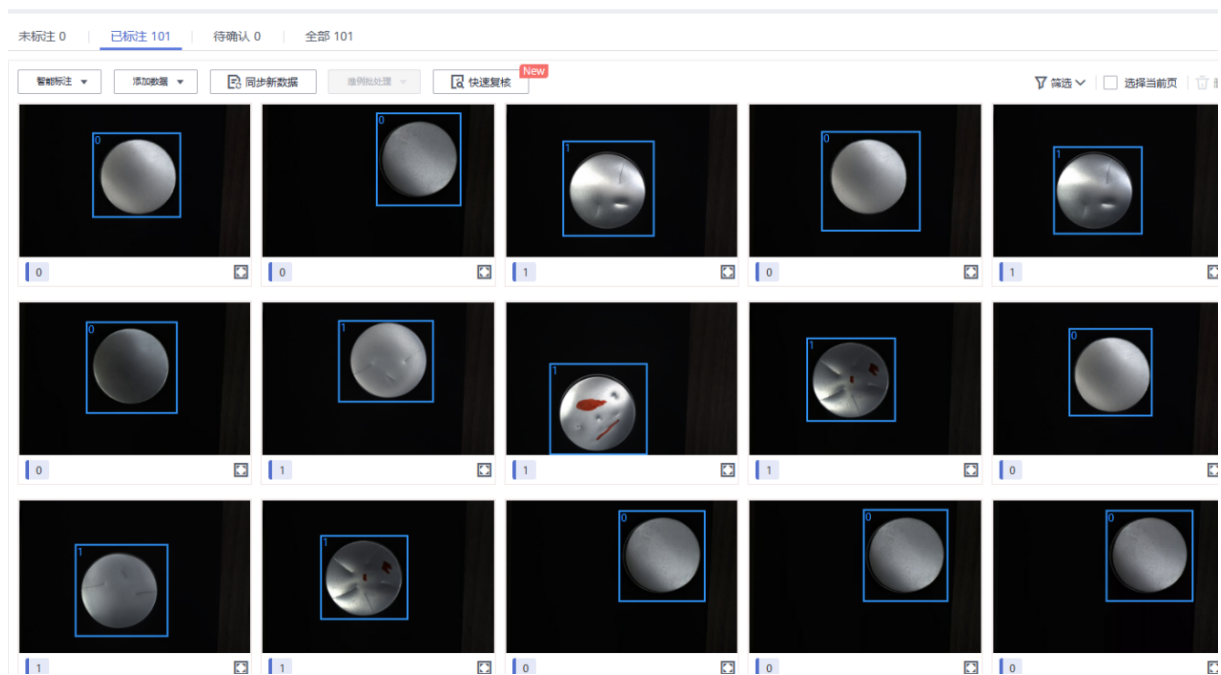
- 在新版自动学习页面单击“实例详情”按钮，前往数据标注页面。单击任意一张图片，进入图片标注界面。

数据标注

运行状况

 请前往 [实例详情](#) 页面进行标注

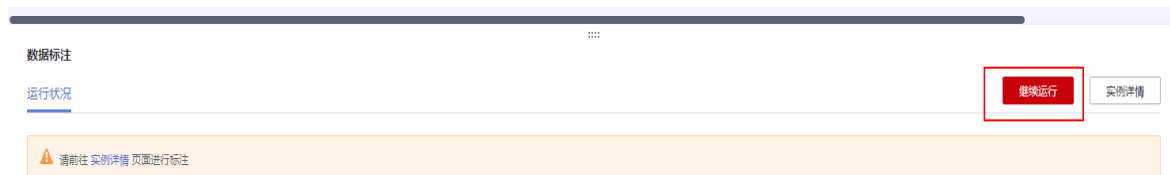
2. 用鼠标框选图片中的物体所在区域，然后在弹出的对话框中选择标签颜色，输入标签名称，例如此示例中的“yunbao”，按“Enter”键完成此标签的添加。标注完成后，左侧图片目录中此图片的状态将显示为“已标注”。
3. 您可以在图片上方或下方单击左右切换键，或者按键盘的左右方向键，选择其他图片，重复上述操作继续进行图片标注。如果一张图片有多个物体，您可以标注多处。
4. 同一个物体检测自动学习项目内，可以增加多个标签，且标签可选择不同颜色，方便识别。使用鼠标完成物体框选后，在弹出的对话框中，选择新的颜色，输入新的标签名称，即可添加一个新的标签。
5. 自动学习项目中，物体检测仅支持矩形标注框。在“数据管理”功能中，物体检测类型的数据集，支持更多类型的标注框。
6. 在标注窗口中，您可以滚动鼠标，放大或缩小图片，方便您快速定位到物体位置。
7. 当图片目录中所有图片都完成标注后，返回“自动学习 workflow”页面，单击“继续运行”按钮，workflow 将会自动发布数据标注版本，并进行下一步训练步骤。



7.2.6. 继续运行

完成数据的确认之后，返回新版自动学习的页面，在数据标注节点单击“继续运行”，工作流将会继续依次运行直到所有节点运行成功。

继续运行



7.2.7. 操作步骤

1. 在新版自动学习页面，单击项目名称进入运行总览页面，单击“数据标注”节点的“实例详情”进入数据标注页面，完成数据标注。
2. 图 1 完成数据标注



3. 返回新版自动学习页面，单击数据标注节点的“继续运行”，然后等待工作流按顺序

进入训练节点。

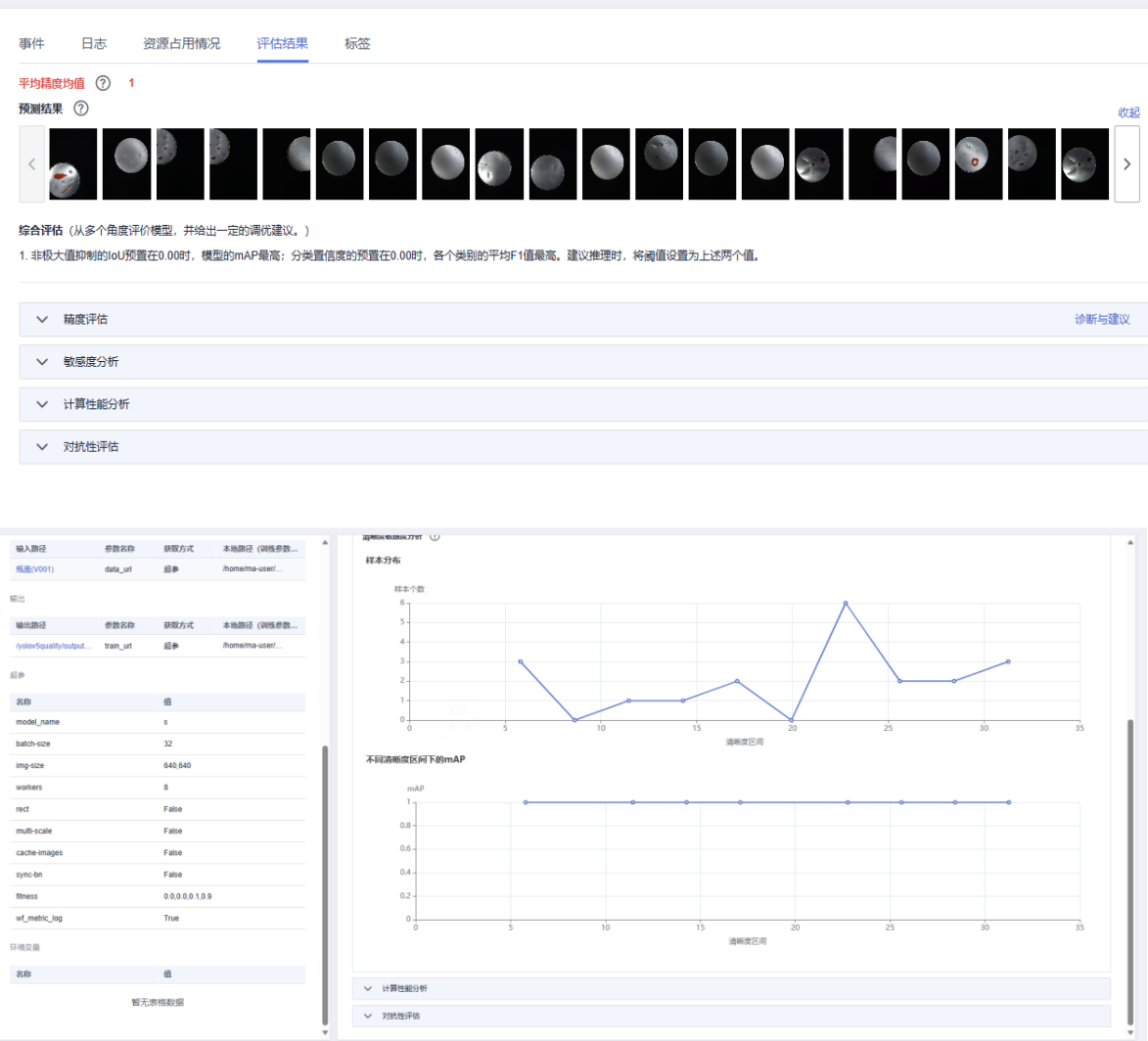
- 4. 模型将会自动进入训练，无需人工介入，训练时间相对较长，建议您耐心等待。如果关闭或退出此页面，系统仍然在执行训练操作。
- 5. 在“物体检测”节点中，待训练状态由“运行中”变为“运行成功”，即完成模型的自动训练。
- 6. 运行成功

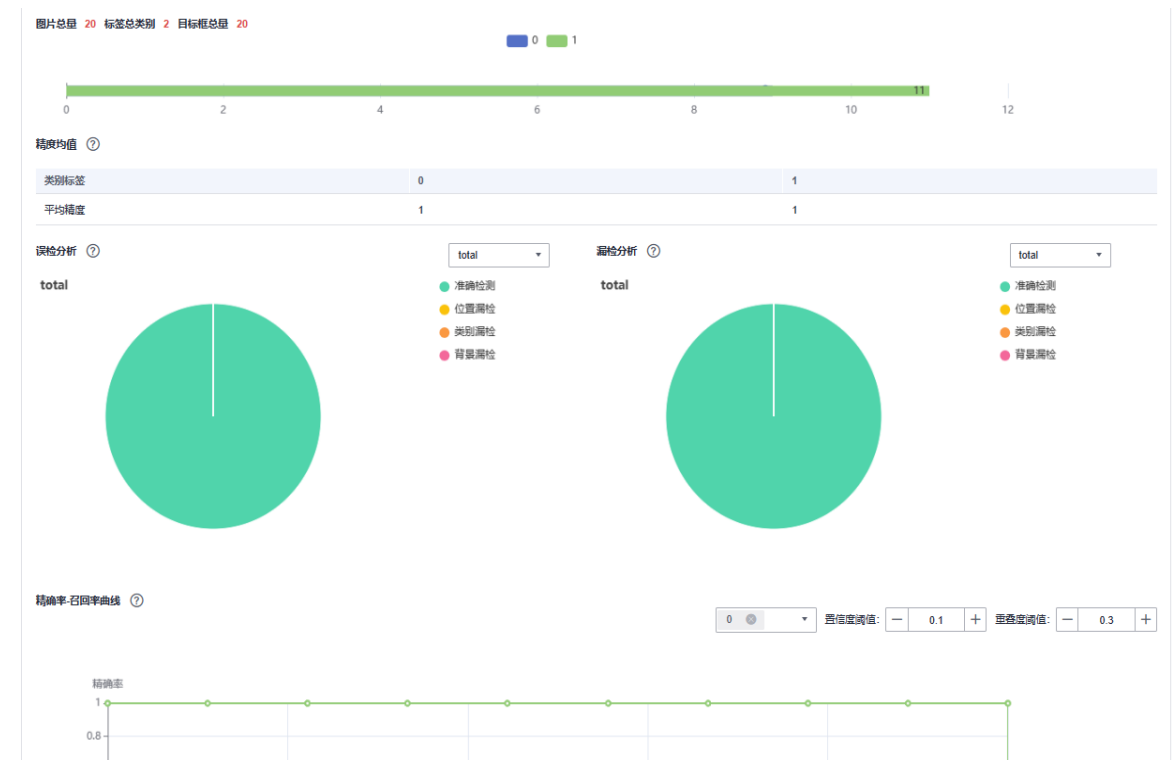


参数	说明
recall: 召回率	被用户标注为某个分类的所有样本中，模型正确预测为该分类的样本比率，反映模型对正样本的识别能力。
precision: 精确率	被模型预测为某个分类的所有样本中，模型正确预测的样本比率，反映模型对负样本的区分能力。
accuracy: 准确率	所有样本中，模型正确预测的样本比率，反映模型对样本整体的识别能力。
f1: F1 值	F1 值是模型精确率和召回率的加权调和平均，用于评价模型的好坏，当 F1 较高时说

明模型效果较好。

7. 训练完成后，您可以单击物体检测节点上方的按钮，查看相关指标信息，如“准确率”、“评估结果”等。





8. 模型评估报告

评估报告

指标数据 图表

Metric	Value
accuracy	0.93
balanced_accuracy	0.77
f1	0.5
mcc	0.509
precision	1
recall	0.33
roc_auc	0.952

7.2.8. 部署上线

部署上线操作即将模型部署为在线服务，并且提供在线的测试 UI 与监控能力。完成模型训练后，可选择准确率理想且训练状态为“运行成功”的版本部署上线。具体操作步骤如下。

1. 在“运行节点”页面中，待服务部署节点的状态变为“等待输入”时，双击“服务部署”进入配置详情页，完成资源的参数配置操作。

2. 服务部署节点状态

运行状况

属性

状态

● 等待输入

启动时间

2023/01/04 10:54:38 GMT+08:00

运行时长

00:00:01

3. 在服务部署页面，选择部署上线使用的资源规格。

4. 资源规格

服务部署

运行状况

属性

状态

● 等待输入

启动时间

2023/06/21 10:16:21 GMT+08:00

运行时长

00:00:01

更新时间

2023/06/21 10:16:23 GMT+08:00

输入

AI应用来源

我的AI应用

来自工作流节点

选择AI应用及版本

zllaccodhi (同步请求)

0.0.1 (正常)

C

资源池

公共资源池

专属资源池

计算节点规格

限时免费|CPU: 1 核 4GB

适合场景：CPU资源规格，满足大多数模型的运行体验

配置费用

¥ 0.00/小时

分流 (%)

-

100

+

计算节点个数

-

1

+

环境变量

增加环境变量

参数

* model_name

zllaccodhi

请输入一个1-64位且只包含大小写字母、中文、数字、中划线或者下划线的名称。工作流第一次运行建议填写新的模型名称，后续运行会自动在该模型上新增版本

* 是否自动停止

开启该选项后，在线服务的运行时间将在您指定的时间后，自动停止，同时服务计费停止

1小时

2小时

4小时

6小时

自定义

5. AI 应用：默认为生成的 AI 应用。

6. AI 应用版本：自动匹配当前使用的 AI 应用版本，支持选择版本。

7. 资源池：默认公共资源池。

8. 分流：默认为 100，输入值必须是 0-100 之间。

9. 计算节点规格：支持 GPU: 1*NVIDIA-P4(8GB)| CPU: 8 核 32GB 、

10. GPU: 1*NVIDIA-T4(16GB)| CPU: 8 核 32GB、CPU: 2 核 8GB、

11. GPU: 1*NVIDIA-P4(8GB)| CPU: 2 核 8GB 以及限时免费规格。

12. 计算节点个数：默认为 1，输入值必须是 1-5 之间的整数。

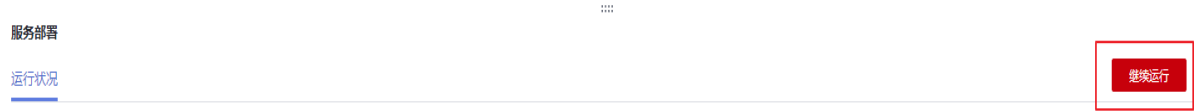
13. 是否自动停止：启用该参数并设置时间后，服务将在指定时间后自动停止。如果不启用此参数，在线服务将一直运行，同时一直收费，自动停止功能可以帮您避免产

生不必要的费用。默认开启自动停止功能，且默认值为“1 小时后”。

14. 目前支持设置为“1 小时后”、“2 小时后”、“4 小时后”、“6 小时后”、“自定义”。如果选择“自定义”的模式，可在右侧输入框中输入 1~24 范围内的任意整数。

15. 完成资源配置后，单击“继续运行”，服务部署节点将继续运行，直至状态变为“运行成功”，至此，已将 AI 应用部署为在线服务。

继续运行



7.2.9. 服务测试

1. 服务部署节点运行成功后，单击“实例详情”可跳转至对应的在线服务详情页面。单击“预测”页签，进行服务测试。

2. 服务测试



- 您也可以“部署上线>在线服务”页面，选择对应的服务类型，进入“在线服务”页面，单击目标服务“操作”列的“预测”，进行服务测试，测试方法和下方陈述操作步骤一致。
- 您可以通过调用代码对服务进行测试，根据部署服务类型的不同。
- 下面的测试，是您在自动学习物体检测项目页面将模型部署上线之后进行服务测试的操作步骤。
- 模型部署完成后，“服务部署”节点，单击“实例详情”按钮，进入服务预测界面，在“预测”页签单击“上传”，选择本地图片进行测试。
- 如下图，点击上传图片



- 单击“预测”进行测试，预测完成后，右侧“预测结果”区域输出结果。如模型准确率不满足预期，可在“数据标注”页签中添加图片并进行标注，重新进行模型训练及部

署上线。

目前只支持 jpg、jpeg、bmp、png 格式的图片。

调用指南

预测

配置更新记录

监控信息

事件

日志

标签

请求路径:

/

选择预测图片文件

上传

重新预测

上传的预测文件大小不能超过8MB，否则上传文件会失败。详情请参见[服务预测请求体大小限制](#)。

预测图片预览



预测结果显示

预测成功

```
1 {
2   "detection_classes": [
3     "1"
4   ],
5   "detection_boxes": [
6     [
7       62,
8       806,
9       1186,
10      1949
11     ]
12   ],
13   "detection_scores": [
14     0.8912829160690308
15   ]
16 }
```

7.3. 自定义训练教程——手势识别

7.3.1. 使用预训练框架

事先在 OBS 桶中，新建用于上传代码和模型存储的文件夹

← → ↑

桶列表 / test-dog-gesture

华北-北京四

|

桶内对象总数: 353

|

存储总

↑ 上传

新建文件夹

↓ 下载

📄 复制

☐

对象名称

⌵

☐

📁 output

☐

📁 yolov5-7.0

然后上传 PC 端的工程文件

←

→

↑

桶列表 / test-dog-gesture / yolov5-7.0

📍 华北-北京四

|

桶内对象总数: 353

|

存储总用量: 134.13 MB

⬆️ 上传

📁 新建文件夹

⬇️ 下载

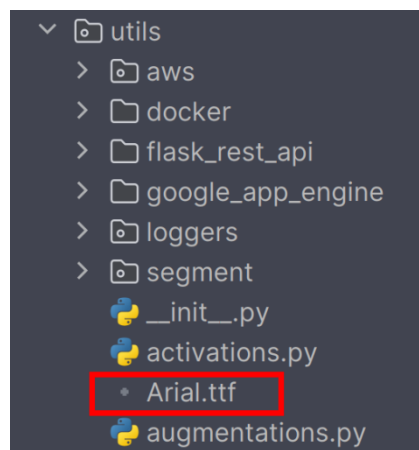
📄 复制

更多 ▾

☐	对象名称	存储类别	大小
☐	📁 .github	--	--
☐	📁 .idea	--	--
☐	📁 __pycache__	--	--
☐	📁 classify	--	--
☐	📁 data	--	--
☐	📁 dataset	--	--
☐	📁 models	--	--
☐	📁 runs	--	--
☐	📁 segment	--	--
☐	📁 utils	--	--
☐	📄 .dockerignore	标准存储	3.61 KB
☐	📄 .gitattributes	标准存储	75 bytes

其中我们需要做出如下修改：

1. 修改字体设置，防止在部署中因为网络原因报错



Error! Unknown document property name.

手动在浏览器的网址中输入: <https://ultralytics.com/assets/Arial.ttf>, 下载字体文件, 然后放在图中显示的目录下, 然后在 `general.py` 文件中做出如下修改, 将字体设置为本地

```
general.py × train.py detect.py data.yaml export.py
488 def check_font(font=FONT, progress=False):
489     # Download font to CONFIG_DIR if necessary
490     font = Path(font)
491     file = CONFIG_DIR / font.name
492     local_font_path = Path('yolov5-7.0/utils') / font.name
493     if local_font_path.exists():
494         return local_font_path
495     # if not font.exists() and not file.exists():
496     #     url = f'https://ultralytics.com/assets/{font.name}'
497     #     LOGGER.info(f'Downloading {url} to {file}...')
498     #     torch.hub.download_url_to_file(url, str(file), progress=progress)
499
```

1. `def check_font(font=FONT, progress=False):`
2. `# Download font to CONFIG_DIR if necessary`
3. `font = Path(font)`
4. `file = CONFIG_DIR / font.name`
5. `local_font_path = Path('yolov5-7.0/utils') / font.name`
6. `if local_font_path.exists():`
7. `return local_font_path`
8. `# if not font.exists() and not file.exists():`
9. `# url = f'https://ultralytics.com/assets/{font.name}'`
10. `# LOGGER.info(f'Downloading {url} to {file}...')`
11. `# torch.hub.download_url_to_file(url, str(file), progress=progress)`

2. 设置模型的保存路径

```
general.py  train.py x detect.py  data.yaml  export.py
381
382     'opt': vars(opt),
383     # 'git': GIT_INFO, # {remote, branch, commit} if a git repo
384     'date': datetime.now().isoformat()
385
386     # Save last, best and delete
387     torch.save(ckpt, last)
388     if best_fitness == fi:
389         torch.save(ckpt, best)
390     if opt.save_period > 0 and epoch % opt.save_period == 0:
391         torch.save(ckpt, w / f'epoch{epoch}.pt')
392
393     # 新增
394     model_save_path = opt.train_url
395     print(model_save_path)
396     save_name = os.path.join(model_save_path, 'model.pt')
397     torch.save(ckpt, save_name)
```

1. # 新增
2. model_save_path = opt.train_url
3. print(model_save_path)
4. save_name = os.path.join(model_save_path, 'model.pt')
5. torch.save(ckpt, save_name)

再设置一个参数

```
general.py  train.py x detect.py  data.yaml  export.py
441
442
443 def parse_opt(known=False):
444     parser = argparse.ArgumentParser()
445     parser.add_argument('--weights', type=str, default=ROOT / 'yolov5s.pt', help='initial weights path')
446     parser.add_argument('--cfg', type=str, default='', help='model.yaml path')
447     parser.add_argument('--data', type=str, default=ROOT / 'data/coco128.yaml', help='dataset.yaml path')
448     parser.add_argument('--train_url', type=str, default="weights", help='Train url for ModelArts')
449     parser.add_argument('--hyp', type=str, default=ROOT / 'data/hyps/hyp.scratch-low.yaml', help='hyperparameters path')
450     parser.add_argument('--epochs', type=int, default=100, help='total training epochs')
451     parser.add_argument('--batch-size', type=int, default=16, help='total batch size for all GPUs, -1 for autobatch')
452     parser.add_argument('--imgsz', '--img', '--img-size', type=int, default=640, help='train, val image size (pixels)')
453     parser.add_argument('--rect', action='store_true', help='rectangular training')
454     parser.add_argument('--resume', nargs='?', const=True, default=False, help='resume most recent training')
```

1. parser.add_argument('--
train_url', type=str, default="weights", help='Train url for ModelArts'
)

3. Git 相关检测操作去除 在 modelarts 建立的环境中 git 相关操作可能因为网络原因导致脚本运行失败, git 相关操作在本实验中无用处, 可以去除, 先找到 train.py 中的导包部分, 再找红色圈出来的

```
general.py  train.py ×  detect.py  data.yaml  export.py
52         get_latest_run, increment_path, init_seeds, intersect_dicts, labels_to_class_weights,
53         labels_to_image_weights, methods, one_cycle, print_args, print_mutation, strip_optimizer,
54         yaml_save)
55 from utils.loggers import Loggers
56 from utils.loggers.comet.comet_utils import check_comet_resume
57 from utils.loss import ComputeLoss
58 from utils.metrics import fitness
59 from utils.plots import plot_evolve
60 from utils.torch_utils import (EarlyStopping, ModelEMA, de_parallel, select_device, smart_DDP, smart_optimizer,
61                               smart_resume, torch_distributed_zero_first)
62
63 LOCAL_RANK = int(os.getenv('LOCAL_RANK', -1)) # https://pytorch.org/docs/stable/elastic/run.html
64 RANK = int(os.getenv('RANK', -1))
65 WORLD_SIZE = int(os.getenv('WORLD_SIZE', 1))
66 # GIT_INFO = check_git_info()
67
68 os.environ["KMP_DUPLICATE_LIB_OK"]="TRUE"
69
```

1. `# GIT_INFO = check_git_info()`
- 2.
3. `os.environ["KMP_DUPLICATE_LIB_OK"]="TRUE"`

```
general.py  train.py ×  detect.py  data.yaml  export.py
376
377         'best_fitness': best_fitness,
378         'model': deepcopy(de_parallel(model)).half(),
379         'ema': deepcopy(ema.ema).half(),
380         'updates': ema.updates,
381         'optimizer': optimizer.state_dict(),
382         'opt': vars(opt),
383         # 'git': GIT_INFO, # {remote, branch, commit} if a git repo
384         'date': datetime.now().isoformat())
```

1. `# 'git': GIT_INFO, # {remote, branch, commit} if a git repo`

注意：确保上面的操作都已完成，才可进行下面的操作。

7.3.2. 文件上传 OBS

事先在 OBS 桶中，新建和上传代码、模型存储文件夹，文件夹和桶名称如下所示：



上传 ModelArts 端的工程文件。

7.3.3. 训练作业

进行训练之前，确保前面的所有步骤已经完成；在“训练管理”->“训练作业”，点击右上角的“创建训练作业”



新建一个训练作业，名称任意即可，同时按照下面配置，配置好路径和参数，在 modelarts 平台配置如下，训练作业名称：

* 名称	job-a931-copy-7122	✓
描述	狗	

1/256

自定义算法的启动文件和数据文件路径：

Error! Unknown document property name.

代码目录

不支持选择跨区域（Region）的OBS桶。只能选择对象存储服务（OBS）桶下的文件夹。

<

test-dog-gesture

请输入桶名称查询

Q

C

新建桶 >>

桶名称

test-dog-gesture

新建文件夹

请输入名称查询

Q

C

名称	最后修改时间	类型	大小
返回上一级			
yolov5-7.0	--	文件夹	--
output	--	文件夹	--

启动文件

不支持选择跨区域（Region）的OBS桶。请选择英文路径和英文命名文件。

<

test-dog-gesture / yolov5-7.0

请输入桶名称查询

Q

C

新建桶 >>

桶名称

test-dog-gesture

新建文件夹

请输入名称查询

Q

C

名称	最后修改时间	类型	大小
返回上一级			
tutorial.ipynb	2024/04/10 15:46:51 GM...	文件	53842 B
yolov5n.pt	2024/04/10 15:46:51 GM...	文件	4062133 B
train.py	2024/04/10 15:55:00 GM...	文件	33904 B
val.py	2024/04/10 15:46:51 GM...	文件	20282 B
setup.cfg	2024/04/10 15:46:51 GM...	文件	1686 B
requirements.txt	2024/04/10 15:46:51 GM...	文件	1541 B
yolov5s.pt	2024/04/10 15:46:51 GM...	文件	1480843...
utils	--	文件夹	--
segment	--	文件夹	--
runs	--	文件夹	--

上一页

下一页

* 创建方式

自定义算法

我的算法

我的订阅

?

* 启动方式

预置框架

自定义

PyTorch

pytorch_1.8.0-cuda_10.2-py_3...

* 代码目录

/test-dog-gesture/yolov5-7.0/

选择

?

* 启动文件

/test-dog-gesture/yolov5-7.0/train.py

选择

?

本地代码目录

/home/ma-user/modelarts/user-job-dir

工作目录

/home/ma-user/modelarts/user-job-dir

×

选择

输入参数:

输入 ?

weights

/test-dog-gesture/yolov5-7.0/yolov5s.pt

数据集

数据存储位置

获取方式

☒ 超参 ☐ 环境变量

--weights=/home/ma-user/modelarts/inputs/weights_0

data

/test-dog-gesture/yolov5-7.0/dataset/data.yaml

数据集

数据存储位置

获取方式

☒ 超参 ☐ 环境变量

--data=/home/ma-user/modelarts/inputs/data_1

cfg

/test-dog-gesture/yolov5-7.0/models/yolov5s.yaml

数据集

数据存储位置

获取方式

☒ 超参 ☐ 环境变量

--cfg=/home/ma-user/modelarts/inputs/cfg_2

输出参数:

输出 ?

train_url

/test-dog-gesture/output/

数据存储位置

获取方式

☒ 超参 ☐ 环境变量

--train_url=/home/ma-user/modelarts/outputs/train_url_0

预下载至本地目录 ? ☒ 不下载 ☐ 下载

训练用到的超参数:

超参

img

=

640

batch

=

16

epoch

=

5

⊕ 增加超参

点击提交

信息确认

作业名	训练信息	价格
job-a931-copy-7122	算法	--
	预置镜像	PyTorch pytorch_1.8.0-cuda_10.2-py_3.7-ubuntu_18.04-x86_64
	代码目录	/test-dog-gesture/yolov5-7.0/
	启动文件	/test-dog-gesture/yolov5-7.0/train.py
	本地代码目录	/home/ma-user/modelarts/user-job-dir
	工作目录	/home/ma-user/modelarts/user-job-dir
	输入	weights=/test-dog-gesture/yolov5-7.0/yolov5s.pt data=/test-dog-gesture/yolov5-7.0/dataset/data.yaml cfg=/test-dog-gesture/yolov5-7.0/models/yolov5s.yaml
	输出	train_url=/test-dog-gesture/output/
	超参	img=640 batch=16 epoch=5
	规格	[限时免费] GPU: 1*NVIDIA-V100(32GB) CPU: 8 核 64GB 780GB
计算节点个数	1	¥ 0.00 /小时

确定

取消

<

job-a931-copy-7122

另存为算法

重建

创建AI应用

终止

作业ID2d1d21f1-404c-415a-86ac-ba3a4b5e729c

作业状态等待中

创建时间2024/04/10 17:12:09 GMT+08:00

运行时长00:00:00

描述

预置镜像PyTorch | pytorch_1.8.0-cuda_10.2-py_3.7-ubuntu_18.04-x86_64

算法名称--

代码目录/test-dog-gesture/yolov5-7.0/

启动文件/test-dog-gesture/yolov5-7.0/train.py

运行用户ID1102

本地代码目录/home/ma-user/modelarts/user-job-dir

工作目录/home/ma-user/modelarts/user-job-dir

事件日志资源占用情况评估结果标签

2024/04/10 17:12:09 — 2024/04/11 17:12:20

全部

请输入事件信息查询

事件类型	事件信息	事件发生时间
正常	[worker-0]耗时: 0.679秒] 训练环境预检完成。	2024/04/10 17:12:17 GMT+0
正常	[worker-0] 训练环境预检中。	2024/04/10 17:12:16 GMT+0
正常	[Pod: modelarts-job-2d1d21f1-404c-415a-86ac-ba3a4b5e729c-worker-0] Started: Started container	2024/04/10 17:12:16 GMT+0
正常	[Pod: modelarts-job-2d1d21f1-404c-415a-86ac-ba3a4b5e729c-worker-0] SuccessfullyCreate: Created container	2024/04/10 17:12:15 GMT+0
正常	[Pod: modelarts-job-2d1d21f1-404c-415a-86ac-ba3a4b5e729c-worker-0] Pulled: Successfully pulled image "swr.cn-north-4.my...	2024/04/10 17:12:15 GMT+0
正常	[Pod: modelarts-job-2d1d21f1-404c-415a-86ac-ba3a4b5e729c-worker-0] Pulling: pulling image "swr.cn-north-4.my...	2024/04/10 17:12:15 GMT+0
正常	[Pod: modelarts-job-2d1d21f1-404c-415a-86ac-ba3a4b5e729c-worker-0] SuccessfullyMountVolume: Successfully ...	2024/04/10 17:12:14 GMT+0
正常	[PodGroup: modelarts-job-2d1d21f1-404c-415a-86ac-ba3a4b5e729c] Scheduled: pod group is ready	2024/04/10 17:12:14 GMT+0
正常	[Pod: modelarts-job-2d1d21f1-404c-415a-86ac-ba3a4b5e729c-worker-0] Scheduled: Successfully assigned defau...	2024/04/10 17:12:14 GMT+0
告警	[PodGroup: modelarts-job-2d1d21f1-404c-415a-86ac-ba3a4b5e729c] Unschedulable: 0/0 tasks in gang unschedu...	2024/04/10 17:12:12 GMT+0

10

总条数: 12

< 1 2 >

跳转

页

等待完成

job-a931-copy-7122

作业ID2d1d21f1-404c-415a-86ac-ba3a4b5e729c

作业状态

已完成

创建时间2024/04/10 17:12:09 GMT+08:00

运行时长00:01:18

描述狗

预置镜像PyTorch | pytorch_1.8.0-cuda_10.2-py_3.7-ubuntu_18.04-x86_64

算法名称--

代码目录/test-dog-gesture/yolov5-7.0/

编辑代码

启动文件/test-dog-gesture/yolov5-7.0/train.py

运行用户ID1102

本地代码目录/home/ma-user/modelarts/user-job-dir

工作目录/home/ma-user/modelarts/user-job-dir

事件日志资源占用情况评估结果标签

2024/04/10 17:12:09 — 2024/04/10 17:12:09

事件类型	事件信息
正常	训练作业运行成功。
正常	[Job: modelarts-job-2d1d21f1-404c-415a-86ac-ba3a4b5e729c]
正常	[worker-0][耗时: 1.032秒] 训练输出 (参数名称: train_u
正常	[worker-0] 训练输出 (参数名称: train_u
正常	[worker-0] 训练结束。退出码0。
正常	[worker-0] GPU进程启动
正常	[worker-0][耗时: 8.730秒] Python依赖包
正常	[worker-0] 正在安装Python依赖包。导入
正常	[worker-0] 训练开始。
正常	训练作业开始运行。

10 总条数: 38 < 1 2 3 4 > 跳转 页

运行成功，然后在模型的保存路径中找到模型文件

桶列表 / test-dog-gesture / output

华北-北京四 | 桶内对象总数: 353 | 存储总用量: 134.13 MB

上传

新建文件夹

下载

复制

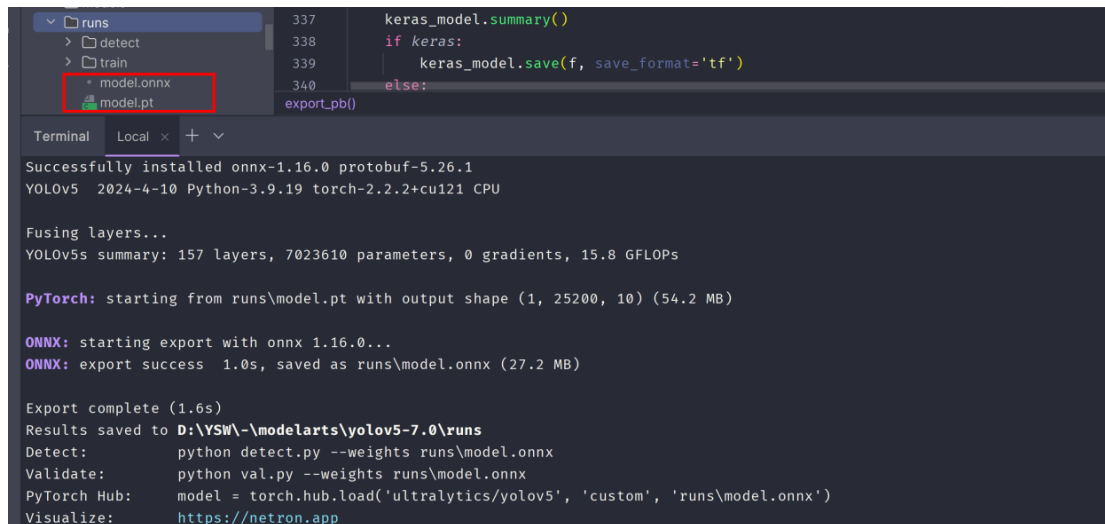
更多

对象名称	存储类别	大小
model.pt	标准存储	54.24 MB

7.3.4. 模型转换

在 obs 中将模型文件下载，然后上传项目文件中的 runs 目录下，然后运行：

```
1. python export.py --weights runs/model.pt --include onnx
```



```
337 keras_model.summary()
338 if keras:
339     keras_model.save(f, save_format='tf')
340 else:
    export_pbl()
```

Terminal Local x + v

Successfully installed onnx-1.16.0 protobuf-5.26.1
YOLOv5 2024-4-10 Python-3.9.19 torch-2.2.2+cu121 CPU

Fusing layers...
YOLOv5s summary: 157 layers, 7023610 parameters, 0 gradients, 15.8 GFLOPs

PyTorch: starting from runs\model.pt with output shape (1, 25200, 10) (54.2 MB)

ONNX: starting export with onnx 1.16.0...
ONNX: export success 1.0s, saved as runs\model.onnx (27.2 MB)

Export complete (1.6s)
Results saved to D:\VSW\...\modelarts\yolov5-7.0\runs

Detect: python detect.py --weights runs\model.onnx
Validate: python val.py --weights runs\model.onnx
PyTorch Hub: model = torch.hub.load('ultralytics/yolov5', 'custom', 'runs\model.onnx')
Visualize: <https://netron.app>

在 runs 目录中成功生成了 model.onnx

将 ONNX 模型文件上传到开发板

```
(base) root@davinci-mini:/opt# ll
total 27888
drwxr-xr-x  2 root root    4096 Apr 12 17:10 ./
drwxr-xr-x 24 root root    4096 Apr  8 2022 ../
-rw-r--r--  1 root root 28545135 Apr 12 17:10 model.onnx
```

运行下面命令

```
1. atc --model=model.onnx --framework=5 --output=model --
    soc_version=Ascend310B4
```

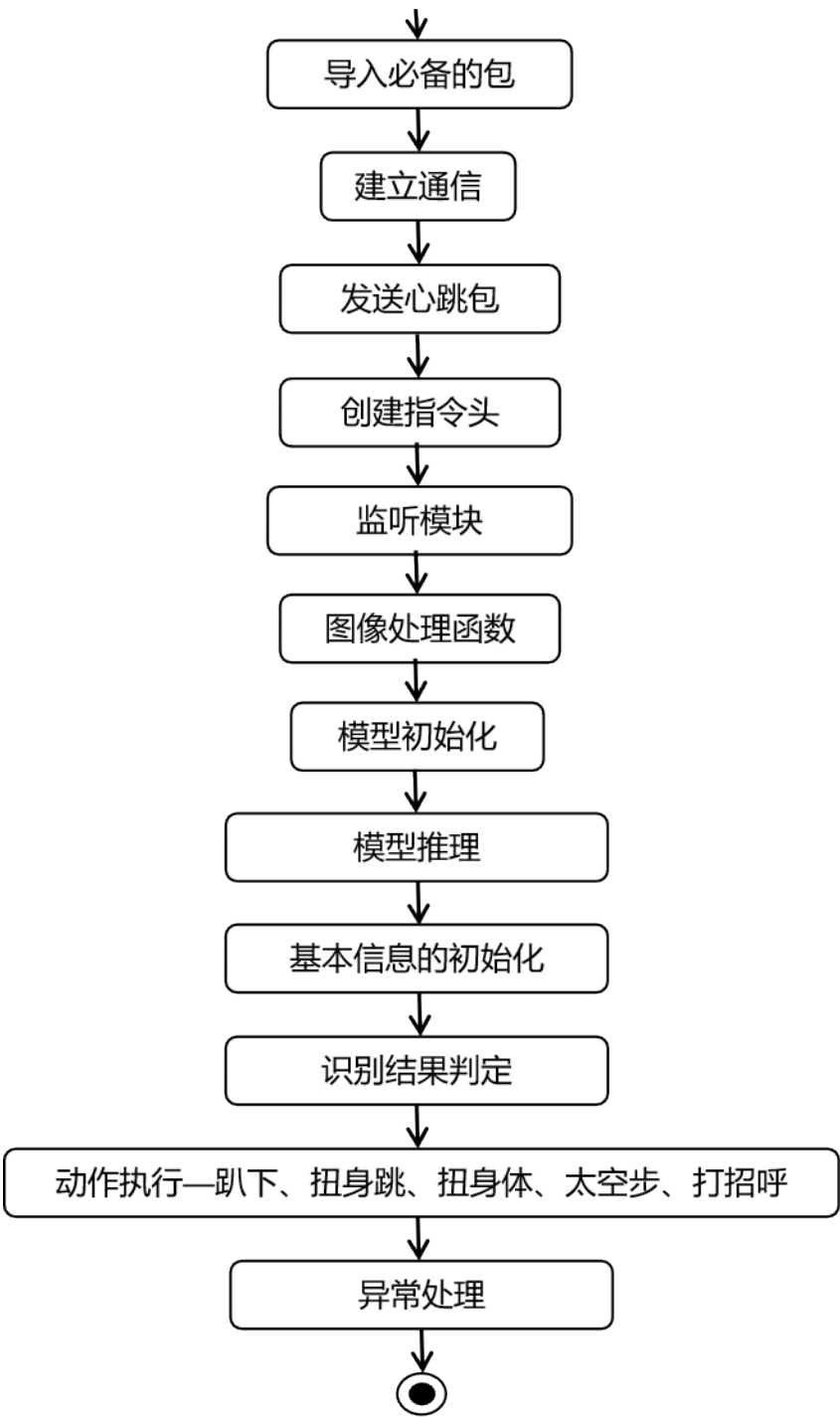
显示转换成功即可

```
(base) root@davinci-mini:/opt# atc --model=model.onnx --framework=5 --output=model --soc_version=Ascend310B4
ATC start working now, please wait for a moment.
...
ATC run success, welcome to the next use.
```

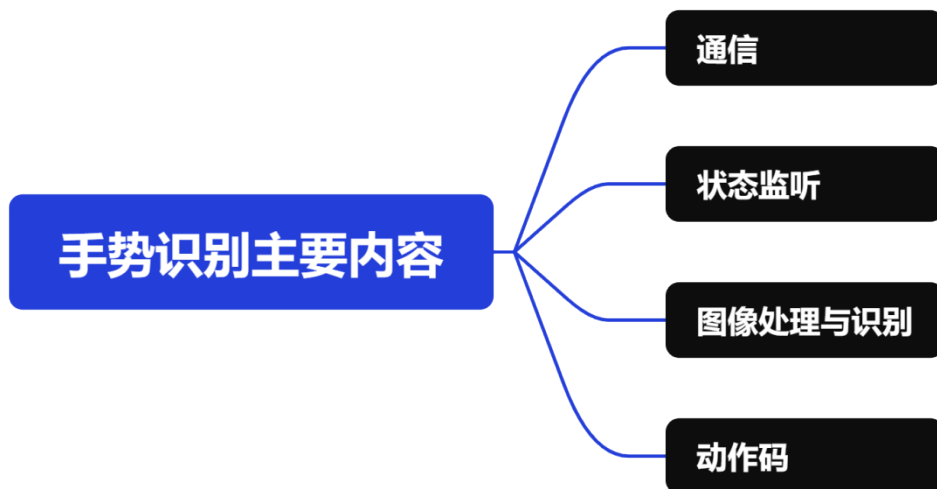
```
(base) root@davinci-mini:/opt# ll
total 42844
drwxr-xr-x  2 root root    4096 Apr 12 17:26 ./
drwxr-xr-x 24 root root    4096 Apr  8 2022 ../
-rw-r-----  1 root root    3022 Apr 12 17:26 fusion_result.json
-rw-----  1 root root 15307128 Apr 12 17:26 model.om
-rw-r--r--  1 root root 28545135 Apr 12 17:10 model.onnx
```

可以看到已经有.om 模型文件输出。

第八章 逻辑实现



8. 1. 导入必备的包



导入手势识别所用到的库，主要包括图像处理、深度学习、推理接口、时间处理、日志记录、类型注解、系统路径设置、相机图像获取、模型前后处理函数，以及与机器狗相关的特定功能模块。具体如下：

```
1.  #!/usr/bin/env python
2.  # -*- coding: utf-8 -*-
3.
4.  import cv2 # 图片处理三方库，用于对图片进行前后处理
5.  import numpy as np # 用于对多维数组进行计算
6.  import torch # 深度学习运算框架，此处主要用来处理数据
7.  from mindx.sdk import Tensor # mxVision 中的 Tensor 数据结构
8.  from mindx.sdk import base # mxVision 推理接口
9.  import time
10. import logging
11. from typing import *
12. import sys
13. sys.path.append('camera/')
14. from HKcamera import getImage
15. from det_utils import get_labels_from_txt, letterbox, scale_coord
    s, nms, draw_bbox # 模型前后处理相关函数
```

```

16.
17.  # 导入定义的机器狗相关的包
18.  from threading_utils.ThreadTemplates import *           # 线程的模板与基本轴指令的发送
19.  from sendcommand import SendToCommand, heartbeat        # 发送简单指令与心跳包
20.  from socketnetwork import network_utils                  # 通信函数
21.  from command.udp_command import *                       # 存放各种结构体与状态数据、指令
22.  from speeds.sportspeed import *                          # 运动精准控制的基础版模型
23.  from robotstatuswatcher.listener import *                # 监听机器狗状态

```

其中监听的函数包，放在了文件夹的 robotstatuswatcher/listener.py 中，后文给出解释。

8.2. 建立通信

简单控制第一步：建立通信链接，位于 socketnetwork/network_utils.py，用于创建一个 UDP 套接字，并设置机器狗的 IP 地址和端口号作为目标地址。该通信链接是所有后续控制和数据传输的基础。不仅仅确保了指令传输的可靠性和及时性，还实现对机器狗的远程控制和监控以及为数据传输提供通道，代码如下：

```

1.  import socket
2.  from typing import *
3.
4.  def setup_socket_and_address(dest_ip = '192.168.1.120', port=4389
5.  3) -> Tuple[socket.socket, Tuple[str, int]]:
6.      # 创建UDP 套接字
7.      sfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

```

```

8.         # 设置目标地址
9.         target_address = (dest_ip, port)
10.        # print(target_address)
11.
12.        return sfd, target_address

```

这段 Python 代码定义了一个函数 `setup_socket_and_address`, 其目的是设置一个 UDP 套接字并返回该套接字及其目标地址。下面是对代码的逐行分析:

`import socket` - 导入 Python 的 `socket` 库, 这个库提供了访问底层网络接口的机制。

`from typing import *` - 从 Python 的 `typing` 模块导入所有类型提示, 用于在函数和方法中添加类型注解, 以提高代码的可读性和健壮性。

`def setup_socket_and_address(dest_ip = '192.168.1.120', port=43893) -> Tuple[socket.socket, Tuple[str, int]]:` - 定义了一个名为 `setup_socket_and_address` 的函数, 它接受两个参数: `dest_ip` 和 `port`, 这两个参数分别指定了目标 IP 地址和端口号, 默认值分别是 `'192.168.1.120'` 和 `43893`。函数的返回类型被注解为 `Tuple[socket.socket, Tuple[str, int]]`, 意味着它将返回一个元组, 其中包含一个 `socket` 对象和一个包含 IP 地址和端口号的元组。

`sfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)` - 创建一个新的 UDP 套接字。`socket.AF_INET` 表示使用 IPv4 地址, `socket.SOCK_DGRAM` 表示使用 UDP 协议。

`target_address = (dest_ip, port)` - 将传入的 IP 地址和端口号组合成一个元组, 这个元组将用作套接字的目标地址。

8.3. 创建指令头

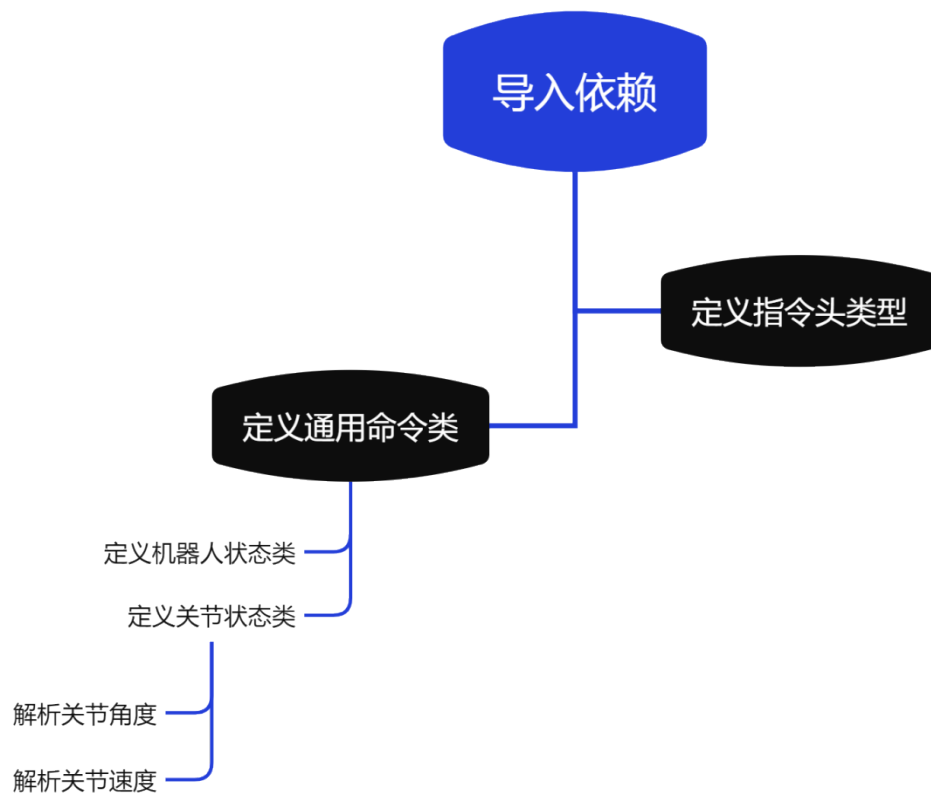
创建指令头主要为了构建通信协议, 提高代码的可读性和可维护性以及支持复杂的指令操作。指令头包含了关键信息, 如指令码、参数大小和类型, 是通信协议的核心组成部分。通过定义指令头, 可以构建和解析复杂的通信协议。

```

1.     action_CommandHead = CommandHead()

```

创建了一个 `CommandHead` 对象, 这个对象包含了指令的头部信息, 如动作代码、序列号等。



```
1.  #!/usr/bin/env python
2.  # -*- coding: utf-8 -*-
3.
4.  import ctypes
5.  import struct
6.  import inspect
7.
8.  class CommandHead:
9.      def __init__(self, code=0, parameters_size=0, type_=0):
10.         self.code = code #
    指令码
```

```

11.         self.parameters_size = parameters_size                #
    指令值
12.         self.type_ = type_                                    #
    指令类型
13.
14.     class Command:
15.         kDataSize = 256
16.
17.         def __init__(self, head=None, data=None):
18.             if head is None:
19.                 head = CommandHead()
20.             if data is None:
21.                 data = [0] * self.kDataSize
22.             self.head = head
23.             self.data = data
24.     class JointStateReceived(CommandHead):
25.         def __init__(self, data):
26.             """
27.
28.             1. 关节信息的头部信息为 4 字节命令字, 4 字节长度, 4 字节类型, 后续为
    正文数据---(后续的所有都是类似)
29.
30.             2. 1 字节(b), 2 字节无符号短整型(H), 4 字节的无符号整型(I), 接着是
    12 个 8 字节的双精度浮点数(12d)
31.
32.             3. code, parameters_size, type_ 的是继承于 CommandHead 头的, 这样
    写是为了方便呈现出, 数据都是基于 CommandHead 发送和接收的
33.
    这样可以做到避免所有的复杂数据全部发在相同名字的 Command 中, 而
    是重新定义一个基于 CommandHead 的结构体
34.

```

```

35.         4.“self.code,” 逗号是为了去除掉由 struct.unpack 操作产生
           的元组问题，可以去掉逗号直接复制
36.
37.         """
38.         self.data = data          # 关节的状态数据，全部接受，再由
           code 来决定分配给谁
39.         CommandHead.code, = struct.unpack('<I', self.data[:4])
40.
41.
42.     class JointAngle(CommandHead):
43.         def __init__(self, joint_state_received):
44.             # 特定于关节角度的数据解析
45.             *self.joint_angles, = struct.unpack('<12d', joint_state_r
           eceived.data[12:])
46.
47.     class JointSpeed(CommandHead):
48.         def __init__(self, joint_state_received):
49.             # 特定于关节速度的数据解析
50.             *self.joint_speeds, = struct.unpack('<12d', joint_state_r
           eceived.data[12:])
51.
52.     class RobotState(CommandHead):
53.         def __init__(self, data):
54.
55.             CommandHead.code, = struct.unpack('<I', data[:4])
56.             CommandHead.parameters_size, = struct.unpack('<I', data[4
           :8])
57.             CommandHead.type_, = struct.unpack('<I', data[8:12])
58.             self.robot_basic_state, = struct.unpack('<I', data[12:16]
           )
           # 机器人基本运动状态

```

```

59.         self.robot_gait_state, = struct.unpack('<I', data[16:20])
           # 机器人步态信息
60.         self.robot_motion_state, = struct.unpack('<i', data[176:1
80]))
           # 机器人动作状态
61.         self.distance_ahead, = struct.unpack('<d', data[-16:-
8]))
           # 雷达前方的距离
62.         self.rear_distance, = struct.unpack('<d', data[-8:])
63.
64.
65.         # self.rpy = struct.unpack('<3d', data[20:44])
           # IMU 角度
66.         # self.rpy_vel = struct.unpack('<3d', data[44:68])
           # IMU 角速度
67.         # self.xyz_acc = struct.unpack('<3d', data[68:92])
           # IMU 加速度
68.         # self.pos_world = struct.unpack('<3d', data[92:116])
           # 机器人在世界坐标系中的位置
69.         # self.vel_world = struct.unpack('<3d', data[116:140])
           # 机器人在世界坐标系中的速度
70.         # self.vel_body = struct.unpack('<3d', data[140:164])
           # 机器人在体坐标系中的速度
71.         # self.touch_down_and_stair_trot = struct.unpack('<I', da
ta[164:168])
           # 此功能暂时未激活。此数据仅用于占位
72.         # self.is_charging = struct.unpack('<b', data[168:169])
           # 暂时未开放
73.         # self.error_state = struct.unpack('<I', data[169:173])
           # 暂时未开放
74.

```

类的继承结构

这些类都继承自 `CommandHead`，它们共享一些基本属性和方法。`CommandHead` 定义了基本信息，而他的子类定义了用于解析命令头部的方法和属性。

`JointStateReceived` 类

作用：这个类用于接收关节状态的原始数据，并且解析出头部信息。

解析过程：使用 `struct.unpack` 从数据的前 4 个字节解析出命令字（code）。这里使用了小端格式（<）和无符号整型（I）。

设计意图：通过继承 `CommandHead`，`JointStateReceived` 可以利用基类中定义的方法来处理命令头信息。

`JointAngle` 类

作用：专门用于从关节状态数据中解析出关节的角度。

解析过程 构造函数接收一个 `JointStateReceived` 实例，然后从实例的数据中（跳过前 12 个字节的头部信息）解析出 12 个双精度浮点数，这些数表示关节的角度。

数据结构：关节角度数据紧跟在命令头之后，每个角度占用 8 个字节。

`JointSpeed` 类

作用：与 `JointAngle` 类似，但用于解析关节的速度。

解析过程：同样接收一个 `JointStateReceived` 实例，并从数据中解析出 12 个双精度浮点数表示关节速度。

`RobotState` 类

作用：解析机器人的整体状态，包括运动状态、步态信息、动作状态、雷达距离等。

解析过程：

首先解析命令头信息，包括命令码、参数大小和类型。

然后解析机器人的基本运动状态、步态信息、动作状态等，每个状态占用 4 个字节。

数据解析的细节

小端格式: `struct.unpack` 中的 `<` 表示小端格式, 这是计算机存储多字节数据时的一种字节顺序, 其中最低有效字节存储在地址的最低位。

数据类型: 使用不同的格式字符来指定数据类型, 例如 `I` 表示无符号整型 (4 字节), `d` 表示双精度浮点数 (8 字节)。

星号(*): 在 `struct.unpack` 中使用星号是为了接收多个返回值, 并将它们解包到多个变量中。

设计哲学

模块化: 每个类负责解析特定部分的数据, 这有助于代码的组织和维护。

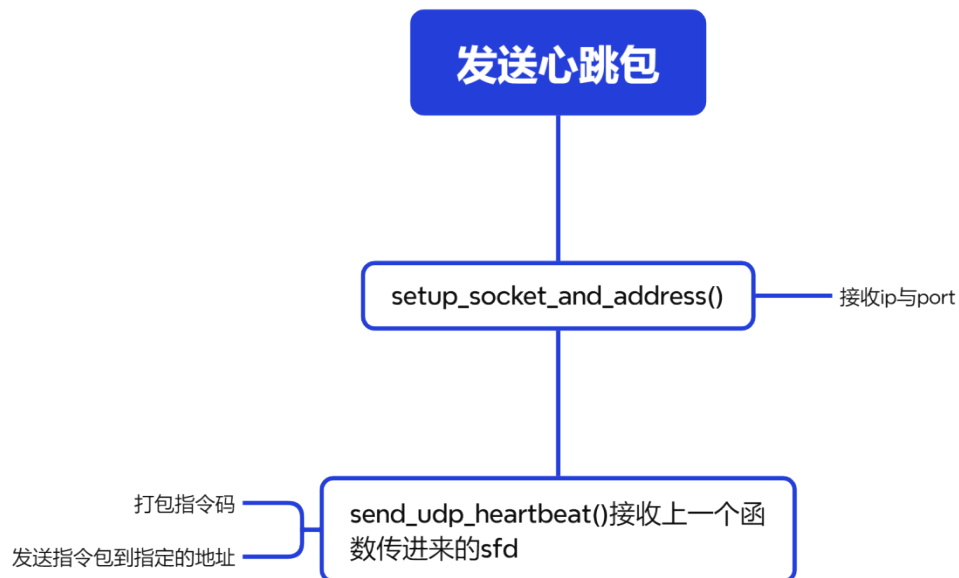
扩展性: 通过继承 `CommandHead`, 新的状态解析类可以轻松地添加, 同时复用头部解析逻辑。

灵活性: 通过注释掉一些解析字段, 代码提供了在未来添加新功能的可能性。

使用场景

这些类用于一个机器狗的控制系统中, 用于实时监控和解析机器人的状态信息, 以便于进行决策和控制。

8. 4. 发送心跳包



心跳包是一种周期性发送的信号，用于确保网络连接的活跃状态，防止因长时间无通信而被网络设备断开连接。这对于实时控制和数据传输至关重要。确保系统在运行过程中不会因为通信问题而出现故障，提高系统的整体可靠性。发送心跳包代码位于 `sendcommand/heartbeat.py`，代码如下：

```
1.  #!/usr/bin/env python
2.  # -*- coding: utf-8 -*-
3.  import socket
4.  import struct
5.  import time
6.
7.  def send_udp_heartbeat(sfd, target_address, code=0x21040001, parameters_size=0, type=0, heartbeat_interval=0.25) -> None:
8.      # 打包心跳指令数据
9.      heartbeat_command = struct.pack('<III', code, parameters_size, type)
```

```

10.
11.     try:
12.         while True:
13.             # 记录发送前的时间
14.             start_time = time.time()
15.
16.             # 发送心跳包
17.             sfd.sendto(heartbeat_command, target_address)
18.
19.             # 发送后的延时
20.             time.sleep(max(0, heartbeat_interval - (time.time() -
start_time)))
21.
22.         except KeyboardInterrupt:
23.             print("Heartbeat sending stopped by user.")
24.         finally:
25.             # 关闭 socket
26.             sfd.close()

```

`network_utils.setup_socket_and_address()` 函数被调用，用于创建一个 UDP 套接字（或其他类型的网络通信接口）并设置机器狗的 IP 地址和端口号作为目标地址。心跳包是一种周期性发送的信号，用于确保网络连接的活跃状态，防止因长时间无通信而被网络设备断开连接。导入所需的模块：`socket` 用于网络通信，`struct` 用于数据打包，`time` 用于时间相关的操作。

定义了 `send_udp_heartbeat` 函数，它接收以下参数：

`sfd`: 一个 `socket` 对象，用于发送数据。

`target_address`: 目标地址的元组，包含 IP 地址和端口号。

`code`: 心跳指令的代码，默认为 `0x21040001`。

`parameters_size`: 参数大小，默认为 `0`。

type: 心跳包的类型, 默认为 0。

heartbeat_interval: 心跳包发送间隔, 默认为 0.25 秒。

使用 `struct.pack` 将心跳指令的数据打包成字节串。参数 `<III` 表示使用小端格式打包三个整数。

10-22. 在一个无限循环中, 执行以下操作:

记录发送前的时间 (第 15 行)。

发送心跳包到目标地址 (第 18 行)。

计算发送后需要等待的时间, 以确保心跳包的发送间隔符合 `heartbeat_interval` (第 21 行)。

23-24. 使用 `try-except` 结构来捕获 `KeyboardInterrupt` 异常, 这通常发生在用户使用 `Ctrl+C` 中断程序时。如果捕获到这个异常, 会打印一条消息表明心跳发送已被用户停止。

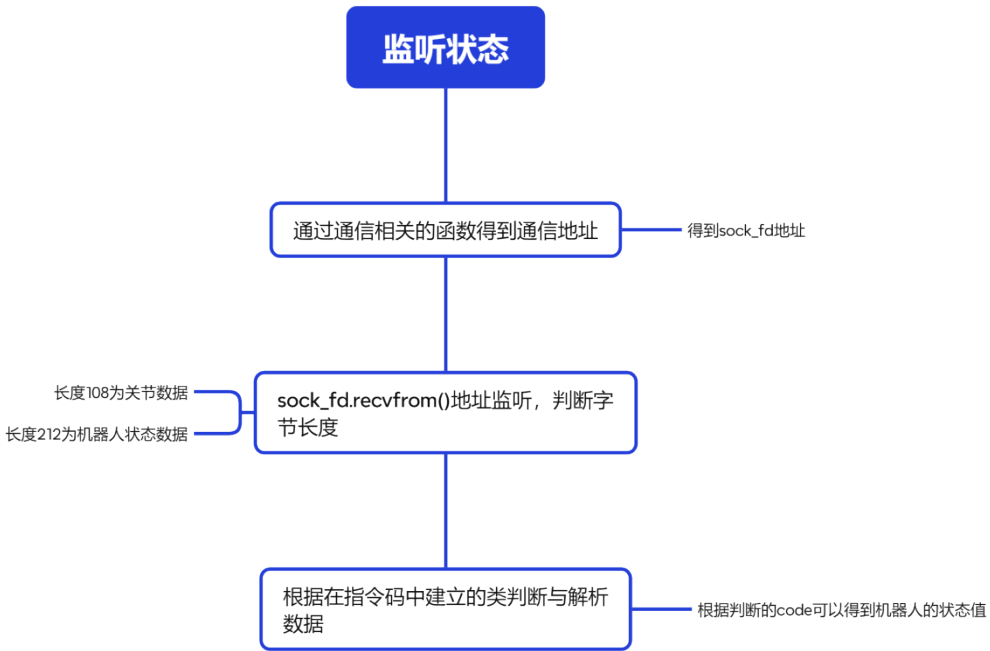
25-27. 使用 `finally` 块来确保无论是否发生异常, `socket` 都会在函数结束时关闭。

启动心跳线程, 代码如下:

```
1.      # 建立通讯
2.      sfd, target_address = network_utils.setup_socket_and_address()
3.
4.      # 定义发送心跳包的线程执行体
5.      # 注意*args 必须以 tuple 形式传入, 包括 sfd 和 target_address
6.      heartbeat_thread = MyRepeatThread("HeartbeatThread", heartbeat.send_u
dp_heartbeat, 0.25, None, sfd, target_address)
7.      heartbeat_thread.start()
```

`heartbeat_thread.start()` 启动了心跳线程, 它将按照设定的时间间隔 (这里是 0.25 秒) 发送心跳包 (`MyRepeatThread` 在后文中会解释)。

8. 5. 监听模块



机器狗的状态值有独有的编码列表，见下表。

值	值	值	状态描述
1	2	3	
1	0	0	趴下状态
1	0	11	正在执行向前跳
4	0	0	准备起立状态
5	0	0	正在起立状态
6	0	0	力控状态（静止站立）且步态为平地低速步态
6	0	1	正在以平地低速步态踏步或正在根据轴指令扭动身体
6	0	2	正在执行扭身体
6	0	4	正在执行扭身跳
6	2	0	力控状态（静止站立）且步态为通用越障步态
6	2	1	正在以通用越障步态踏步
6	4	0	力控状态（静止站立）且步态为平地中速步态

6	4	1	正在以平地中速步态踏步
6	5	0	力控状态（静止站立）且步态为平地高速步态
6	5	1	正在以平地高速步态踏步
6	6	0	力控状态（静止站立）且步态为抓地越障步态
6	6	1	正在以抓地越障步态踏步
6	12	1	正在执行太空步
6	13	0	力控状态（静止站立）且步态为高踏步越障步态
6	13	1	正在以高踏步越障步态踏步
7	0	0	正在趴下状态
8	0	0	失控保护状态
9	0	0	姿态调整状态
11	0	0	正在执行翻身
17	0	0	回零状态
18	0	0	正在执行后空翻
20	0	0	正在执行打招呼

```

1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3.
4. # 导入 sys 模块，提供访问解释器的变量和函数
5. import sys
6.
7. # 将上级目录添加到模块搜索路径中
8. sys.path.append('../')
9.
10. # 从相对路径的 command 包中导入 udp_command 模块
11. from command.udp_command import *
12.
13. # 从 socketnetwork 模块导入 network_utils

```

```

14. from socketnetwork import network_utils
15.
16. # 使用 network_utils 中的 set_up_recvfrom_socket_and_address 函数设置
    并返回套接字和接收地址
17. sock_fd = network_utils.set_up_recvfrom_socket_and_address()
18.
19. # 定义 status_listener 函数, 用于监听接收到的数据
20. def status_listener():
21.     # 无限循环, 持续监听
22.     while True:
23.         # 使用 sock_fd 接收数据, 假设接收的数据不会超过 1024 字节
24.         recv_data, _ = sock_fd.recvfrom(1024)
25.         # 计算接收到的数据长度
26.         recv_num = len(recv_data)
27.         # 如果接收到的数据长度为 108, 认为数据完整
28.         if recv_num == 108:
29.             # 使用 JointStateReceived 类解析接收到的数据
30.             dr = JointStateReceived(recv_data)
31.             # 如果解析后的 code 属性为 2306, 表示是关机角度的指令
32.             if dr.code == 2306:
33.                 # 创建 JointAngle 实例, 访问 JointStateReceived 实例的
                    data 属性
34.                 joint_angle = JointAngle(dr)
35.                 # 打印接收到的关节角度数据
36.                 # print("Received Joint Angles:", joint_angle.joi
                    nt_angles)
37.
38.                 # 如果解析后的 code 属性为 2307, 表示是关节角速度的指令
39.                 elif dr.code == 2307:
40.                     # 创建 JointSpeed 实例

```

```

41.         joint_speed = JointSpeed(dr)
42.         # 打印接收到的关节速度数据
43.         # print("Received Joint Speed:", joint_speed.join
            t_speeds)
44.
45.         # 如果接收到的数据长度为 212，认为数据是机器人状态数据
46.         elif recv_num == 212:
47.             # 使用 RobotState 类解析接收到的数据
48.             dr, status_list_temp = RobotState(recv_data), []
49.             # 如果解析后的 code 属性为 2305，表示是机器状态数据
50.             if dr.code == 2305:
51.                 # 如果机器人基本运动状态不为 0，添加状态到列表
52.                 if dr.robot_basic_state != 0:
53.                     status_list_temp.append(dr.robot_basic_state)
54.                     status_list_temp.append(dr.robot_gait_state)
55.                     status_list_temp.append(dr.robot_motion_state)
56.                 # 返回状态列表
57.                 return status_list_temp

```

导入模块和设置系统路径：

导入 sys 模块，并添加上级目录到系统路径中，以便能够导入上级目录中的模块。

导入自定义模块：

从 command.udp_command 模块导入相关的类和函数，这些包括用于解析机器狗状态数据的类。

从 socketnetwork 模块导入 network_utils，它包含用于设置网络通信的函数。

设置 UDP 接收套接字和地址：

使用 network_utils.set_up_recvfrom_socket_and_address() 函数设置 UDP 套接字，用于接收机器狗发送的数据。

定义状态监听函数 `status_listener`:

这个函数包含一个无限循环, 持续接收机器人发送的数据。

使用 `sock_fd.recvfrom(1024)`接收数据, 其中 1024 是缓冲区的大小, 假设接收的数据不会超过这个大小。

检查接收到的数据长度:

如果接收到的数据长度为 108 字节, 认为数据长度正确, 然后根据数据进行解析。

解析关机角度数据:

如果数据的代码 `code` 是 2306, 使用 `JointStateReceived` 类解析接收到的数据, 并进一步使用 `JointAngle` 类获取关机角度信息。

解析关节角速度数据:

如果数据的代码 `code` 是 2307, 使用 `JointSpeed` 类解析接收到的关节角速度信息。

解析机器人状态数据:

如果接收到的数据长度为 212 字节, 认为数据长度正确, 然后使用 `RobotState` 类解析机器人的状态数据。

如果解析后的数据代码 `code` 是 2305, 表示机器人的基本运动状态数据, 将其解析并存储到 `status_list_temp` 列表中。

返回机器人状态列表:

如果解析成功, 并且机器人的基本运动状态不为空, 将状态信息作为一个列表返回。

8.6. 图像处理函数

该函数提供了一套图像处理功能, 主要目的是用于图像的预处理、目标检测以及后处理。用于辅助目标检测任务, 从图像预处理到最终的检测结果可视化, 提供了一套完整的流程。

```
1. import time # 导入time模块, 用于时间相关的操作
2. import cv2 # 导入OpenCV库, 用于图像处理
3. import numpy as np # 导入NumPy库, 用于数学运算
```

```

4. import torch # 导入 PyTorch 库, 用于深度学习
5. import torchvision # 导入 TorchVision 库, 用于图像相关的操作
6.
7. # 定义 Letterbox 函数, 用于将图像调整到指定的新形状
8. def letterbox(img, new_shape=(640, 640), color=(114, 114, 114), auto
    =False, scaleFill=False, scaleup=True):
9.     # 获取当前图像的尺寸 [height, width]
10.     shape = img.shape[:2]
11.
12.     # 如果 new_shape 是整数, 则将新形状设置为宽高相等的正方形
13.     if isinstance(new_shape, int):
14.         new_shape = (new_shape, new_shape)
15.
16.     # 计算宽度和高度的缩放比例
17.     r = min(new_shape[0] / shape[0], new_shape[1] / shape[1])
18.
19.     # 如果 scaleup 为 False, 则限制缩放比例不超过 1, 即不放大图像
20.     if not scaleup:
21.         r = min(r, 1.0)
22.
23.     # 计算缩放后的图像尺寸
24.     new_unpad = int(round(shape[1] * r)), int(round(shape[0] * r))
25.
26.     # 计算宽度和高度的填充量
27.     dw, dh = new_shape[1] - new_unpad[0], new_shape[0] -
        new_unpad[1] # wh padding
28.
29.     # 如果 auto 为 True, 则使用最小矩形, 填充量应为 64 的倍数
30.     if auto:
31.         dw, dh = np.mod(dw, 64), np.mod(dh, 64) # wh padding

```

```

32.
33.     # 如果 scaleFill 为 True, 则不进行缩放, 直接使用新尺寸填充
34.     elif scaleFill:
35.         dw, dh = 0.0, 0.0
36.         new_unpad = (new_shape[1], new_shape[0]) # 使用新尺寸
37.         # 重新计算宽度和高度的缩放比例
38.         ratio = new_shape[1] / shape[1], new_shape[0] / shape[0]
39.
40.         # 将填充量分为上下和左右两部分
41.         dw /= 2
42.         dh /= 2
43.
44.         # 如果需要调整图像尺寸, 则进行缩放
45.         if shape[:: -1] != new_unpad:
46.             img = cv2.resize(img, new_unpad, interpolation=cv2.INTER_
                LINEAR)
47.
48.         # 计算上、下、左、右边界的填充量, 并进行四舍五入
49.         top, bottom = int(round(dh - 0.1)), int(round(dh + 0.1))
50.         left, right = int(round(dw - 0.1)), int(round(dw + 0.1))
51.
52.         # 使用 cv2.copyMakeBorder 函数添加边框, 颜色为 color 指定的颜色
53.         img = cv2.copyMakeBorder(img, top, bottom, left, right, cv2.B
            ORDER_CONSTANT, value=color)
54.
55.         # 返回调整尺寸并添加边框后的图像, 缩放比例, 以及左右和上下的填充量
56.         return img, ratio, (dw, dh)
57.
58. # 定义 non_max_suppression 函数, 用于执行非极大值抑制 (NMS)
59. def non_max_suppression(

```

```

60.     prediction,                # 预测结果张量
61.     conf_thres=0.25,           # 置信度阈值, 默认为 0.25
62.     iou_thres=0.45,           # 交并比阈值, 默认为 0.45
63.     classes=None,             # 要检测的类别列表, 如果为 None, 则检测
    所有类别
64.     agnostic=False,           # 是否忽略类别
65.     multi_label=False,        # 是否允许多标签
66.     labels=(),                # 先验标签列表
67.     max_det=300,              # 每张图像最大检测数量
68.     nm=0,                     # 掩码数量
69. )
70.
71.     在推理结果上执行 NMS, 以去除重叠的检测框
72.     返回值 :
73.         每张图像的检测结果列表, 格式为 (n,6+nm) 张量 [xyxy, conf, cls]
74.
75.
76.     # 如果 prediction 是列表或元组, 取第一个元素作为预测结果
77.     if isinstance(prediction, (list, tuple)):
78.         prediction = prediction[0]
79.
80.     # 获取预测结果所在的设备 (CPU 或 GPU)
81.     device = prediction.device
82.
83.     # 检查是否在使用 Apple MPS, 如果使用 MPS, 则将预测结果转换到 CPU
84.     mps = 'mps' in device.type
85.     if mps:
86.         prediction = prediction.cpu()
87.
88.     # 获取批量大小

```

```

89.     bs = prediction.shape[0]
90.
91.     # 计算类别数量
92.     nc = prediction.shape[2] - nm - 5
93.
94.     # 根据置信度阈值过滤预测结果
95.     xc = prediction[..., 4] > conf_thres
96.
97.     # 检查置信度和交并比阈值的有效性
98.     assert 0 <= conf_thres <= 1, f'Invalid Confidence threshold {
        conf_thres}, valid values are between 0.0 and 1.0'
99.     assert 0 <= iou_thres <= 1, f'Invalid IoU {iou_thres}, valid
        values are between 0.0 and 1.0'
100.
101.    # 设置
102.    # min_wh = 2 # 检测框的最小宽度和高度 (像素)
103.    max_wh = 7680 # 检测框的最大宽度和高度 (像素)
104.    max_nms = 30000 # 传入 torchvision.ops.nms() 的最大检测框数量
105.    time_limit = 0.5 + 0.05 * bs # NMS 执行时间限制
106.
107.    # 如果类别数大于1, 则允许每个检测框有多个标签
108.    multi_label &= nc > 1
109.
110.    # 记录开始时间
111.    t = time.time()
112.
113.    # 掩码的起始索引
114.    mi = 5 + nc
115.
116.    # 初始化输出列表, 用于存储每张图像的检测结果

```

```

117.     output = [torch.zeros((0, 6 + nm), device=prediction.device)]
        * bs
118.
119.     # 遍历批量中的每张图像
120.     for xi, x in enumerate(prediction):
121.         # 应用约束条件, 例如最小宽度和高度
122.         # x[((x[..., 2:4] < min_wh) | (x[..., 2:4] > max_wh)).any
            (1), 4] = 0
123.
124.         # 根据置信度阈值过滤预测结果
125.         x = x[xc[xi]]
126.
127.         # 如果启用自动标注, 将先验标签添加到预测结果中
128.         if labels and len(labels[xi]):
129.             lb = labels[xi]
130.             v = torch.zeros((len(lb), nc + nm + 5), device=x.device)
131.             v[:, :4] = lb[:, 1:5] # 边界框坐标
132.             v[:, 4] = 1.0 # 置信度
133.             v[range(len(lb)), lb[:, 0].long() + 5] = 1.0 # 类别
134.             x = torch.cat((x, v), 0)
135.
136.         # 如果没有检测框剩余, 处理下一张图像
137.         if not x.shape[0]:
138.             continue
139.
140.         # 计算置信度, 将对象置信度与类别置信度相乘
141.         x[:, 5:] *= x[:, 4:5]
142.
143.         # 将中心点坐标和宽高转换为左上角和右下角坐标

```

```

144.         box = xywh2xyxy(x[:, :4])
145.
146.         # 获取掩码信息
147.         mask = x[:, mi:]
148.
149.         # 根据是否允许多标签, 构建检测矩阵
150.         if multi_label:
151.             i, j = (x[:, 5:mi] > conf_thres).nonzero(as_tuple=False).T
152.             x = torch.cat((box[i], x[i, 5 + j, None], j[:, None].float(), mask[i]), 1)
153.         else: # 只选择最佳类别
154.             conf, j = x[:, 5:mi].max(1, keepdim=True)
155.             x = torch.cat((box, conf, j.float(), mask), 1)[conf.view(-1) > conf_thres]
156.
157.         # 按类别过滤检测结果
158.         if classes is not None:
159.             x = x[(x[:, 5:6] == torch.tensor(classes, device=x.device)).any(1)]
160.
161.         # 检查检测框数量
162.         n = x.shape[0] # 检测框数量
163.         if not n: # 没有检测框
164.             continue
165.         elif n > max_nms: # 检测框数量过多
166.             x = x[x[:, 4].argsort(descending=True)[:max_nms]] # 按置信度排序
167.         else:

```

```

168.         x = x[x[:, 4].argsort(descending=True)] # 按置信度排
           序
169.
170.         # 执行批量 NMS
171.         c = x[:, 5:6] * (0 if agnostic else max_wh) # 类别偏移
172.         boxes, scores = x[:, :4] + c, x[:, 4] # 检测框 (偏移类别),
           分数
173.         i = torchvision.ops.nms(boxes, scores, iou_thres) # NMS
174.
175.         # 如果检测数量超过最大检测限制, 进行限制
176.         if i.shape[0] > max_det:
177.             i = i[:max_det]
178.
179.         # 将当前图像的检测结果添加到输出列表中
180.         output[xi] = x[i]
181.
182.         # 如果在使用 Apple MPS, 则将结果移回原始设备
183.         if mps:
184.             output[xi] = output[xi].to(device)
185.
186.         # 如果 NMS 执行时间超过时间限制, 打印警告并退出循环
187.         if (time.time() - t) > time_limit:
188.             print(f'WARNING ⚠️ NMS time limit {time_limit:.3f}s e
               xceeded')
189.             break
190.
191. # 返回最终的检测结果列表
192. return output
193.

```

```

194. # 定义 xywh2xyxy 函数，将边界框从 [x, y, w, h] 格式转换为
    [x1, y1, x2, y2] 格式
195. def xywh2xyxy(x):
196.     # 如果 x 是 torch.Tensor 类型，则克隆数据；如果是 numpy 数组，则复
        制数据
197.     y = x.clone() if isinstance(x, torch.Tensor) else np.copy(x)
198.     # 计算左上角 x 坐标
199.     y[:, 0] = x[:, 0] - x[:, 2] / 2
200.     # 计算左上角 y 坐标
201.     y[:, 1] = x[:, 1] - x[:, 3] / 2
202.     # 计算右下角 x 坐标
203.     y[:, 2] = x[:, 0] + x[:, 2] / 2
204.     # 计算右下角 y 坐标
205.     y[:, 3] = x[:, 1] + x[:, 3] / 2
206.     return y # 返回转换后的边界框
207.
208. # 定义 get_labels_from_txt 函数，从文本文件中读取标签并创建字典
209. def get_labels_from_txt(path):
210.     labels_dict = dict() # 初始化标签字典
211.     with open(path) as f: # 打开文件
212.         for cat_id, label in enumerate(f.readlines()): # 遍历文件
            中的每一行
213.             labels_dict[cat_id] = label.strip() # 将读取的标签去除
                两端空白后存入字典
214.     return labels_dict # 返回标签字典
215.
216. # 定义 scale_coords 函数，调整坐标点以适应不同尺寸的图像
217. def scale_coords(img1_shape, coords, img0_shape, ratio_pad=None):
218.     # 如果 ratio_pad 为 None，则根据 img0_shape 计算缩放比例和填充量
219.     if ratio_pad is None:

```

```

220.         gain = min(img1_shape[0] / img0_shape[0], img1_shape[1] /
    img0_shape[1]) # 计算缩放比例
221.         pad = (img1_shape[1] -
    img0_shape[1] * gain) / 2, (img1_shape[0] -
    img0_shape[0] * gain) / 2 # 计算填充量
222.     else:
223.         gain = ratio_pad[0][0]
224.         pad = ratio_pad[1]
225.
226.     # 应用 x 方向的填充并调整坐标点
227.     coords[:, [0, 2]] -= pad[0]
228.     # 应用 y 方向的填充并调整坐标点
229.     coords[:, [1, 3]] -= pad[1]
230.     # 应用缩放比例调整坐标点
231.     coords[:, :4] /= gain
232.     # 裁剪坐标点, 确保它们在有效范围内
233.     clip_coords(coords, img0_shape)
234.     return coords # 返回调整后的坐标点
235.
236. # 定义 clip_coords 函数, 裁剪边界框坐标, 确保它们在图像尺寸范围内
237. def clip_coords(boxes, shape):
238.     # 如果 boxes 是 torch.Tensor 类型, 则逐个快速裁剪
239.     if isinstance(boxes, torch.Tensor):
240.         boxes[:, 0].clamp_(0, shape[1]) # 裁剪 x1 坐标
241.         boxes[:, 1].clamp_(0, shape[0]) # 裁剪 y1 坐标
242.         boxes[:, 2].clamp_(0, shape[1]) # 裁剪 x2 坐标
243.         boxes[:, 3].clamp_(0, shape[0]) # 裁剪 y2 坐标
244.     # 如果 boxes 是 numpy 数组, 则整体快速裁剪
245.     else:

```

```

246.         boxes[:, [0, 2]] = boxes[:, [0, 2]].clip(0, shape[1]) #
           裁剪 x1 和 x2
247.         boxes[:, [1, 3]] = boxes[:, [1, 3]].clip(0, shape[0]) #
           裁剪 y1 和 y2
248.
249. # 定义 nms 函数，作为非极大值抑制（NMS）的封装，调用
           non_max_suppression 函数
250. def nms(box_out, conf_thres=0.4, iou_thres=0.5):
251.     try:
252.         # 尝试使用 multi_label=True 的 NMS
253.         boxout = non_max_suppression(box_out, conf_thres=conf_thr
           es, iou_thres=iou_thres, multi_label=True)
254.     except:
255.         # 如果出错，则使用默认的 NMS
256.         boxout = non_max_suppression(box_out, conf_thres=conf_thr
           es, iou_thres=iou_thres)
257.     return boxout # 返回 NMS 处理后的结果
258.
259. # 定义 draw_bbox 函数，在图像上绘制边界框和标签
260. def draw_bbox(bbox, img0, color, wt, names):
261.     det_result_str = '' # 初始化检测结果字符串
262.     for idx, class_id in enumerate(bbox[:, 5]): # 遍历边界框和类别
           ID
263.         if float(bbox[idx][4]) < float(0.05): # 如果置信度小于
           0.05，则跳过
264.             continue
265.         print(str(idx) + ' ' + names[int(class_id)]) # 打印边界框
           索引和类别名称
266.         # 在图像上绘制矩形边界框

```

```

267.         img0 = cv2.rectangle(img0, (int(bbox[idx][0]), int(bbox[idx][1])), (int(bbox[idx][2]), int(bbox[idx][3])), color, wt)
268.         # 在边界框左上角绘制类别名称
269.         img0 = cv2.putText(img0, str(idx) + ' ' + names[int(class_id)], (int(bbox[idx][0]), int(bbox[idx][1] + 16)), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 1)
270.         # 在边界框下方绘制置信度
271.         img0 = cv2.putText(img0, '{:.2f}'.format(bbox[idx][4]), (int(bbox[idx][0]), int(bbox[idx][1] + 32)), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 1)
272.         # 将检测结果添加到字符串
273.         det_result_str += '{} {} {} {} {}{}\n'.format(names[bbox[idx][5]], str(bbox[idx][4]), bbox[idx][0], bbox[idx][1], bbox[idx][2], bbox[idx][3])
274.         return img0 # 返回绘制了边界框和标签的图像

```

letterbox: 这个函数用于将图像调整到指定的新尺寸，同时保持长宽比，通过添加边框来填充图像。它还计算了缩放比例和填充尺寸。

non_max_suppression: 这个函数实现了非极大值抑制（NMS），用于从预测结果中去除重叠的检测框。它接受多个参数，包括置信度阈值、IoU 阈值等，并返回经过 NMS 处理的检测结果。

xywh2xyxy: 将坐标格式从(x, y, w, h)转换为(x1, y1, x2, y2)，即从中心点和宽高转换为左上角和右下角的坐标。

get_labels_from_txt: 从文本文件中读取类别标签，并将它们存储在字典中。

scale_coords: 将坐标从一个图像尺寸调整到另一个图像尺寸，考虑了缩放和平移。

clip_coords: 将坐标限制在给定图像尺寸的范围，确保检测框不会超出图像边界。

nms: 一个包装函数，用于调用 **non_max_suppression** 函数并返回 NMS 处理后的检测框。

draw_bbox: 在图像上绘制检测框，并在框旁边显示类别名称和置信度。

8.7. 速度函数

这段代码提供了用于控制设备运动的三个函数，包括直线行驶、左右平移和左右旋转。每个运动函数都接受一些参数，如运动的距离或角度和速度档位，并计算出所需的时间和速度。代码中使用了数学公式来根据距离或角度计算时间，这些公式可能是基于物理模型或实验数据得出的。旋转函数中使用了映射字典来将角度转换为时间，是为了简化控制逻辑或适应特定的运动要求。

```
1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3.
4. from command.udp_command import * # 存放各种结构
   体与状态数据、指令
5.
6. def go_straight(long, speedgear=3):
7.     # 档位速度只是参考值，具体速度按照实际来判断，默认速度是3档
8.     speedgear_ditc = {
9.         1:7000,
10.        2:7500,
11.        3:8000,
12.        4:8700,
13.        5:9000,
14.        6:10500,
15.    }
16.
```

```

17.     val = speedgear_ditc[speedgear]
18.
19.     if long < 0:
20.         val = -val
21.         speed_per_second_meter = (-1.5059e-08) * val ** 2 - (4.1944e-
04) * val - 2.1804
22.         times = abs(long / speed_per_second_meter)
23.     else:
24.         speed_per_second_meter = (-7.237e-
09) * val ** 2 + 0.0002933 * val - 1.643
25.         times = long / speed_per_second_meter
26.     return [times, val]
27.
28.
29. def translate_left_and_right(long, speedgear=3):
30.     # 档位速度只是参考值，具体速度按照实际来判断，默认速度是3档
31.     speedgear_ditc = {
32.         1: 14000,
33.         2: 18000,
34.         3: 21000,
35.         4: 24000,
36.         5: 27000,
37.         6: 34000,

```

```

38.     }
39.     val = speedgear_ditc[speedgear]
40.     if long < 0:
41.         val = -val
42.         speed_per_second_meter = -(1.6e-05) * val - 0.2256
43.         times = abs(long / speed_per_second_meter)
44.     else:
45.         speed_per_second_meter = (1.517e-05) * val - 0.1748
46.         times = long / speed_per_second_meter
47.     return [times, val]
48.
49.
50. def revolve_left_and_right(angle):
51.     def find_closest_output(input_value):
52.         # 判断角度是否超过了360度, 超过了则需要则算掉
53.         if input_value > 360:
54.             input_value = input_value / (input_value % 360)
55.
56.         # 输出到输入的映射字典
57.         output_to_input_map = {
58.             0: 0,
59.             15: 0.1,

```

60.	30: 0.2,
61.	45: 0.53,
62.	60: 0.6,
63.	75: 0.9,
64.	90: 1.3,
65.	105: 1.5,
66.	120: 1.9,
67.	135: 2.2,
68.	150: 2.3,
69.	165: 2.5,
70.	167: 2.8,
71.	185: 2.9,
72.	195: 3.1,
73.	210: 3.4,
74.	225: 3.7,
75.	240: 4,
76.	255: 4.3,
77.	270: 4.5,
78.	285: 4.7,
79.	300: 5,
80.	315: 5.3,
81.	330: 5.6,

```

82.         345: 5.8,
83.         360: 5.9,
84.     }
85.     # 获取所有输出键并将其转换为列表
86.     keys = list(output_to_input_map.keys())
87.     # 寻找与输入值最接近的输出键
88.     closest_key = min(keys, key=lambda x: abs(x - input_value))
89.     # 返回与最接近键关联的输入值
90.     return output_to_input_map[closest_key]
91.
92.
93.     times = find_closest_output(abs(angle)) # 获取时间与角度的关系
94.     if angle < 0:
95.         val = -10000
96.     else:
97.         val = 10000
98.
99.     return [times, val]
100.
101.
102.

```

go_straight(long, speedgear=3):

这个函数用于控制设备直线行驶。

long: 表示要行驶的距离，如果为负值，表示反向行驶。

speedgear: 表示设备的档位，影响速度，其默认值为 3。

函数首先定义了一个字典 `speedgear_ditc`，将档位映射到一个速度值。

根据 `long` 的正负，选择相应的速度计算公式来计算所需的时间 `times`。

如果 `long` 为负，速度值 `val` 取反，并使用一个二次方程来计算速度（每秒钟行驶的距离）。

如果 `long` 为正，使用另一个二次方程来计算速度。

最终，函数返回所需的时间和速度值。

`translate_left_and_right(long, speedgear=3):`

这个函数用于控制设备左右平移。

参数和 `go_straight` 函数类似，但速度值是不同的字典 `speedgear_ditc`。

根据 `long` 的正负，选择相应的线性方程来计算时间。

如果 `long` 为负，速度值 `val` 取反，并使用一个线性方程来计算速度。

如果 `long` 为正，使用另一个线性方程来计算速度。

最终，函数返回平移所需的时间和速度值。

`revolve_left_and_right(angle):`

这个函数用于控制设备左右旋转。

angle: 表示要旋转的角度，如果为负值，表示逆时针旋转。

函数内部定义了一个辅助函数 `find_closest_output`，它将输入的角度映射到一个最接近的输出值，这个映射是通过一个预定义的字典 `output_to_input_map` 实现的。

如果输入的角度超过了 360 度，会通过取余数的方式来调整角度。

根据角度的绝对值，找到最接近的角度对应的输出值，这个输出值代表了旋转的时间

times。

如果角度为负，速度值 `val` 设为-10000，表示逆时针旋转；如果为正，则设为 10000，表示顺时针旋转。

最终，函数返回旋转所需的时间和速度值。

8.8. 模型初始化

定义一个初始化模型的函数。主要作用是初始化深度学习模型并加载相关的标签信息，以便进行模型推理和结果解释。以下是对文件的总体总结和作用的描述：

```
1.     def model_init(model_path, device_id, label_path) -> Tuple['base.  
model', Dict[int, str]]:  
2.         # 初始化资源和变量  
3.         base.mx_init() # 初始化 mxVision 资源  
4.         model = base.model(modelPath=model_path, deviceId=device_id)  
         # 初始化 base.model 类  
5.         labels_dict = get_labels_from_txt(label_path) # 得到类别信  
         息，返回序号与类别对应的字典  
6.         return model, labels_dict
```

函数定义：

`model_init` 函数接受三个参数：`model_path`（模型路径），`device_id`（设备 ID），和 `label_path`（标签文件路径）。

函数返回一个元组，包含一个 `base.model` 类的实例和一个字典。字典的键是整数，值是字符串，表示类别的序号和对应的名称。

初始化 `mxVision` 资源：

调用 `base.mx_init()` 函数来初始化 `mxVision` 库的资源。`mxVision` 是一个推理框架，通常用于深度学习模型的部署和推理。

加载模型：

使用 `base.model(modelPath=model_path, deviceId=device_id)` 创建一个 `base.model` 类的实例。这个实例将用于加载和执行深度学习模型。

`modelPath` 参数指定了模型文件的路径, `deviceId` 指定了要使用的设备 ID (例如, CPU 或 GPU)。

加载标签信息:

调用 `get_labels_from_txt(label_path)` 函数来从文本文件中加载标签信息。这个函数应该返回一个字典, 其中包含类别的序号 (作为键) 和对应的类别名称 (作为值)。

返回值:

函数返回两个对象: 初始化的模型实例和标签字典。这样, 调用者就可以使用这些对象来进行模型推理和结果的解释。

8.9. 模型推理

这一部分主要作用是执行图像推理过程, 并返回推理结果的列表。从图像的获取到最终结果的解析和返回, 提供了一个端到端的解决方案。以下是对文件的总体总结和作用的描述

```
1.     def Image_inference(model, labels_dict) -> list:
2.
3.         try:
4.             while True:
5.                 img = getImage()
6.                 frame = img.copy()
7.
8.                 # 数据前处理
9.                 img, scale_ratio, pad_size = letterbox(frame, new_shape=[640, 640]) # 对图像进行缩放与填充, 保持长宽比
10.
```

```

11.          # 用来存放推理结构，三个元素，[idx: 识别到的序列号（指的是
            一次性识别到4个东西，那么序列号就是0,1,2,3），class_id: 识别到的类型，
pred_all[idx][4]: 识别率]

12.          ill_sets = []

13.

14.          img = img[:, :, :-
1].transpose(2, 0, 1) # BGR to RGB, HWC to CHW

15.          img = np.expand_dims(img, 0).astype(np.float32) # 将
            形状转换为 channel first (1, 3, 640, 640)，即扩展第一维为 batchsize

16.          img = np.ascontiguousarray(img) / 255.0 # 转换为内存
            连续存储的数组

17.          img = Tensor(img) # 将numpy 转为转为Tensor 类

18.

19.          # 模型推理，得到模型输出

20.          # model = base.model(modelPath=model_path, deviceId=D
            EVICE_ID) # 初始化 base.model 类

21.          output = model.infer([img])[0] # 执行推理。输入数据类型:
            List[base.Tensor]，返回模型推理输出的 List[base.Tensor]

22.

23.          # 后处理

24.          output.to_host() # 将Tensor 数据转移到 Host 侧

25.          output = np.array(output) # 将数据转为 numpy array 类
            型

26.          boxout = nms(torch.tensor(output), conf_thres=0.4, io
            u_thres=0.5) # 利用非极大值抑制处理模型输出，conf_thres 为置信度阈值，
            iou_thres 为iou 阈值

27.          pred_all = boxout[0].numpy() # 转换为numpy 数组

28.          scale_coords([640, 640], pred_all[:, :4], frame.shape
            , ratio_pad=(scale_ratio, pad_size)) # 将推理结果缩放到原始图片大小

```

```

29.             # labels_dict = get_labels_from_txt('ceshi/demo_labels.txt') # 得到类别信息，返回序号与类别对应的字典
30.             # img_dw = draw_bbox(pred_all, frame, (0, 255, 0), 2, labels_dict) # 画出检测框、类别、概率
31.
32.             for idx, class_id in enumerate(pred_all[:, 5]):
33.                 logging.info(str(idx) + ' ' + labels_dict[int(class_id)])
34.                 # 识别率保留两位小数
35.                 ill_sets.append([idx, int(class_id), round(pred_all[idx][4], 2)])
36.
37.             if ill_sets == []:
38.                 pass
39.             else:
40.                 return ill_sets
41.
42.
43.         except KeyboardInterrupt:
44.             pass
45.
46.         finally:
47.             pass

```

函数定义：

Image_inference 函数接受两个参数：model（深度学习模型的实例）和 labels_dict（类别标签的字典）。

函数返回一个列表，其中包含识别结果的详细信息。

无限循环获取图像：

使用 `getImage()` 函数获取图像，并复制一份用于后续处理。

数据前处理：

使用 `letterbox` 函数对图像进行缩放和填充，以适应模型的输入尺寸。

初始化识别结果列表：

创建一个空列表 `ill_sets`，用于存储推理结果。

图像格式转换：

将图像从 BGR 颜色空间转换为 RGB，并将数据格式从 HWC（高度、宽度、通道）转换为 CHW（通道、高度、宽度）。

数据归一化：

将图像数据转换为内存连续的数组，并进行归一化处理。

模型推理：

使用 `model.infer` 方法对图像进行推理，获取模型的输出。

后处理：

将推理结果从设备内存转移到主机内存，并转换为 NumPy 数组。

使用非极大值抑制（NMS）算法处理模型输出，以去除重叠的检测框。

坐标缩放：

将 NMS 处理后的检测框坐标缩放回原始图像的大小。

解析识别结果：

遍历处理后的识别结果，记录序列号、类别 ID 和识别率，并添加到 `ill_sets` 列表。

返回识别结果：

如果 `ill_sets` 为空，则不执行任何操作 否则，返回包含识别结果的 `ill_sets` 列表。

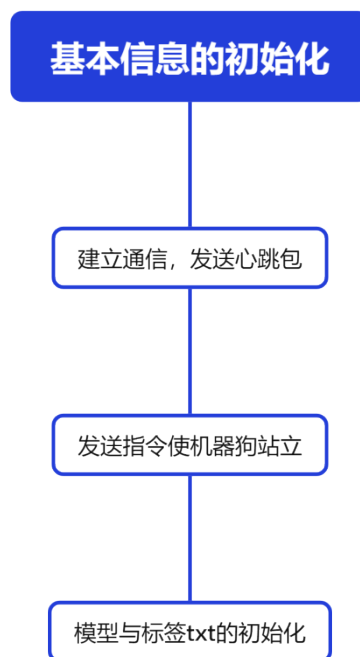
异常处理：

使用 `try-except` 结构捕获 `KeyboardInterrupt` 异常,允许用户通过 `Ctrl+C` 中断程序。

确保资源释放:

`finally` 块可以放置清理资源的代码。

8.10.基本信息的初始化



主要是对机器狗的信息初始化与目标设备的通信, 下面是详细的代码解释:

```
1.  if __name__ == "__main__":
2.
3.      # 建立通讯
4.      sfd, target_address = network_utils.setup_socket_and_address(
5.
6.      # 建立心跳包线程, 按照心跳函数中的频率发送心跳包
```

```

7.         heartbeat_thread = MyRepeatThread("HeartbeatThread", heartbea
t.send_udp_heartbeat, 0.25, None, sfd, target_address)
8.         heartbeat_thread.start()
9.
10.        # 站立指令
11.        SendToCommand.perform_action(sfd, target_address, 0x21010202,
0, 0)
12.
13.        # 变量初始化
14.        DEVICE_ID = 0                                # 设备
id
15.        model_path = 'gesture_models/gesture.om'      # 模型
路径
16.        label_path = 'gesture_models/predefined_classes.txt' # 标签
地址
17.
18.        # 模型与标签txt 的初始化
19.        model, labels_dict = model_init(model_path, DEVICE_ID, label_
path)
20.
21.        # 创建一个简单指令的结构体实例
22.        action_CommandHead = CommandHead()

```

1.脚本入口点

if `__name__ == "__main__"`: 这一行是 Python 中的常规用法,确保当脚本被直接运行时,下面的代码块会被执行。

2. 建立通信

`sfd, target_address = network_utils.setup_socket_and_address()`: 这行代码调用了 `network_utils` 模块中的 `setup_socket_and_address` 函数,用于初始化网络通信所需的套接字 (`sfd`) 和目标地址 (`target_address`)。这用于后续的数据发送和接收。

3. 建立心跳包线程

`heartbeat_thread = MyRepeatThread(...)`：创建了一个名为 `HeartbeatThread` 的 `MyRepeatThread` 线程实例，用于定期发送心跳包。心跳包是一种网络通信机制，用于检测连接是否仍然有效。

`heartbeat.send_udp_heartbeat`：这是心跳线程将要执行的动作函数，它使用 UDP 协议发送心跳包。

`0.25`：心跳包发送的间隔时间，这里设置为每 0.25 秒发送一次。

`None`：没有设置最大时间阈值，因此心跳线程将一直运行直到被外部停止。

`sfd, target_address`：作为参数传递给心跳函数，用于指定发送心跳包的套接字和目标地址。

4. 发送站立指令

`SendToCommand.perform_action(sfd, target_address, 0x21010202, 0, 0)`：这行代码调用了 `SendToCommand` 模块中的 `perform_action` 函数，发送了一个站立指令到指定的设备。这里的 `0x21010202` 是一个指令码，用于告诉设备执行站立动作。

5. 变量初始化

`DEVICE_ID`：设备 ID，用于标识特定的设备或机器人。

`model_path` 和 `label_path`：分别指定了模型文件和标签文件的路径。这些文件用于机器学习或模式识别任务。

6. 模型与标签 txt 的初始化

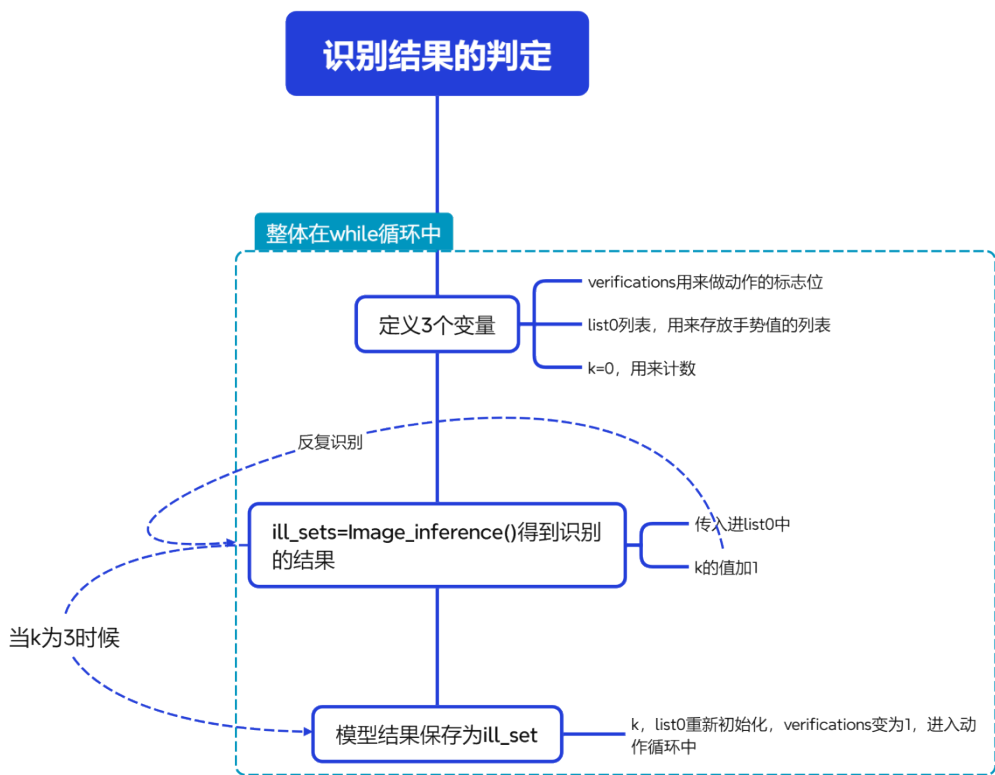
`model, labels_dict = model_init(model_path, DEVICE_ID, label_path)`：调用 `model_init` 函数来加载模型和标签字典。

7. 创建指令结构体实例

`action_CommandHead = CommandHead()`：创建了一个 `CommandHead` 类型的实例，这是用于构造或解析发送给设备的命令。

这段代码演示了如何设置网络通信、启动心跳线程、发送特定的控制指令、初始化模型和标签，以及创建用于发送命令的结构体实例。

8.11. 识别结果判定



主要实现实现一个基于手势识别的判定逻辑,用于确定是否连续三次检测到相同的手势,如果是,则触发机器狗的动作执行循环。

```
1.     verifications = 0 # 进入狗动作的标志位
2.     list0 = []       # 存放手势值的列表
3.     k = 0           # 计数
4.     while True:
5.         ill_sets = Image_inference(model, labels_dict)
6.         print("ill_sets: ", ill_sets)
7.         # print('ill_sets 的长度: ', len(ill_sets))
8.         list0.append(ill_sets[0][1])
```

```

9.         k += 1
10.        if k == 3:
11.            # 将列表转换为集合，集合中的元素都是唯一的
12.            unique_elements = set(list0)
13.
14.            # 如果集合中的元素个数为1，则表示列表中的三个元素都相同
15.            if len(unique_elements) == 1:
16.                verifications = 1 # 当连续三次都是统一个手势，标志
                位归1，进入到动作循环中
17.                k, list0 = 0, [] # 归0，重新计数
18.            else:
19.                k, list0 = 0, []
20.            pass

```

代码的位于 `gesture_main.py` 中，解释如下所示：

初始化变量：

verifications：标志位，初始值为 0，用于判断是否连续三次检测到相同的手势。

list0：空列表，用于存储手势值。

k：计数器，初始值为 0，用于记录连续检测到相同手势的次数。

无限循环：

使用 `while True` 创建一个无限循环，这将不断执行循环内的代码。

调用图像推理函数：

调用 `Image_inference(model, labels_dict)` 函数进行图像推理，获取识别结果，并将其存储在 `ill_sets` 变量中。

打印识别结果：

使用 `print` 函数打印 `ill_sets` 的内容。

存储手势值：

如果 `ill_sets` 不为空,将第一个识别结果的类别 ID (`ill_sets[0][1]`)添加到 `list0` 列表中。

递增计数器:

将计数器 `k` 递增。

判断是否连续三次相同手势:

当计数器 `k` 等于 3 时, 执行以下操作:

将 `list0` 转换为集合 `unique_elements`, 以去除重复元素并检查元素是否唯一。

如果 `unique_elements` 的长度为 1, 说明三次手势相同, 将 `verifications` 设置为 1, 表示可以执行机器狗的动作。

重置计数器 `k` 和列表 `list0` 为初始状态。

重置逻辑:

如果 `unique_elements` 的长度不为 1, 即三次手势不相同, 重置 `k` 和 `list0` 但不改变 `verifications` 的值, 继续循环以重新计数和检测。

动作执行循环:

当 `verifications` 为 1 时, 根据机器狗识别到的手势, 进入动作执行循环中做出相应的动作。

```
1. while verifications:
2.     for ill_set in ill_sets:
3.         # 只执行接受到的第一个动作手势, 不接受其他的同时执行两个
4.         .....
```

上面的循环就是动作执行的循环

8.12.动作执行一趴下

```
1.
2. if ill_set[1] == 2: # 'fist' 趴下 ACTION_STAND_DOWN
```

```

3.             if status_listener() == [6, 0, 0]: # 监听
               当前是否是站立状态，站立状态满足趴下动作
4.             action_CommandHead.code = dog_actions
               .get("ACTION_STAND_DOWN")
5.             SendToCommand.perform_action(sfd, tar
               get_address, action_CommandHead.code, action_CommandHead.paramete
               rs_size,action_CommandHead.type_)
6.
7.             while 1:
8.                 if status_listener() == [1, 0, 0]
               : # 监听当前是否是趴下动作
9.                     print("趴下成功！")
10.                    time.sleep(5)
11.                    SendToCommand.perform_action(
               sfd, target_address, 0x21010202, 0, 0)
12.                    verifications = 0 # 动作结束，
               跳出循环，重新开始识别
13.                    break

```

条件判断

if ill_set[1]==2: 这一行检查 ill_set 列表中索引为 1 的元素是否等于 2。如果是，表示识别到的手势是“fist”，即用户做出了一个特定的手势，设备应该执行趴下的动作。

状态监听

if status_listener() == [6, 0, 0]: 这一行调用 status_listener()函数，该函数返回设备当前的状态。这里期望的状态是[6, 0, 0]，这表示设备处于站立状态。只有当设备处于站立状态时，才能执行趴下的动作。

动作命令设置

action_CommandHead.code = dog_actions.get("ACTION_STAND_DOWN") 这一行从 dog_actions 字典中获取与“趴下”动作对应的代码，并将其赋值给

`action_CommandHead.code`。`dog_actions` 是一个预定义的字典，存储了动作名称和它们对应的代码。

发送动作命令

`SendToCommand.perform_action(sfd, target_address, action_CommandHead.code, action_CommandHead.parameters_size, action_CommandHead.type_)` 这一行调用 `SendToCommand` 类的 `perform_action` 方法，发送动作命令到设备。这个方法需要几个参数：

`sfd`: 是一个 `socket` 文件描述符，用于与设备通信。

`target_address`: 设备的地址。

`action_CommandHead.code`: 动作代码。

`action_CommandHead.parameters_size`: 参数大小，表示动作命令中参数的数量或大小。

`action_CommandHead.type_`: 动作类型。

循环监听动作执行

`while 1`: 这是一个无限循环，用于持续检查设备是否执行了趴下动作。

动作完成判断

`if status_listener() == [1, 0, 0]`: 在循环内部，再次调用 `status_listener()` 函数来检查设备的状态。如果返回的状态是 `[1, 0, 0]`，这表示设备已经执行了趴下的动作。

动作成功反馈

`print("趴下成功！")` 如果设备执行了趴下动作，打印一条成功消息。

暂停和发送结束命令

`time.sleep(5)` 打印成功消息后，程序会暂停 5 秒。这是为了确保设备完成趴下动作，或者等待设备处理命令。

`SendToCommand.perform_action(sfd, target_address, 0x21010202, 0, 0)` 然后再次调用 `perform_action` 方法，发送一个特定的命令（这里是一个十六进制数 `0x21010202`），用于告诉设备趴下动作已经完成或需要执行其他操作。

循环退出

`verifications = 0` 设置 `verifications` 变量为 0，这是用于控制循环退出的条件。在这个上下文中，它表示动作已经完成，循环应该结束。

跳出循环

`break` 这个关键字用于跳出当前的无限循环。

8.13.动作执行—扭身跳

```
1.             elif ill_set[1] == 7: # 'one' 扭身跳
                ACTION_TWIST_JUMP
2.             if status_listener() == [6, 0, 0]: # 监听
                当前是否是站立状态，站立状态满足扭身跳动作
3.                 action_CommandHead.code = dog_actions
                .get("ACTION_TWIST_JUMP")
4.                 SendToCommand.perform_action(sfd, tar
                get_address, action_CommandHead.code, action_CommandHead.paramete
                rs_size,action_CommandHead.type_)
5.                 while 1:
6.                     if status_listener() == [6, 0, 4]
                        :# 监听当前是否是扭身跳动作
7.                         print("扭身跳! ")
8.                         verifications = 0 # 动作结束，
                        跳出循环，重新开始识别
9.                         break
```

条件判断

`elif ill_set[1] == 7`: 这一行是 `if` 语句的一个 `elif`（即“else if”）分支。它检查 `ill_set` 列表中索引为 1 的元素是否等于 7，如果是，表示识别到的手势是“one”，设备应该执行扭身跳的动作。

状态监听

`if status_listener() == [6, 0, 0]:` 这一行调用 `status_listener()` 函数来检查设备是否处于站立状态。站立状态（用 `[6, 0, 0]` 表示）是执行扭身跳动作的前提条件。

动作命令设置

`action_CommandHead.code = dog_actions.get("ACTION_TWIST_JUMP")` 这一行从 `dog_actions` 字典中获取与“扭身跳”动作对应的代码，并将其赋值给 `action_CommandHead.code` 属性。

发送动作命令

`SendToCommand.perform_action(sfd, target_address, action_CommandHead.code, action_CommandHead.parameters_size, action_CommandHead.type_)` 这一行调用 `SendToCommand` 类的 `perform_action` 方法，向设备发送执行扭身跳动作的命令。

循环监听动作执行

`while 1:` 这是一个无限循环，用于持续检查设备是否执行了扭身跳动作。

动作完成判断

`if status_listener() == [6, 0, 4]:` 在循环内部，再次调用 `status_listener()` 函数来检查设备的状态。如果返回的状态是 `[6, 0, 4]`，这表示设备已经执行了扭身跳动作。

动作成功反馈

`print("扭身跳！")` 如果设备执行了扭身跳动作，打印一条成功消息。

循环退出

`verifications = 0` 设置 `verifications` 变量为 0，这通常表示动作已经完成，循环应该结束。

跳出循环

`break` 这个关键字用于跳出当前的无限循环。

这段代码的逻辑与前面解释的“趴下”动作的代码块非常相似。主要区别在于：

识别到的手势编号不同（这里是 7，代表“one”手势）。

对应的动作代码不同（这里是“扭身跳”动作）。

状态监听函数 `status_listener()` 期望返回的状态值不同（这里是 `[6, 0, 4]`，表示扭身跳动作）。

8.14.动作执行—扭身体

```
1.         elif ill_set[1] == 9: # 'peace' 扭身体
           ACTION_TWIST_BODY
2.         if status_listener() == [6, 0, 0]: # 监听
           当前是否是站立状态，站立状态满足扭身体动作
3.         action_CommandHead.code = dog_actions
           .get("ACTION_TWIST_BODY")
4.         SendToCommand.perform_action(sfd, tar
           get_address, action_CommandHead.code, action_CommandHead.paramete
           rs_size, action_CommandHead.type_)
5.         while 1:
6.             if status_listener() == [6, 0, 2]
               :# 监听当前是否是扭身体动作
7.                 print("扭身体! ")
8.                 verifications = 0 # 动作结束,
               跳出循环，重新开始识别
9.                 break
```

条件判断

`elif ill_set[1] == 9:` 这是一个 `elif`（`else if`）分支，它在前一个 `if` 或 `elif` 条件不满足时检查 `ill_set` 列表中索引为 1 的元素是否等于 9。如果是，表示识别到的手势是“peace”，设备应该执行扭身体的动。

状态监听

`if status_listener() == [6, 0, 0]`: 这一行调用 `status_listener()`函数来检查设备是否处于站立状态。站立状态（用`[6, 0, 0]`表示）是执行扭身体动作的前提条件。

动作命令设置

`action_CommandHead.code = dog_actions.get("ACTION_TWIST_BODY")` 这一行从 `dog_actions` 字典中获取与“扭身体”动作对应的代码，并将其赋值给 `action_CommandHead.code` 属性。

发送动作命令

`SendToCommand.perform_action(sfd, target_address, action_CommandHead.code, action_CommandHead.parameters_size, action_CommandHead.type_)` 这一行调用 `SendToCommand` 类的 `perform_action` 方法，向设备发送执行扭身体动作的命令。

循环监听动作执行

`while 1`: 这是一个无限循环，用于持续检查设备是否执行了扭身体动作。

动作完成判断

`if status_listener() == [6, 0, 2]`: 在循环内部，再次调用 `status_listener()`函数来检查设备的状态。如果返回的状态是`[6, 0, 2]`，这表示设备已经执行了扭身体动作。

动作成功反馈

`print("扭身体！")` 如果设备执行了扭身体动作，打印一条成功消息。

循环退出

`verifications = 0` 设置 `verifications` 变量为 0，这通常表示动作已经完成，循环应该结束。

跳出循环

`break` 这个关键字用于跳出当前的无限循环。

这段代码的逻辑与之前解释的“趴下”和“扭身跳”动作的代码块非常相似。主要区别在于：

识别到的手势编号不同（这里是 9，代表“peace”手势）。

对应的动作代码不同（这里是“扭身体”动作）。

状态监听函数 `status_listener()` 期望返回的状态值不同（这里是 `[6, 0, 2]`，表示扭身体动作）。

8.15. 动作执行—太空步

```
1.             elif ill_set[1] == 10: # 'rock' 太空步
                ACTION_MOONWALK
2.             if status_listener() == [6, 0, 0]: # 监听
                当前是否是站立状态，站立状态满足太空步动作
3.                 action_CommandHead.code = dog_actions
                .get("ACTION_MOONWALK")
4.                 SendToCommand.perform_action(sfd, tar
                get_address, action_CommandHead.code, action_CommandHead.paramete
                rs_size, action_CommandHead.type_)
5.                 while 1:
6.                     if status_listener() == [6, 12, 1
                ]: # 监听当前是否是扭身跳动作
7.                         print("太空步")
8.                         time.sleep(4)
9.                         SendToCommand.perform_action(
                sfd, target_address, 0x21010202, 0, 0)
10.                    verifications = 0 # 动作结束，
                跳出循环，重新开始识别
11.                    break
```

条件判断

`elif ill_set[1] == 10`: 这是另一个 `elif` (else if) 分支，它在前一个 `if` 或 `elif` 条件不满足时检查 `ill_set` 列表中索引为 1 的元素是否等于 10。如果是，表示识别到的手势是“rock”，设备应该执行太空步的动作。

状态监听

`if status_listener() == [6, 0, 0]:` 这一行调用 `status_listener()` 函数来检查设备是否处于站立状态。站立状态（用 `[6, 0, 0]` 表示）是执行太空步动作的前提条件。

动作命令设置

`action_CommandHead.code = dog_actions.get("ACTION_MOONWALK")` 这一行从 `dog_actions` 字典中获取与“太空步”动作对应的代码，并将其赋值给 `action_CommandHead.code` 属性。

发送动作命令

`SendToCommand.perform_action(sfd, target_address, action_CommandHead.code, action_CommandHead.parameters_size, action_CommandHead.type_)` 这一行调用 `SendToCommand` 类的 `perform_action` 方法，向设备发送执行太空步动作的命令。

循环监听动作执行

`while 1:` 这是一个无限循环，用于持续检查设备是否执行了太空步动作。

动作完成判断

`if status_listener() == [6, 12, 1]:` 在循环内部，再次调用 `status_listener()` 函数来检查设备的状态。如果返回的状态是 `[6, 12, 1]`，这表示设备已经执行了太空步动作。

动作成功反馈

`print("太空步")` 如果设备执行了太空步动作，打印一条成功消息。

暂停和发送结束命令

`time.sleep(4)` 在打印成功消息后，程序会暂停 4 秒。这是为了确保设备完成太空步动作，或者等待设备处理命令。

`SendToCommand.perform_action(sfd, target_address, 0x21010202, 0, 0)` 然后再次调用 `perform_action` 方法，发送一个特定的命令（这里是一个十六进制数 `0x21010202`），用于告诉设备太空步动作已经完成或需要执行其他操作。

循环退出

`verifications = 0` 设置 `verifications` 变量为 0，这通常表示动作已经完成，循环应该结束。

跳出循环

`break` 这个关键字用于跳出当前的无限循环。

这段代码的逻辑与之前解释的“趴下”、“扭身跳”和“扭身体”动作的代码块非常相似。主要区别在于：

识别到的手势编号不同（这里是 10，代表“rock”手势）。

对应的动作代码不同（这里是“太空步”动作）。

状态监听函数 `status_listener()` 期望返回的状态值不同（这里是 [6, 12, 1]，表示太空步动作）。

暂停时间不同（这里是 4 秒）。

8.16. 动作执行—打招呼

```
1.             elif ill_set[1] == 16: # 'three2' 打招呼
                ACTION_GREET
2.             if status_listener() == [6, 0, 0]: # 监听
                当前是否是站立状态，站立状态满足打招呼动作
3.                 action_CommandHead.code = dog_actions
                .get("ACTION_GREET")
4.                 SendToCommand.perform_action(sfd, tar
                get_address, action_CommandHead.code, action_CommandHead.paramete
                rs_size, action_CommandHead.type_)
5.                 while 1:
6.                     if status_listener() == [20, 0, 0
                ]: # 监听当前是否是打招呼动作
7.                         print("打招呼")
```

```
8.                                     verifications = 0 # 动作结束,  
                                     跳出循环, 重新开始识别  
9.                                     break
```

条件判断

`elif ill_set[1] == 16`: 这是另一个 `elif` (else if) 分支, 它在前一个 `if` 或 `elif` 条件不满足时检查 `ill_set` 列表中索引为 1 的元素是否等于 16。如果是, 表示识别到的手势是 “three2”, 设备应该执行打招呼的动作。

状态监听

`if status_listener() == [6, 0, 0]`: 这一行调用 `status_listener()` 函数来检查设备是否处于站立状态。站立状态 (用 `[6, 0, 0]` 表示) 是执行打招呼动作的前提条件。

动作命令设置

`action_CommandHead.code = dog_actions.get("ACTION_GREET")` 这一行从 `dog_actions` 字典中获取与 “打招呼” 动作对应的代码, 并将其赋值给 `action_CommandHead.code` 属性。

发送动作命令

`SendToCommand.perform_action(sfd, target_address, action_CommandHead.code, action_CommandHead.parameters_size, action_CommandHead.type_)` 这一行调用 `SendToCommand` 类的 `perform_action` 方法, 向设备发送执行打招呼动作的命令。

循环监听动作执行

`while 1`: 这是一个无限循环, 用于持续检查设备是否执行了打招呼动作。

动作完成判断

`if status_listener() == [20, 0, 0]`: 在循环内部, 再次调用 `status_listener()` 函数来检查设备的状态。如果返回的状态是 `[20, 0, 0]`, 这表示设备已经执行了打招呼动作。

动作成功反馈

`print("打招呼")` 如果设备执行了打招呼动作, 打印一条成功消息。

循环退出

`verifications = 0` 设置 `verifications` 变量为 0，这通常表示动作已经完成，循环结束。

跳出循环

`break` 这个关键字用于跳出当前的无限循环。

8.17.异常处理

```
1.
                                     elif ill_set[1] not in [2, 7, 9, 10, 16]:
2.                                     logging.info("手势不在动作集合中，请重新做动
                                     作")
3.                                     verifications = 0 # 动作结束，跳出循环，重
                                     新开始识别
```

条件判断

`elif ill_set[1] not in [2, 7, 9, 10, 16]:` 这是另一个 `elif`（else if）分支，它在前一个 `if` 或 `elif` 条件不满足时检查 `ill_set` 列表中索引为 1 的元素是否不在列表 `[2, 7, 9, 10, 16]` 中。这些数字代表特定的手势识别结果，每个数字对应一个特定的动作。如果 `ill_set[1]` 的值不在这些值中，表示识别到的手势不是预期的动作之一。

日志记录

`logging.info("手势不在动作集合中，请重新做动作")` 这一行使用 `logging` 模块记录一条信息日志。这条日志告诉用户，当前识别到的手势不在预定义的动作集合中，需要重新做出一个有效的手势。

循环退出

`verifications = 0` 这一行将 `verifications` 变量设置为 0。在代码的其他部分，这个变量用于控制循环的退出条件。当 `verifications` 被设置为 0 时，如果存在一个外部循环依赖于这个变量的值，那么这个循环会退出，允许程序重新开始手势识别过程。

代码逻辑

这段代码是整个手势识别流程的“清理”或“重置”部分。它确保如果程序识别到一个

无效的手势，它会通知用户并准备重新开始手势识别过程，而不是继续执行错误的或不期望的动作。

第九章 运行示例

首先在文件夹的目录下执行先执行这个指令

```
. /usr/local/Ascend/mxVision-5.0.RC3/set_env.sh && . /usr/local/Ascend/ascend-toolkit/set_env.sh
```

然后再输入

```
python gesture_main.py
```

在对机器狗做出相对应的手势即可。

```
(base) root@davinci-mini:/opt/lab/GestureRecognition# . /usr/local/Ascend/mxVision-5.0.RC3/set_env.sh && . /usr/local/Ascend/ascend-toolkit/set_env.sh
(base) root@davinci-mini:/opt/lab/GestureRecognition# python gesture_main.py
Begin to initialize log.
The output directory of logs file exist.
WARNING: Logging before InitGoogleLogging() is written to STDERR
I20240703 14:55:57.105859 62864 FileUtils.cpp:331] The input file is empty
I20240703 14:55:57.105924 62864 FileUtils.cpp:475] Check Other group permission: Current permission is 4, but required no greater than 0.
Save logs information to specified directory.
Find 1 devices!

u3v device: [0]
device model name: MV-C5060-10UC-PRO
user serial number: DA0275184
/usr/local/miniconda3/lib/python3.9/site-packages/torchvision/io/image.py:13: UserWarning: Failed to load image Python extension:
warn(f"Failed to load image Python extension: {e}")
成功连接到ip: 192.168.1.100, port: 43897
Starting HeartbeatThread
INFO:root:0 three2
ill_sets: [[0, 16, 0.89]]
INFO:root:0 three2
ill_sets: [[0, 16, 0.9]]
INFO:root:0 three2
ill_sets: [[0, 16, 0.93]]
打招呼
```